A project report on

MACROGRAPH AI

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning)

by

HARIPRASAADH K (23BAI1061)



SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

November, 2026

MACROGRAPH AI

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning)

by

HARIPRASAADH K (23BAI1061)



VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

November, 2026

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

DECLARATION

I hereby declare that the thesis entitled "MACROGRAPH AI" submitted by HARIPRASAADH K (23BAI1061), for the award of the degree of Bachelor of Technology in Computer Science and Engineering, Vellore Institute of Technology, Chennai is a record of bonafide work carried out by me under the supervision of [GUIDE NAME].

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Chennai

Date: **Signature of the Candidate**

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

School of Computer Science and Engineering

CERTIFICATE

This is to certify that the report entitled "MACROGRAPH AI" is prepared and submitted by HARIPRASAADH K (23BAI1061) to Vellore Institute of Technology, Chennai, in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning) is a bonafide record carried out under my guidance. The project fulfills the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Signature of the Guide:

Name: [GUIDE NAME]

Date:

Signature of the Examiner	Signature of the Examiner
Name:	Name:
Date:	Date:

Approved by the Head of Department,
Computer Science and Engineering
(Artificial Intelligence and Machine Learning)

Name: Dr. Tulasi Prasad Sariki

Date:

(Seal of School of Computer Science and Engineering)

ABSTRACT

Anyone who studies the Indian economy has to work with statistics from many agencies, each with its own release calendar, file format and definition of the reference period. Large language models can turn such data into readable commentary, but they also produce figures that never appeared in any official release. In economic analysis a single invented number can reverse a conclusion, so output of that kind cannot be used. This project builds Macrograph AI, a multi-agent platform for Indian macroeconomic analysis in which every reported number has to be traced back to an official source.

The economy is split across ten sector agents covering the real sector, agriculture, prices, monetary conditions, finance, capital markets, fiscal affairs, the external sector, labour and services. An agent owns a fixed set of indicators and reads them only through its own FastMCP data server. If the live source fails, the server falls back to a cached snapshot, and if that is missing too it returns an explicit unavailable status. Agents talk to each other over a typed agent-to-agent protocol written for this project. The protocol allows only seven declared sector-to-sector exchanges, blocks delegation loops and rejects any successful response that arrives without source provenance. A four-node LangGraph orchestrator selects agents from their agent cards, gathers their observations, looks up transmission paths in a causal knowledge graph of twenty indicators and twelve typed edges, optionally simulates a shock, and writes a report in which each observation shows its period, status and provenance hash. Around this core sit an edge evidence audit, an anomaly radar, trajectory and twin views, a simulated monetary policy committee, a daily brief and a claim verification library that recomputes numerical claims against content-addressed evidence. The web client offers a cited dashboard, streaming chat, an agent registry and a causal laboratory.

The code base has about 37,000 lines of Python (tests excluded) and 11,300 lines of TypeScript, with 455 test functions; a full run passed 550 of 551 executed cases. In an end-to-end run, a cross-sector question on crude oil was answered from live official data in about eleven seconds. The platform's own audit could not confirm any of the twelve causal edges from the series currently available, and the scenario engine is a heuristic rather than an estimated model, so both are reported as assumptions and

simulations. The next steps are to place claim verification in the answer path, estimate the causal edges from long series and compare the system against baselines.

ACKNOWLEDGEMENT

I express my deep sense of gratitude to [GUIDE NAME], [DESIGNATION], School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for constant guidance and encouragement throughout this project, and above all for teaching me patience in my work. Working under an expert in the field of [AREA] has been a valuable opportunity that went well beyond the academic requirements of the course.

I thank Dr. G. Viswanathan, Honourable Chancellor; Mr. Sankar Viswanathan, Dr. Sekar Viswanathan and Dr. G. V. Selvam, Vice Presidents; Dr. Sandhya Pentareddy, Executive Director; Ms. Kadhambari S. Viswanathan, Assistant Vice-President; Dr. V. S. Kanchana Bhaaskaran, Vice-Chancellor; Dr. T. Thyagarajan, Pro-Vice Chancellor, VIT Chennai; and Dr. P. K. Manoharan, Additional Registrar, for providing an excellent working environment during the course.

I am grateful to Dr. Viswanathan V, Dean, and to Dr. Nithyanandam P, Dr. Suganya G and Dr. Sweetlin Hemalatha C, Associate Deans, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for their time and support.

I sincerely thank Dr. Tulasi Prasad Sariki, Head of the Department, B.Tech. Computer Science and Engineering (Artificial Intelligence and Machine Learning), and the Project Coordinators for their support and encouragement in taking up and completing this work.

I thank all the faculty and staff of Vellore Institute of Technology, Chennai, who helped me acquire the knowledge this project needed, and my fellow students Arulprasaad and Nithish Kumar, with whom I discussed and built parts of the code base. Finally, I thank my parents and friends for their support and encouragement.

Place: Chennai

Date: **Hariprasaadh K**

CONTENTS

CONTENTS vi

LIST OF FIGURES ix

LIST OF TABLES xi

LIST OF ACRONYMS xii

CHAPTER 1
INTRODUCTION

1.1 INTRODUCTION 1

1.2 OVERVIEW OF MACROECONOMIC INTELLIGENCE SYSTEMS 2

1.3 CHALLENGES PRESENT IN AUTOMATED MACROECONOMIC ANALYSIS 2

1.4 PROJECT STATEMENT 3

1.5 OBJECTIVES 3

1.6 SCOPE OF THE PROJECT 4

1.7 CONTRIBUTIONS OF THE WORK 5

1.8 ORGANISATION OF THE REPORT 6

CHAPTER 2
BACKGROUND

2.1 INTRODUCTION 8

2.2 SURVEY ON LLM-BASED AGENTS FOR ECONOMICS AND FINANCE 8

2.3 RETRIEVAL-AUGMENTED GENERATION IN FINANCE 9

2.4 KNOWLEDGE GRAPHS FOR MACROECONOMICS 10

2.5 AGENT INTEROPERABILITY PROTOCOLS 11

2.6 CAUSALITY AND SCENARIO ANALYSIS IN MACROECONOMICS 12

2.7 TRUSTWORTHY GENERATION AND HALLUCINATION 13

2.8 COMPARATIVE SUMMARY OF RELATED WORK 13

2.9 RESEARCH GAPS AND MOTIVATION 14

2.10 TECHNOLOGIES USED 14

CHAPTER 3
SYSTEM REQUIREMENTS AND DESIGN

3.1 INTRODUCTION 16

3.2 REQUIREMENTS ANALYSIS 16

3.3 SYSTEM CONTEXT 17

3.4 LAYERED ARCHITECTURE 18

3.5 DESIGN OF A SECTOR AGENT 19

3.6 INDICATOR OWNERSHIP AND DEPENDENCIES 20

3.7 DATA MODEL 21
3.8 DESIGN DECISIONS AND RATIONALE 22

CHAPTER 4
PROTOCOL DESIGN

4.1 INTRODUCTION 24
4.2 THE AGENT-TO-AGENT LAYER 24
4.3 AGENT CARDS, REGISTRY AND ROUTING 28
4.4 CROSS-SECTOR EXCHANGES 28
4.5 THE MCP DATA LAYER 29
4.6 PROVENANCE ACROSS THE PROTOCOLS 30
4.7 TRACEABILITY 31

CHAPTER 5
IMPLEMENTATION

5.1 INTRODUCTION 32
5.2 DEVELOPMENT ENVIRONMENT 32
5.3 THE ORCHESTRATOR 32
5.4 CHAT INTERFACE AND ROUTING 35
5.5 SECTOR AGENTS AND DATA ACQUISITION 35
5.6 STORAGE 37
5.7 ROBUSTNESS AND OFFLINE OPERATION 38
5.8 CODE ORGANISATION AND STATISTICS 38

CHAPTER 6
KNOWLEDGE GRAPH AND SCENARIO ENGINE

6.1 INTRODUCTION 39
6.2 ONTOLOGY OF INDICATORS AND TRANSMISSION EDGES 39
6.3 GRAPH ENGINE AND QUERIES 41
6.4 SCENARIO ENGINE 42
6.5 WORKED EXAMPLE: CRUDE OIL SHOCK 43

CHAPTER 7
RESEARCH LAB, DAILY BRIEF AND CLAIM VERIFICATION

7.1 INTRODUCTION 45
7.2 SHOCKWAVE: MONTHLY TRAJECTORIES 45
7.3 TWIN: BASE WORLD VERSUS SHOCKED WORLD 45
7.4 ANOMALY RADAR 46
7.5 EDGE EVIDENCE AUDIT 46
7.6 SIMULATED MONETARY POLICY COMMITTEE 48
7.7 CONSENSUS AND MODEL CARD 50
7.8 DAILY EARLY-WARNING BRIEF 51
7.9 CLAIM VERIFICATION 51

CHAPTER 8
USER INTERFACE AND API

8.1 INTRODUCTION 54
8.2 FRONTEND ARCHITECTURE 54
8.3 LANDING PAGE AND AUTHENTICATION 55
8.4 DASHBOARD 56
8.5 AGENT RESEARCH WORKSPACE 57
8.6 A2A AGENT REGISTRY 58
8.7 CAUSAL LAB 59
8.8 HTTP INTERFACE 61

CHAPTER 9
TESTING, RESULTS AND DISCUSSION

9.1 INTRODUCTION 62
9.2 TESTING STRATEGY 62
9.3 TEST RESULTS 62
9.4 END-TO-END ORCHESTRATOR RUN 64
9.5 EDGE EVIDENCE AUDIT RESULT 65
9.6 ANOMALY RADAR AND SIMULATED COMMITTEE 66
9.7 PLANNED EVALUATION 67
9.8 LIMITATIONS 67
9.9 LESSONS LEARNED DURING DEVELOPMENT 68
9.10 DISCUSSION 69

CHAPTER 10
CONCLUSION AND FUTURE WORK

10.1 SUMMARY OF THE WORK 70
10.2 ACHIEVEMENT OF THE OBJECTIVES 70
10.3 CONTRIBUTIONS AND INSIGHTS 71
10.4 FUTURE WORK 71
10.5 CONCLUDING REMARKS 73

REFERENCES 74
APPENDICES 77

APPENDIX 1 REPOSITORY STRUCTURE AND CONFIGURATION 78
APPENDIX 2 CANONICAL INDICATORS OF THE KNOWLEDGE GRAPH 79
APPENDIX 3 SAMPLE REPORT GENERATED BY THE ORCHESTRATOR 80
APPENDIX 4 SELECTED SOURCE CODE LISTINGS 82
APPENDIX 5 ERROR TAXONOMY AND OPEN ISSUES 85

LIST OF FIGURES

1.1 Mapping of identified problems to project objectives and implemented components. 4
1.2 Scope of the project: implemented components, prototypes and items that are out of scope. 5
2.1 Separation of the agent-to-agent reasoning channel (A2A) and the agent-to-data channel (MCP). 12
3.1 System context of Macrograph AI: users, external data sources and external services. 18
3.2 Layered architecture of Macrograph AI. 19
3.3 Standard internal structure of a sector agent package. 20
3.4 Indicator ownership and the seven declared sector-to-sector A2A dependencies. 21
3.5 Entity-relationship diagram of the canonical economic data layer. 22
4.1 Validation pipeline applied by the A2A server to every request. 26
4.2 Sequence of an A2A delegation in which a sector agent requests a peer indicator. 28
4.3 An agent retrieving data through its own FastMCP server, with the fallback order. 30
4.4 Propagation of provenance from the official source to the final report. 31
5.1 The four-node LangGraph orchestrator and the state keys written by each node. 33
5.2 Sections of the cited intelligence report produced by the synthesis node. 35
5.3 Data acquisition pipeline shared by the sector agents. 36
5.4 Primary data sources feeding each sector agent. 37
6.1 Causal transmission graph of 20 indicators connected by 12 typed edges (lag in months and sign). 40
6.2 Structure, storage and consumers of the knowledge graph. 42
6.3 Steps of shock propagation and the modules that use its output. 43
6.4 Simulated monthly change from baseline after the shock (step response, produced by the Shockwave module). 44
7.1 Research lab modules, API endpoints and user interface tabs. 45
7.2 Per-hop share of attenuation along the path from Brent crude to GDP growth (Twin module, real output). 46
7.3 Edge evidence audit: seeded priors tested against live series. 48
7.4 Simulation of a six-seat Monetary Policy Committee with a deterministic rule baseline. 50
7.5 Claim verification pipeline from retrieved evidence to rendered text and audit record. 52
7.6 Proposed integration of the claim verifier into orchestrator synthesis (not yet implemented). 53

8.1 React component hierarchy of the application. 55
8.2 Landing page of Macrograph AI. 56
8.3 Dashboard, all-sectors tab with ticker bar and early-warning brief panel. 56
8.4 Dashboard, finance and banking tab: credit expansion, asset quality gauges and lending rates. 57
8.5 Server-Sent Events protocol between the chat workspace and the gateway. 58
8.6 A2A agent registry view listing the Agent Cards of the sector agents. 59
8.7 Causal lab, path tab: transmission path from the repo rate to GDP growth. 60
8.8 Causal lab, Shockwave tab: animated propagation of a Brent crude shock. 60
8.9 Causal lab, MPC tab: simulated six-seat committee labelled as simulated. 61
9.1 Number of test functions per test area (455 functions). 64
9.2 Overlapping monthly observations available for each causal edge (minimum required: 10). 66
10.1 Target architecture after planned improvements. 73
A1.1 Repository layout of Macrograph AI. 78
A5.1 Taxonomy of A2A error codes. 85

LIST OF TABLES

1.1 Organisation of the report 7
1.2 Development phases derived from the repository history 7
2.1 Comparison of related work and Macrograph AI 13
2.2 Technologies used in Macrograph AI 15
3.1 Functional requirements 16
3.2 Non-functional requirements 17
3.3 Indicator ownership by sector agent 21
3.4 Principal design decisions and their rationale 23
4.1 Fields of the principal A2A messages 24
4.2 A2A error codes 27
4.3 Declared sector-to-sector A2A dependencies 29
5.1 Development and runtime environment 32
5.2 Source lines by component (tests excluded, 12 October 2026) 38
6.1 The twelve causal edges of the knowledge graph 41
6.2 Simulated transmission of a USD 20 per barrel Brent increase (default baselines) 43
8.1 Principal HTTP endpoints 61
9.1 Test functions by area (12 October 2026) 63
9.2 Observations collected in the end-to-end run 64
9.3 Result of the edge evidence audit 65
9.4 Votes of the simulated committee (rule-based mode) 67
10.1 Status of the project objectives 71
10.2 Proposed roadmap for future work 72
A1.1 Principal configuration variables 78
A2.1 Canonical indicators of the ontology 79
A3.1 Indicator matrix of the generated report 80
A3.2 Scenario table of the generated report (live baselines for CPI and repo rate) 80
A3.3 Source provenance table of the generated report 81
A4.1 Listing: A2A request and response models (core/protocols/a2a/messages.py) 82
A4.2 Listing: Shortest transmission path query (core/knowledge_graph/networkx_engine.py) 82
A4.3 Listing: Scenario propagation loop (core/econometrics/causal_engine.py) 83
A4.4 Listing: Rolling z-score of the anomaly radar (core/research/anomaly.py) 83
A5.1 Principal open issues from the code review 85

LIST OF ACRONYMS

A2A	Agent-to-Agent protocol
ADF	Augmented Dickey-Fuller test
API	Application Programming Interface
BH	Benjamini-Hochberg false discovery rate procedure
BoP	Balance of Payments
BVAR	Bayesian Vector Autoregression
CGA	Controller General of Accounts
CPI	Consumer Price Index
CSV	Comma-Separated Values
DBIE	Database on Indian Economy (Reserve Bank of India)
DFD	Data Flow Diagram
EPFO	Employees' Provident Fund Organisation
GDP	Gross Domestic Product
GFCF	Gross Fixed Capital Formation
GST	Goods and Services Tax
GVA	Gross Value Added
HTTP	Hypertext Transfer Protocol
IIP	Index of Industrial Production
IMF	International Monetary Fund
JSON	JavaScript Object Notation
KG	Knowledge Graph
LLM	Large Language Model
LPA	Long Period Average (rainfall)
MCP	Model Context Protocol
MoSPI	Ministry of Statistics and Programme Implementation
MPC	Monetary Policy Committee
NPA	Non-Performing Asset
PLFS	Periodic Labour Force Survey
PMI	Purchasing Managers' Index
RAG	Retrieval-Augmented Generation
RBI	Reserve Bank of India
SDF, MSF	Standing Deposit Facility, Marginal Standing Facility
SQL	Structured Query Language
SSE	Server-Sent Events
UI	User Interface
VAR	Vector Autoregression
WALR	Weighted Average Lending Rate
WPI	Wholesale Price Index
YoY	Year on Year

Chapter 1
Introduction

1.1 INTRODUCTION

Suppose a policy analyst wants to know what a rise in crude oil prices will do to the Indian economy. Inflation data come from the Ministry of Statistics and Programme Implementation (MoSPI). Policy rates and credit figures come from the Reserve Bank of India (RBI), fiscal accounts from the Controller General of Accounts, trade data from the Directorate General of Commercial Intelligence and Statistics, and market prices from the stock exchanges. These agencies do not share a format, a release frequency or even a common idea of what period a number refers to. Before the analyst can write one sentence of interpretation, a good deal of time goes into collecting, cleaning and lining up the series.

Large language models (LLMs) can now write economic commentary on request, and the commentary usually reads well. Reading well is a different property from being right. Ask a model for the latest Consumer Price Index (CPI) reading and it may return a believable figure that no official release contains. The problem of unsupported output is well documented (Ji et al., 2023). In economics it is especially costly, because one wrong number can flip the direction of an argument. Retrieval-augmented generation (Lewis et al., 2020) helps by putting retrieved text in front of the model, yet the model still types out the numbers itself and nothing checks that it copied them correctly.

Macrograph AI approaches the problem as a systems question rather than a prompting question. The Indian economy is divided among ten sector agents. Each agent owns a set of indicators and may read them only from official sources, through a data server that belongs to that agent alone. Agents cooperate over a typed agent-to-agent protocol, a LangGraph orchestrator coordinates them, and a causal knowledge graph links the indicators. In the final report, every number carries its source, observation period and a provenance hash. If a source cannot be reached, the report says the value is unavailable; it does not fill the gap with an estimate.

1.2 OVERVIEW OF MACROECONOMIC INTELLIGENCE SYSTEMS

By a macroeconomic intelligence system I mean software that gathers macroeconomic observations, relates them to one another and presents conclusions to a person who has to make a decision. Until recently such systems were either dashboards or econometric toolkits. Dashboards display series from one or two sources and leave the interpretation to the reader. Toolkits can estimate models such as vector autoregressions, but the analyst must prepare the data and specify the model first. Neither kind of tool can take a free-form question, for instance how an oil shock feeds into inflation and the policy rate, and return an answer.

LLM-based agents have recently been applied to economic and financial problems. Hao and Xie (2025) use them to simulate heterogeneous households responding to policy, Lin et al. (2026) to simulate survey respondents forming macroeconomic expectations, and Xiao et al. (2025) to reproduce the structure of a trading firm with analyst, researcher and trader roles. Separately, knowledge graphs of economic variables have been used to guide retrieval and forecasting (Yang et al., 2020; Zhu et al., 2025), and Kato (2026) grounds economic narratives in model computations that were actually executed. Chapter 2 discusses these papers in more depth.

Two open protocols make it easier to assemble such a system from independent parts. The Model Context Protocol (MCP) defines how an agent calls tools and reads data (Anthropic, 2024), and the Agent2Agent (A2A) protocol defines how agents find each other and hand over tasks (Google, 2025). Macrograph AI uses MCP for data access and its own typed A2A layer for cooperation between agents. The two are kept apart on purpose: reasoning may pass between agents, but data reaches an agent only from the agent that owns it.

1.3 CHALLENGES PRESENT IN AUTOMATED MACROECONOMIC ANALYSIS

Several problems had to be solved before an automated analyst for India could be trusted, and they shaped most of the design decisions described later.

The first is the state of the sources themselves. Official indicators arrive as APIs, spreadsheets, PDF bulletins and HTML tables, with different base years, units and release calendars. The second is the unsupported number: a model can state a value

found in no release, and because the value looks reasonable it is hard to catch afterwards. A third problem appears as soon as more than one agent is involved. If two agents fetch the same indicator on their own, they may pick up different vintages, and the final report then disagrees with itself.

Causality adds a further difficulty. Macroeconomic variables affect one another with delays and sometimes with opposite signs, and a system that ignores these channels can describe each series but cannot explain how a shock spreads. Provenance is also usually weak. A reader should be able to find out which table a number came from, which period it covers and when it was fetched, and narrative text almost never says. Government endpoints are sometimes slow or down, so the system has to keep working with partial data, say what is missing and resist inventing substitutes. Finally, simulated results and assumed relationships need to be labelled as such, since a heuristic projection presented as a measurement misleads the reader.

1.4 PROJECT STATEMENT

To design, implement and evaluate a multi-agent platform for Indian macroeconomic intelligence in which (i) specialised sector agents own their indicators and retrieve them from official sources, (ii) agents cooperate through a typed, auditable protocol that separates reasoning from data access, (iii) a causal knowledge graph and a transparent scenario engine relate indicators across sectors, and (iv) every claim in the generated report is traceable to a source, an observation period and a provenance record, with missing data reported as unavailable.

1.5 OBJECTIVES

Seven objectives followed from the problem statement. Figure 1.1 links each problem to its objective and to the component built to address it.

1. **O1. Sector-wise data access.** Provide one data-access server and one local store for each of ten macroeconomic sectors, backed by official sources.
2. **O2. Coordinated multi-agent reasoning.** Implement a typed agent-to-agent protocol with discovery, routing, loop protection and tracing, and an orchestrator that decomposes questions and merges answers.

3. **O3. Source-grounded answers.** Enforce at protocol level that successful responses carry source provenance, and return structured unavailable states instead of estimates.
4. **O4. Explicit causal model.** Build a knowledge graph of cross-sector transmission channels with lag, sign and evidence class, and use it to explain and simulate shocks.
5. **O5. Traceable provenance.** Attach source, table, period, retrieval time and a hash to every observation and record the delegation tree of every conversation.
6. **O6. Research instruments.** Provide tools that test the model against data (edge evidence audit, anomaly radar), explore it interactively (trajectory and twin views) and verify claims against evidence.
7. **O7. Usable interface.** Provide a web interface with a cited dashboard, streaming chat, causal lab and agent registry.

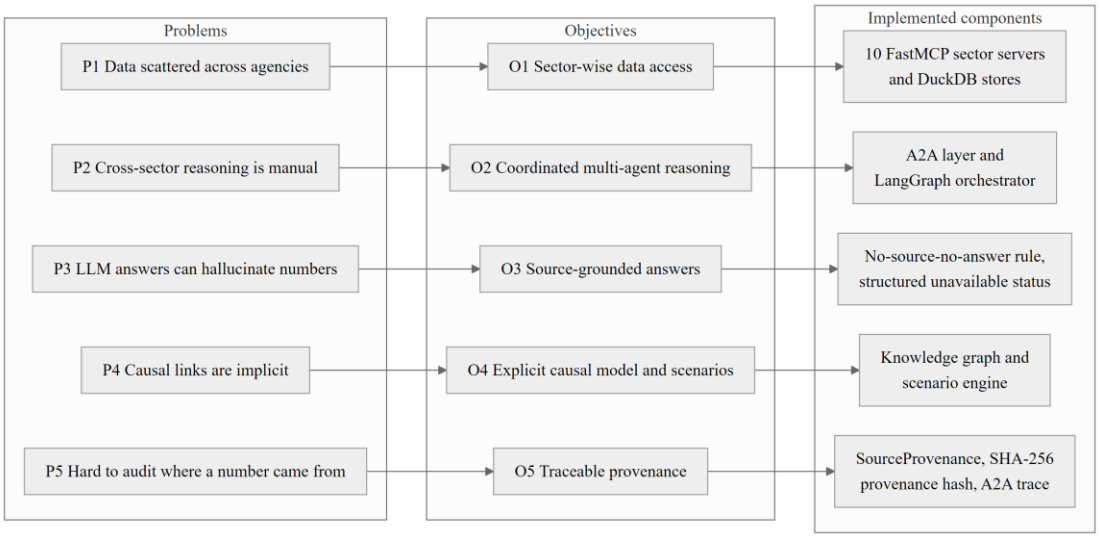


Figure 1.1: Mapping of identified problems to project objectives and implemented components.

1.6 SCOPE OF THE PROJECT

The platform covers ten sectors of the Indian economy: real sector, agriculture, prices and inflation, monetary conditions, finance and banking, capital markets, fiscal affairs, the external sector, labour and employment, and services. Each sector is a Python package with a data-access server, a local DuckDB store, an agent and an interface for the agent-to-agent protocol. The orchestrator, the knowledge graph, the scenario engine, the research lab, the daily brief and the web application complete the

system. Figure 1.2 separates what is implemented from what is a prototype and what is out of scope.

The scope has clear limits. The platform does not give investment advice, does not forecast with statistical guarantees and does not claim causal identification. Retrieval over policy documents (for example RBI minutes) is not implemented, so every report states that documentary evidence is unavailable. The claim verification library is implemented and tested but is not yet part of the answer path.

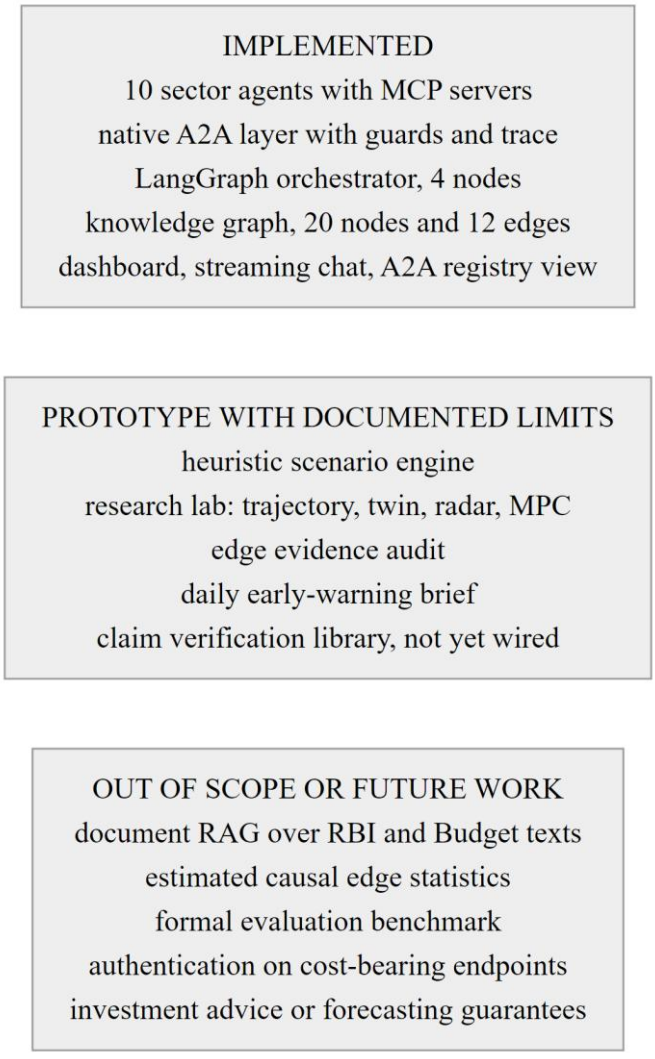


Figure 1.2: Scope of the project: implemented components, prototypes and items that are out of scope.

1.7 CONTRIBUTIONS OF THE WORK

The main contribution is a ten-agent architecture in which indicator ownership is enforced by the protocol. Only seven sector-to-sector exchanges are declared, and the server refuses all others. The agent-to-agent layer itself was written for this project; it

guards against excessive depth, too many hops, cycles and duplicate requests, returns structured error codes and records a trace of every conversation. Data access runs over MCP with a fixed fallback order from live data to a cached snapshot to an unavailable status.

On the analytical side, the project contributes a causal knowledge graph whose edges are graded on a five-tier evidence scale, together with a graph-attenuation scenario engine whose assumptions are written down and shown to the user. An edge evidence audit separates the priors I entered by hand from relationships that the data actually support, and an anomaly radar classifies unusual readings according to whether connected indicators moved at the same time. A claim verification library recomputes numerical claims with exact decimal arithmetic against content-addressed evidence, so model prose is never shown as fact without a check. All of this is exposed through a web application with a cited dashboard, streaming chat, an A2A registry and an interactive causal lab, and is covered by 455 test functions that execute as 551 cases.

1.8 ORGANISATION OF THE REPORT

The remaining chapters are summarised in Table 1.1. Development ran from July to October 2026; the phases in Table 1.2 are reconstructed from the commit history of the repository.

Table 1.1: Organisation of the report

Chapter	Title	Content
1	Introduction	Motivation, challenges, problem statement, objectives, scope, contributions.
2	Background	Survey of LLM agents, retrieval, knowledge graphs, protocols, causality and verification; research gaps.
3	System Requirements and Design	Requirements, architecture, data model and design decisions.
4	Protocol Design	Agent-to-agent protocol, MCP data layer, routing, loop protection and provenance.
5	Implementation	Orchestrator, sector agents, data acquisition and resilience.
6	Knowledge Graph and Scenario Engine	Ontology, graph queries, scenario algorithm and worked examples.
7	Research Lab and Verification	Trajectory, twin, anomaly radar, MPC, edge audit, daily brief, claim verification.
8	User Interface and API	Web application, streaming protocol and HTTP interface with screenshots.
9	Testing, Results and Discussion	Test strategy, measured results, evaluation plan and limitations.
10	Conclusion and Future Work	Summary, contributions and roadmap.

Table 1.2: Development phases derived from the repository history

Period (2026)	Phase	Main outcomes
16 to 25 July	Design and first agent	Product design document, literature papers, implementation plan, first real-sector agent.
6 to 7 August	Core protocols and sectors	A2A and MCP integration for the real sector; finance and capital-market sector agents.
23 August	Data layer and graph	FastMCP, streaming and DuckDB modernisation; canonical data layer, ingestion pipeline, first causal knowledge graph.
9 to 30 September	Foundations	Architecture documents, A2A foundation, landing page, refactoring.
2 to 6 October	Sector completion	Finance end to end and chat interface; monetary, external, capital-market, labour, real, agriculture, prices, services and fiscal sectors.
9 to 11 October	Integration and research	A2A in the orchestrator, dashboard charts, graph hardening, research lab, verification library, sector fixes.

Chapter 2
Background

2.1 INTRODUCTION

Macrograph AI draws on several bodies of work that rarely meet in one place: LLM agents applied to economics and finance, retrieval-augmented generation for financial documents, knowledge graphs of economic variables, protocols for agent interoperability, econometric methods for causality, and research on hallucination. Seven papers were collected as the base literature at the start of the project, and they are discussed below together with the classical references that the design depends on. The chapter ends with a comparison table, the gaps that motivated the project and the technologies chosen for the implementation.

2.2 SURVEY ON LLM-BASED AGENTS FOR ECONOMICS AND FINANCE

2.2.1 AGENT-BASED MODELLING WITH LANGUAGE MODELS

An agent, in the classical definition, is a computer system situated in an environment that perceives it and acts autonomously to meet its design objectives (Wooldridge & Jennings, 1995). Large language models have renewed interest in agents because a model can plan, call tools and communicate in natural language. Generative agents that combine a language model with memory and reflection have been shown to produce believable individual and social behaviour (Park et al., 2023), and the ReAct pattern interleaves reasoning traces with tool actions (Yao et al., 2023). The sector agents and the orchestrator in this project build directly on these ideas.

2.2.2 ECONOMIC AND POLICY ANALYSIS WITH MULTIPLE LANGUAGE MODELS

Hao and Xie (2025) propose a Multi-LLM-Agent-Based (MLAB) framework for economic and public policy analysis. They first evaluate five LLMs on two-period consumption allocation problems, with and without an explicit utility function, and then map the models to educational groups and income brackets so that differences in analytical capability stand in for heterogeneity among households. Interest-income taxation serves as the case study. The work shows that LLM agents can be used as heterogeneous economic actors, but it simulates stylised decision problems and does not retrieve observed macroeconomic data.

Lin et al. (2026) introduce LLM-based economic agents that simulate macroeconomic expectations in survey experiments. Each agent has modules that retrieve personal characteristics, prior expectations and dynamic external information. The framework is validated by recapitulating three representative survey designs, and the authors report that the generated expectation distributions are highly similar to human data and that priors matter a great deal for matching the distributions. For this project the useful lesson is that structured retrieval modules around the model improve its output, which is also how the sector agents of Macrograph AI are organised.

2.2.3 MULTI-AGENT FINANCIAL TRADING

TradingAgents (Xiao et al., 2025) organises language-model agents as a trading firm: fundamental, sentiment and technical analysts, Bull and Bear researchers who debate market conditions, a risk management team and traders with different risk profiles. The authors report gains in cumulative returns, Sharpe ratio and maximum drawdown over baseline models. The framework illustrates the benefit of role specialisation and structured debate. All roles, however, read from a shared data feed, and no mechanism makes a single agent responsible for a given data point.

2.2.4 THE AI ECONOMIST

The AI Economist of Zheng et al. (2022) uses two-level deep multi-agent reinforcement learning to design taxation policy. It addresses a different problem, optimal policy design inside a simulated economy, but it is a landmark for the use of multi-agent methods in economics. Kato (2026) uses a related name for a different system, discussed in Section 2.4.

2.3 RETRIEVAL-AUGMENTED GENERATION IN FINANCE

Retrieval-augmented generation (RAG) combines a parametric language model with a non-parametric memory that is searched at generation time (Lewis et al., 2020). Retrieved passages are added to the prompt so that the model can base its answer on documents. RAG has become the usual remedy for hallucination in knowledge-intensive tasks, although its quality depends strongly on how the documents are chunked, indexed and re-ranked.

Dadopoulos et al. (2025) study this question for financial filings. They benchmark a multi-stage metadata-driven RAG architecture on the FinanceBench dataset, combining pre-retrieval filtering, post-retrieval re-ranking and enriched embeddings.

They find that a powerful re-ranker is essential for precision, that the largest gains come from embedding chunk metadata together with text (contextual chunks), and that a custom metadata re-ranker is a cost-effective alternative to commercial re-rankers. Two points carry over to this project. The study shows that retrieval quality in finance depends on metadata, which in Macrograph AI means source, table, period and unit. It also targets document questions, whereas the macroeconomic questions addressed in this project are mostly answered from numerical series, which is why this project retrieves structured observations through tools.

2.4 KNOWLEDGE GRAPHS FOR MACROECONOMICS

A knowledge graph represents entities and typed relationships as a directed labelled graph (Hogan et al., 2021). In economics a node can be a variable and an edge can state that one variable influences another. Yang et al. (2020) construct a knowledge graph of macroeconomic variables, including alternative big-data variables, by applying natural language processing to academic literature and research reports. They use the graph as prior knowledge for variable selection in forecasting and report significantly higher forecasting accuracy than statistical variable selection, especially for long-run forecasts. My graph is far smaller and was built by hand, but it does the same job of encoding prior knowledge about transmission channels.

Zhu et al. (2025) propose KG2RAG, a Knowledge Graph-Guided RAG framework. After a semantic retrieval step provides seed chunks, the method expands them through the knowledge graph and organises the expanded chunks into coherent paragraphs. Experiments on HotpotQA and its variants show that the approach improves both response quality and retrieval quality, which suggests that graph structure adds fact-level links that separate chunks do not have.

The most closely related system is the AI economist agent of Kato (2026). It is a model-grounded, RAG-based agentic framework that uses a knowledge graph with typed nodes for text evidence, model specifications and model runs. The language model plans the analysis, retrieves evidence, selects a model and writes the report, but quantitative claims are not produced by the language model; they come from executed model computations recorded in the graph. The framework is evaluated on two applications (U.S. inflation persistence with Federal Reserve policy, and commercial real estate refinancing stress) and compares LLM-only, RAG-only and model-

grounded GraphRAG conditions. Its main finding is qualitative: once the generator is required to use retrieved evidence, graph paths and executed model output, a fluent narrative becomes an analysis that can be traced. Macrograph AI follows the same principle that numbers must not come from the language model, but reaches it by a different route. It grounds numbers in sector-owned official observations, enforces provenance in a multi-agent protocol, labels the evidence class of each causal edge and targets the Indian economy.

2.5 AGENT INTEROPERABILITY PROTOCOLS

The Model Context Protocol (MCP), introduced by Anthropic (2024), defines a client-server standard through which applications expose tools and data to language-model clients. Tool definitions have typed parameters, so an agent can discover and call them uniformly. In Macrograph AI every sector exposes its data through an MCP server built with the FastMCP library.

The Agent2Agent (A2A) protocol, announced by Google (2025), addresses a different problem: how autonomous agents built by different teams discover each other's capabilities and exchange tasks. It introduces agent cards that advertise skills and a task-oriented message exchange. Macrograph AI implements a native typed layer inspired by these concepts, adding guards for loop protection, mandatory source provenance and a declared dependency graph, and keeps A2A strictly for reasoning while MCP is strictly for data. Figure 2.1 contrasts the two channels.

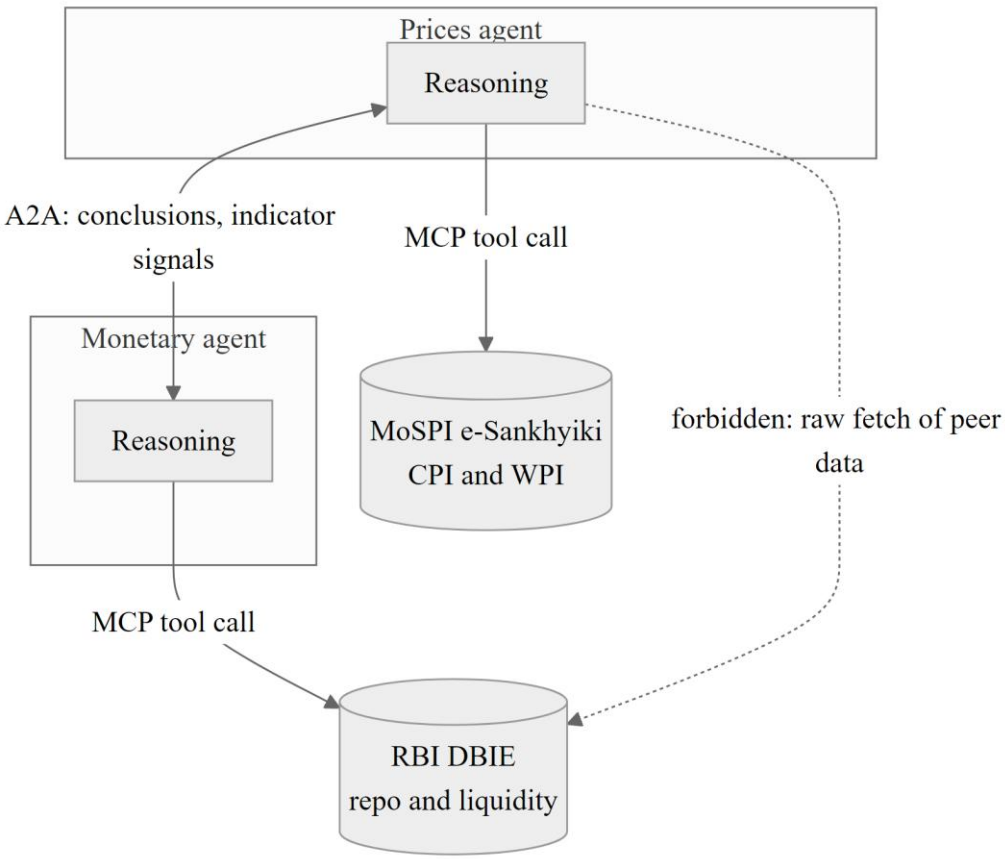


Figure 2.1: Separation of the agent-to-agent reasoning channel (A2A) and the agent-to-data channel (MCP).

2.6 CAUSALITY AND SCENARIO ANALYSIS IN MACROECONOMICS

Macroeconomists distinguish association from causation. Granger (1969) defined a predictive notion of causality: a variable X Granger-causes Y if past values of X improve the prediction of Y beyond the information in past values of Y. Sims (1980) introduced vector autoregressions as a way to study the dynamic effects of shocks without imposing a full structural model. Jordà (2005) proposed local projections, which estimate impulse responses horizon by horizon and are robust to misspecification of the dynamic system. Pearl (2009) formalised structural causal models and the conditions under which causal effects can be identified from data. Time-series tests also require care: series may need to be differenced when a unit root is present (Dickey & Fuller, 1979), and testing many relationships requires control of the false discovery rate (Benjamini & Hochberg, 1995).

These references are a useful yardstick for the scenario engine described in Chapter 6. Its propagation rule is a transparent heuristic over a hand-built graph and does not

estimate impulse responses, so the project reports its results as simulated and uses Granger tests only to audit whether edges are supported by the available data. Replacing the heuristic with local projections is the central item of future work.

2.7 TRUSTWORTHY GENERATION AND HALLUCINATION

Ji et al. (2023) survey hallucination in natural language generation and separate content that contradicts the source from content that the source cannot confirm. Applied to economic text, this means a generated figure has to be checked against the retrieved evidence. The literature describes three kinds of mitigation: grounding generation in retrieved text, constraining the model to call tools for facts, and checking generated claims after generation. Macrograph AI combines the first two in its agents and implements the third as a claim verification library that recomputes numerical claims and accepts qualitative claims only as exact source sentences.

2.8 COMPARATIVE SUMMARY OF RELATED WORK

Table 2.1 compares the reviewed systems along the dimensions that matter for this project.

Table 2.1: Comparison of related work and Macrograph AI

System	Approach	How numbers are grounded	Provenance and verification	Domain
Hao & Xie (2025)	Heterogeneous LLM households for policy analysis	Simulated decision problems	Not addressed	Public policy
Lin et al. (2026)	Macroeconomic expectations in survey experiments	Retrieved priors and external information	Validated against human survey data	Survey experiments
Xiao et al. (2025)	LLM trading firm	Shared market and news data feeds	Backtest metrics; no per-claim verification	Equity trading
Yang et al. (2020)	Knowledge graph for variable selection	Graph extracted from literature by NLP	Forecast accuracy evaluation	Macro forecasting
Zhu et al. (2025)	KG-guided RAG	Retrieved chunks expanded by graph	Retrieval and response quality metrics	Open-domain QA
Dadopoulos et al. (2025)	Metadata-driven RAG	Contextual document chunks	FinanceBench accuracy	Corporate filings
Kato (2026)	Model-grounded GraphRAG economist	Executed model computations in the graph	Traceable evidence and model-run nodes	U.S. inflation and bank stress
Macrograph AI (this work)	Ownership-constrained multi-agent platform	Sector-owned official observations through MCP	Protocol-level provenance, edge evidence audit, claim verifier (library)	Indian macroeconomy

2.9 RESEARCH GAPS AND MOTIVATION

Role-specialised agents, knowledge graphs and grounded generation have each been studied in some depth. Putting them together for a platform that answers cross-sector questions about one economy from official statistics exposed four gaps.

The first gap concerns data ownership. Multi-agent systems usually let every agent read the same feed. A rule that gives each indicator exactly one owner, and lets other agents obtain it only through declared exchanges, would prevent duplicate and conflicting values, yet I did not find such a rule stated or enforced in the systems reviewed.

The second gap is the evidence behind causal edges. Knowledge graphs are normally presented as if every edge were established. For honest scenario analysis it matters whether a relationship was typed in by a designer or supported by data, and the user should be able to see which is which.

The third gap is between grounding and checking. Grounding lowers the rate of hallucination, but it does not show that a stated figure equals the retrieved one; only a deterministic recomputation of the claim against the evidence can do that. The fourth gap is geographic. Published systems focus on the United States and on equity markets, whereas a platform for India has to deal with MoSPI, RBI and fiscal sources and with their particular conventions.

2.10 TECHNOLOGIES USED

Table 2.2 lists the technologies used in the implementation and the role of each. PydanticAI and a vector database appeared in early design documents but are not part of the implemented system.

Table 2.2: Technologies used in Macrograph AI

Layer	Technology	Role
Language and runtime	Python 3.12, uv	Backend implementation and dependency management
Orchestration	LangGraph (LangChain, n.d.)	Stateful four-node orchestration graph
Data tools	FastMCP (Lowin, n.d.)	One MCP tool server per sector
API gateway	FastAPI (Ramírez, n.d.), Uvicorn	HTTP and Server-Sent Events interface
Models and validation	Pydantic v2 (Pydantic Services Inc., n.d.)	Typed messages, observations and claims
Analytical store	DuckDB (Raasveldt & Mühleisen, 2019), SQLite	Sector stores, canonical store, claim audits
Graph and statistics	NetworkX (Hagberg et al., 2008), statsmodels (Seabold & Perktold, 2010), pandas	Knowledge graph engine, Granger tests, series handling
Language models	Groq-hosted open-weight models through langchain-groq; offline fallback	Narrative synthesis and routing; deterministic mode without a key
Frontend	React 18, TypeScript, Vite, Tailwind CSS, GSAP, Mermaid	Dashboard, chat, causal lab and registry views

Chapter 3
System Requirements and Design

3.1 INTRODUCTION

The requirements below were written before most of the sectors existed and were revised as the prototype exposed problems. For example, the first end-to-end run of the finance sector on 2 October 2026 was followed the same day by a fix to data integrity, routing and citations. The rest of the chapter covers the system context, the layered architecture, the internal structure of a sector agent, the ownership model and the data model. The protocols are treated in Chapter 4 and the implementation in Chapter 5.

3.2 REQUIREMENTS ANALYSIS

3.2.1 FUNCTIONAL REQUIREMENTS

The functional requirements in Table 3.1 come from the objectives in Chapter 1 and from what an analyst would expect the tool to do. Table 3.2 lists the non-functional requirements.

Table 3.1: Functional requirements

ID	Requirement	Objective
FR1	Each of the ten sectors shall expose its indicators through its own MCP server.	O1
FR2	Each sector shall keep a local DuckDB store and tag every value as live, cached or unavailable.	O1, O3
FR3	A user shall be able to ask a question to one sector agent directly or to the orchestrator.	O2, O7
FR4	The orchestrator shall route a question to relevant agents using their Agent Cards.	O2
FR5	Agents shall exchange requests and responses as typed messages with correlation identifiers.	O2
FR6	The protocol shall reject delegation loops, over-deep chains, duplicates and undeclared exchanges.	O2
FR7	A successful response shall carry source provenance; failures shall return structured errors.	O3, O5
FR8	The system shall map transmission paths between indicators in a knowledge graph.	O4
FR9	The system shall simulate a user-specified shock and report its labelled results.	O4
FR10	The report shall list observations with period, status and provenance hash, and name unavailable agents.	O3, O5
FR11	The system shall test graph edges against data and detect anomalies in available series.	O6
FR12	The web application shall provide a dashboard, streaming chat, causal lab and registry view.	O7

3.2.2 NON-FUNCTIONAL REQUIREMENTS

Table 3.2: Non-functional requirements

ID	Requirement	How it is addressed
NFR1	Traceability: every figure can be traced to a source.	SourceProvenance on every response; provenance hash on observations.
NFR2	Resilience: partial failure must not abort the run.	Structured errors, partial status, cached fallback, retries with exponential backoff.
NFR3	Modularity: sectors can be added without changing others.	Registry bootstrap is the only module naming sectors; no sector imports another.
NFR4	Safety: loops and runaway delegation must be impossible.	Depth 5, 25 requests per conversation, cycle and duplicate detection.
NFR5	Security of data access: no SQL injection.	Parameterised DuckDB queries; typed Pydantic validation.
NFR6	Availability without credentials.	Deterministic offline synthesis when no language model key is configured.
NFR7	Maintainability.	Pydantic v2 models, a 400-line module guideline (exceptions are listed in Chapter 9), 455 automated test functions.
NFR8	Honest labelling.	SIM, CACHED, heuristic, SEEDED and EMPIRICAL labels in payloads and interface.

3.3 SYSTEM CONTEXT

Figure 3.1 shows the system boundary. Analysts and researchers use a web application; developers can call the HTTP interface directly. The platform reads official sources through MCP servers and web interfaces, calls a language model provider for narrative text, optionally mirrors the knowledge graph to Neo4j and uses Google Identity Services for sign-in.

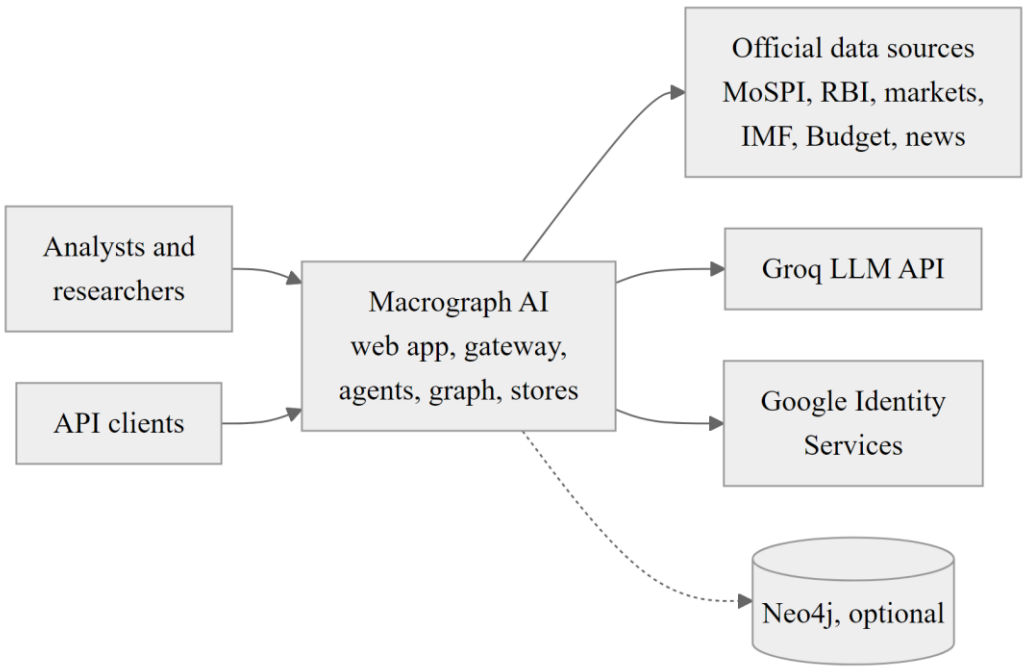


Figure 3.1: System context of Macrograph AI: users, external data sources and external services.

3.4 LAYERED ARCHITECTURE

Figure 3.2 shows the seven layers of the system. Calls only ever go downward. The presentation layer is a React application. The gateway layer is a FastAPI application that exposes the chat, dashboard, research and brief routers, the analysis and simulation endpoints, the A2A interface and the mounted sector sub-applications. The orchestration layer is a LangGraph state graph. The agent layer contains ten sector agents. The protocol layer provides the A2A server and client and the MCP servers and client. The knowledge and analysis layer holds the ontology and graph engine, the scenario engine, the research lab and the claim verification library. The data layer holds the DuckDB and SQLite stores and reaches out to the official sources and the language model API.

In the development configuration all agents run inside one Uvicorn process, so the A2A transport is in-process. The Vite development server proxies API calls to the gateway on port 8000. DuckDB files are local, and Neo4j is optional and disabled when no URI is configured.

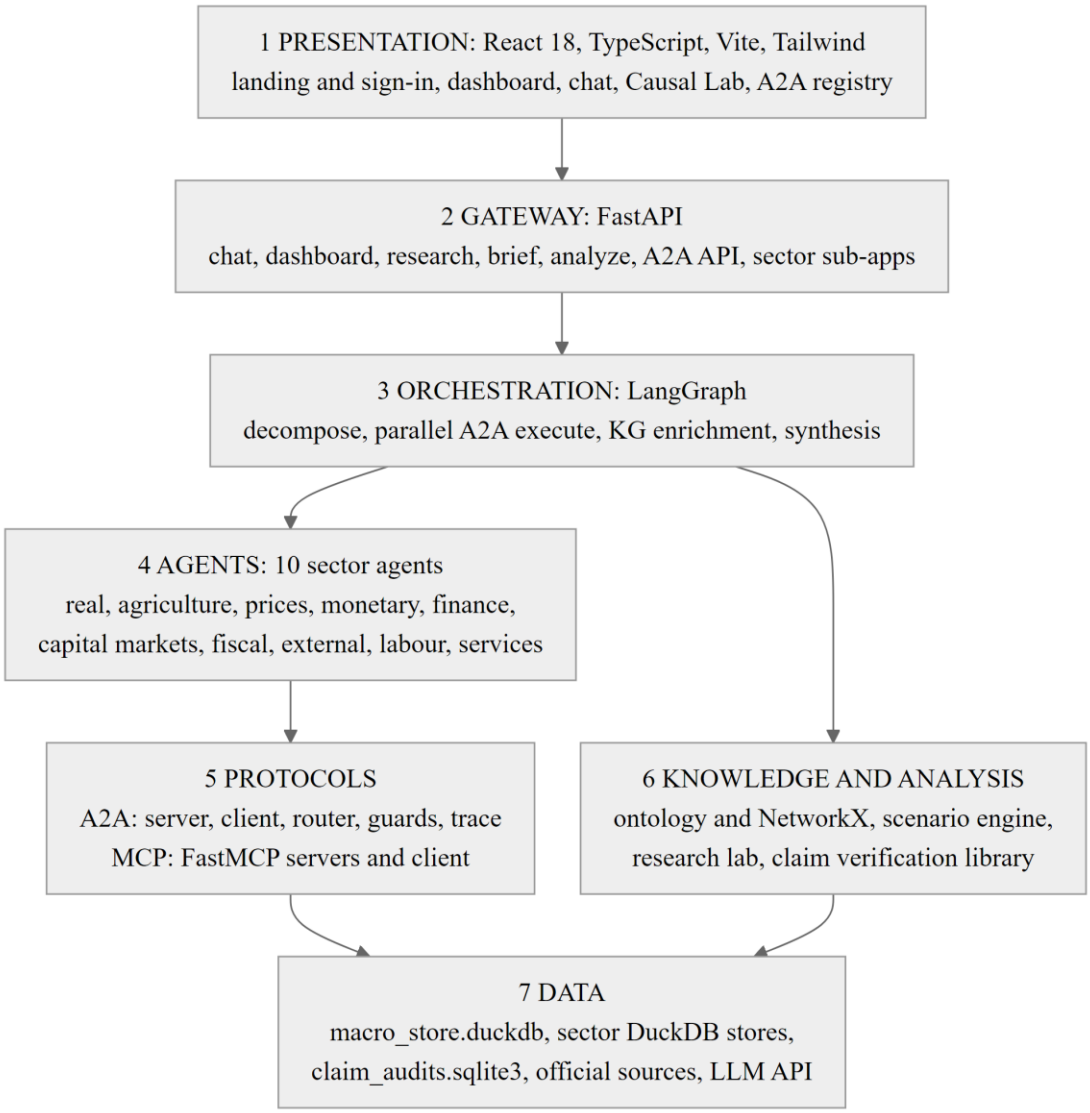


Figure 3.2: Layered architecture of Macrograph AI.

3.5 DESIGN OF A SECTOR AGENT

All sector packages share the structure in Figure 3.3, though some sectors are further along than others; prices, fiscal and real are the most complete. The client module fetches data with asynchronous HTTP calls and exponential-backoff retries; parsers normalise the payloads; Pydantic models validate them; the database module defines the DuckDB schema and caches results using parameterised SQL; the MCP server exposes typed tools; the agent node reasons over the data; the executor exposes the agent to the A2A server; and the sub-application exposes HTTP routes.

A single-sector question can bypass the orchestrator. Each agent module provides a check of the form *is_direct_<sector>_query*, and the shared helper *core/sector_reasoning.py* provides evidence-grounded reasoning for direct queries.

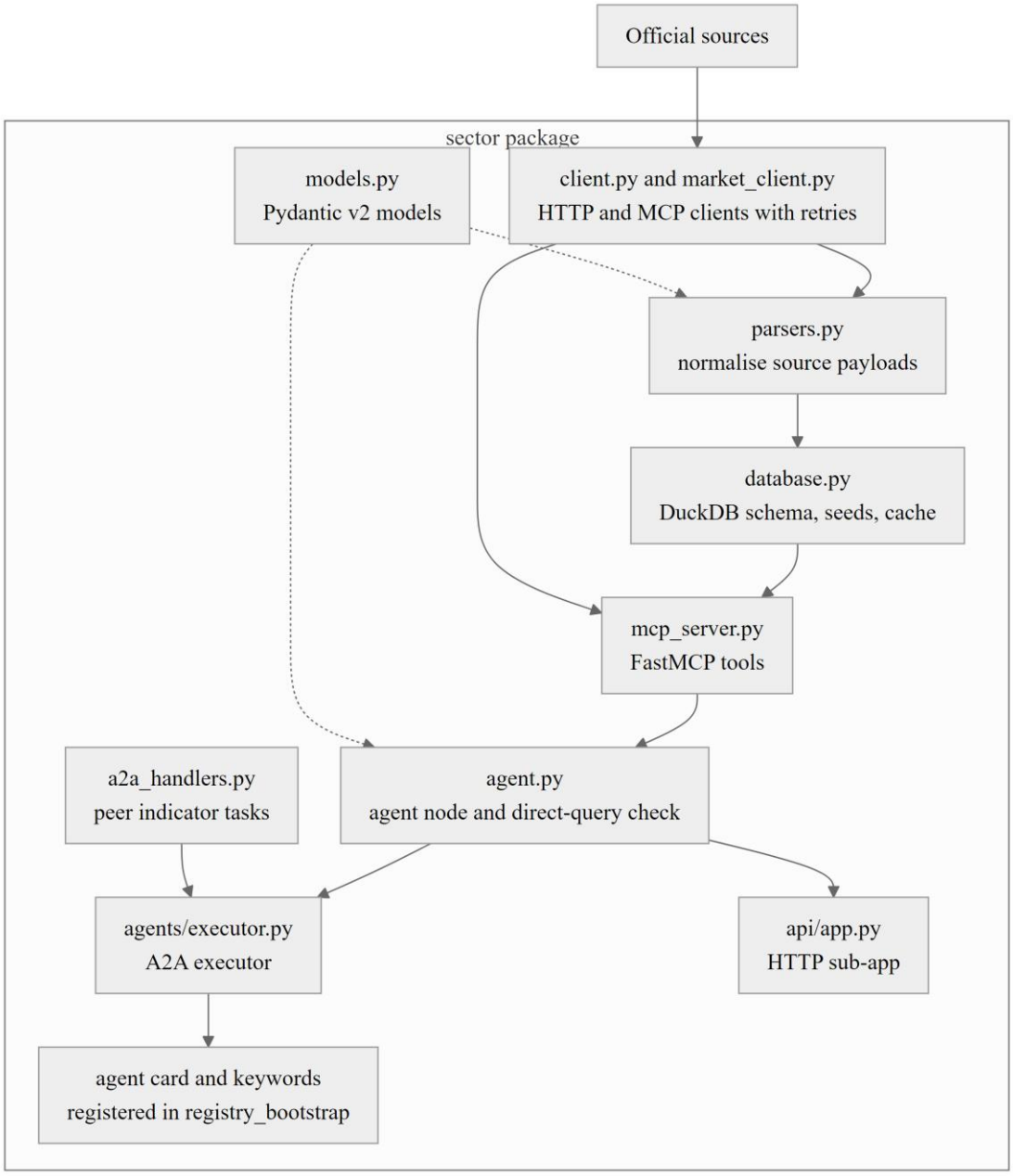


Figure 3.3: Standard internal structure of a sector agent package.

3.6 INDICATOR OWNERSHIP AND DEPENDENCIES

The rule that the rest of the design depends on is single ownership: every indicator has exactly one owner agent, for example CPI and WPI belong to the prices agent and the repo rate belongs to the monetary agent. Another agent that needs an owned indicator must obtain it from the owner through an A2A exchange and must never recompute or re-fetch it. Only seven sector-to-sector exchanges are declared (Figure 3.4); the orchestrator may call every agent, and all other pairs are refused with the error code *DEPENDENCY_NOT_ALLOWED*.

Table 3.3 lists the owner of each group of indicators.

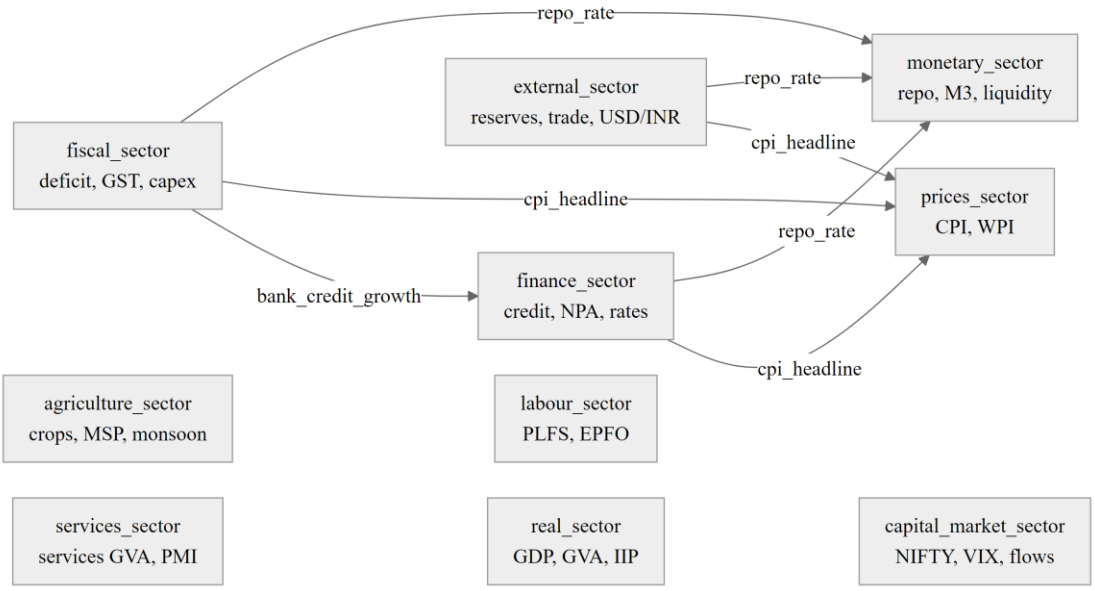


Figure 3.4: Indicator ownership and the seven declared sector-to-sector A2A dependencies.

Table 3.3: Indicator ownership by sector agent

Owner agent	Primary indicators
prices_sector	CPI, WPI, food, fuel and core inflation
monetary_sector	Repo rate, SDF, MSF, M1 and M3, liquidity
finance_sector	Bank credit growth, NPA ratios, lending and deposit rates
fiscal_sector	Fiscal deficit, GST collections, Union capital expenditure
external_sector	Trade balance, forex reserves, balance of payments, USD/INR
real_sector	GDP, GVA, IIP, gross fixed capital formation
capital_market_sector	NIFTY indices, India VIX, FII and DII flows, G-Sec yields
agriculture_sector	Crop production, MSP, mandi prices, monsoon rainfall
labour_sector	PLFS unemployment, labour force participation, EPFO payroll additions
services_sector	Services GVA, services PMI, IT and ITeS exports

3.7 DATA MODEL

The canonical data layer in *core/data/schema.py* separates facts, relationships and simulations. Figure 3.5 shows the four principal entities. A *CanonicalIndicator* describes what can be measured, with a hierarchical identifier such as *in.macro.prices.cpi_headline*, a unit, a frequency and a source authority. A *CanonicalObservation* is one measured value with its period, retrieval timestamp, revision status, data status and a provenance hash. A *CausalRelationship* links two indicators with a type, lag, sign, p-value field, confidence and mechanism text. A *ScenarioResult* records a simulated shock together with the identifiers of the relationships that carried it.

Each sector also keeps its own tables in DuckDB, for example the monetary store has policy rates, money supply, system liquidity and monetary stance tables, and every store has a *fetch_log* table. The canonical store *macro_store.duckdb* additionally keeps the daily briefs and a news cache.

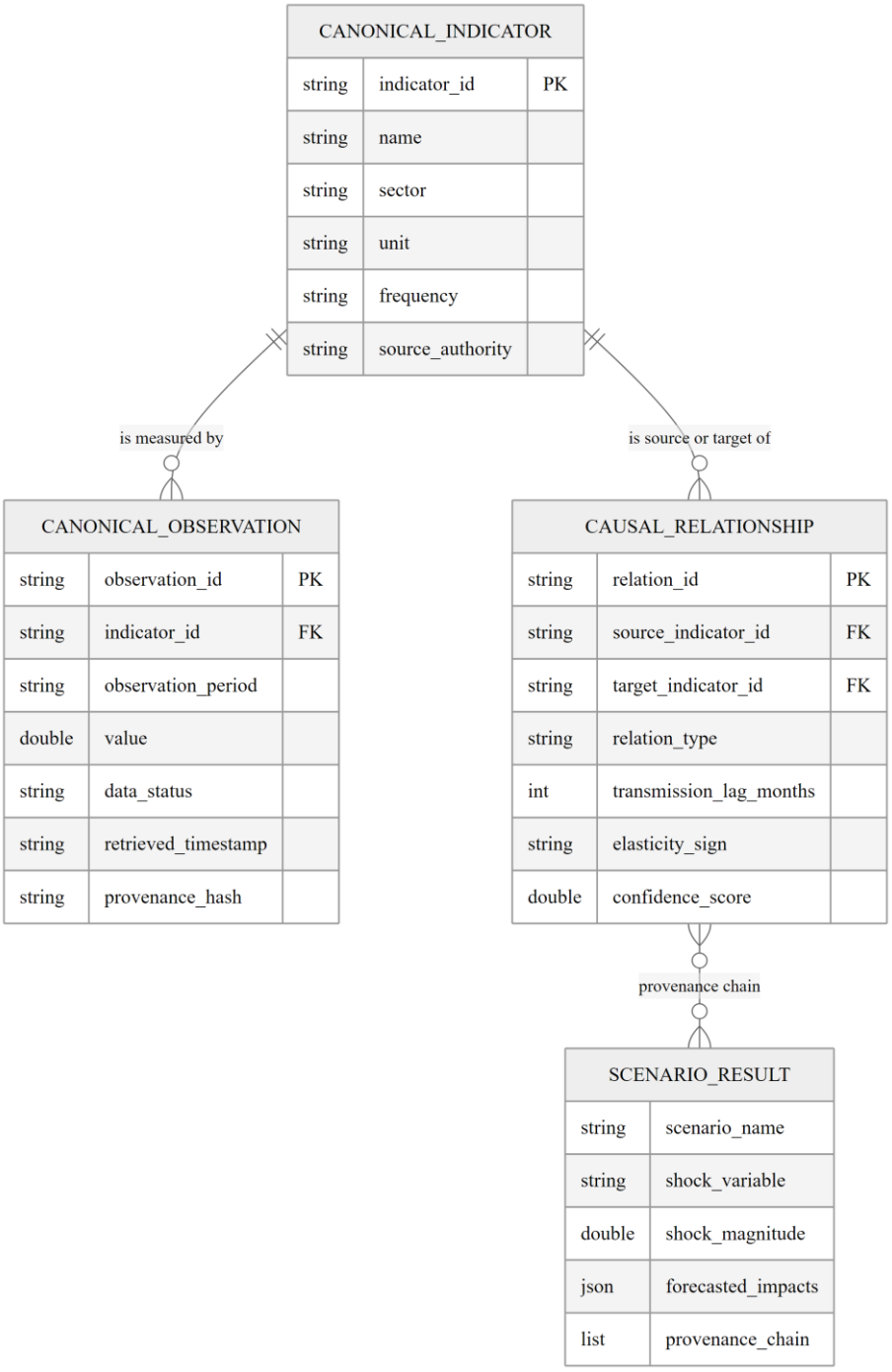


Figure 3.5: Entity-relationship diagram of the canonical economic data layer.

3.8 DESIGN DECISIONS AND RATIONALE

Table 3.4 records the principal design decisions and the reason for each.

Table 3.4: Principal design decisions and their rationale

Decision	Rationale
Native typed A2A layer instead of a library	The project needed guards, mandatory provenance and a declared dependency graph, which are easier to enforce in one server.
One MCP server per sector	Keeps data access inside the owner and removes the need for cross-sector data calls.
In-process transport by default	All agents share one gateway; an HTTP transport can replace it without changing callers.
DuckDB for stores	Embedded analytical SQL with no server to operate; parameterised queries.
NetworkX for the graph, Neo4j optional	In-memory queries are sufficient for twenty nodes; Neo4j mirrors the ontology for persistence.
Deterministic fallbacks everywhere	Allows demonstration, testing and operation without credentials.
Heuristic scenario engine labelled SIM	Transparent and explainable; replaced later by estimated models (Chapter 10).
Structured unavailable status	Prevents silent substitution of invented values.

Chapter 4
Protocol Design

4.1 INTRODUCTION

Two channels connect the parts of Macrograph AI, and each has one job. The agent-to-agent (A2A) channel carries reasoning, meaning delegated tasks, structured conclusions and indicator signals. The MCP channel carries data, as typed tool calls against official sources and local stores. No agent ever fetches another agent's data. If it needs a peer's indicator, it asks the peer over A2A. The sections below cover the A2A layer, the MCP data layer and how provenance survives the trip through both.

4.2 THE AGENT-TO-AGENT LAYER

4.2.1 MESSAGES

The A2A layer lives in *backend/core/protocols/a2a*. All messages are strict Pydantic v2 models that forbid extra fields. An *A2ARequest* carries a request identifier, sender, receiver, task, parameters, conversation identifier, the identifier of its parent request, the current delegation depth and the chain of agents that have already been called. An *A2AResponse* carries the status (success, partial or failed), a result, a list of *SourceProvenance* records, structured errors, optional follow-up requests and metadata.

Table 4.1 lists the fields of the principal messages.

Table 4.1: Fields of the principal A2A messages

Message	Fields
A2ARequest	request_id, sender_agent, receiver_agent, task, parameters, context, conversation_id, parent_request_id, depth, call_chain, timeout_seconds, timestamp
A2AResponse	request_id, sender_agent, receiver_agent, status, result, sources, errors, follow_ups, metadata, timestamp
SourceProvenance	source_name, source_url, dataset, table, reporting_period, retrieved_at, page_or_section, record_reference
A2AErrorInfo	code, message, agent
IndicatorSignal	owner_agent, observations (indicator_id, label, value, unit, observation_period, data_status)

4.2.2 THE A2A SERVER

All requests pass through one *A2AServer*. Its *handle* method applies a fixed pipeline (Figure 4.1). The receiver must be registered; the loop guard must pass; the sender

must either be unrestricted (the orchestrator and the gateway) or hold a declared dependency on the receiver; the task must be in the receiver's supported tasks and have a registered handler; the parameters must match the task's parameter model. The handler then runs under a timeout, and the result must have a valid type and, unless the response failed, at least one source. A violation at any step produces a failed response with a stable error code, so an exception raised inside one agent never reaches another.

Requiring sources on every non-failed response is how the project rule “No Source, No Answer” is enforced in code rather than left to each agent. A handler cannot return a bare answer; it must return a *HandlerOutput* whose sources the server preserves unchanged. Table 4.2 lists the error codes.

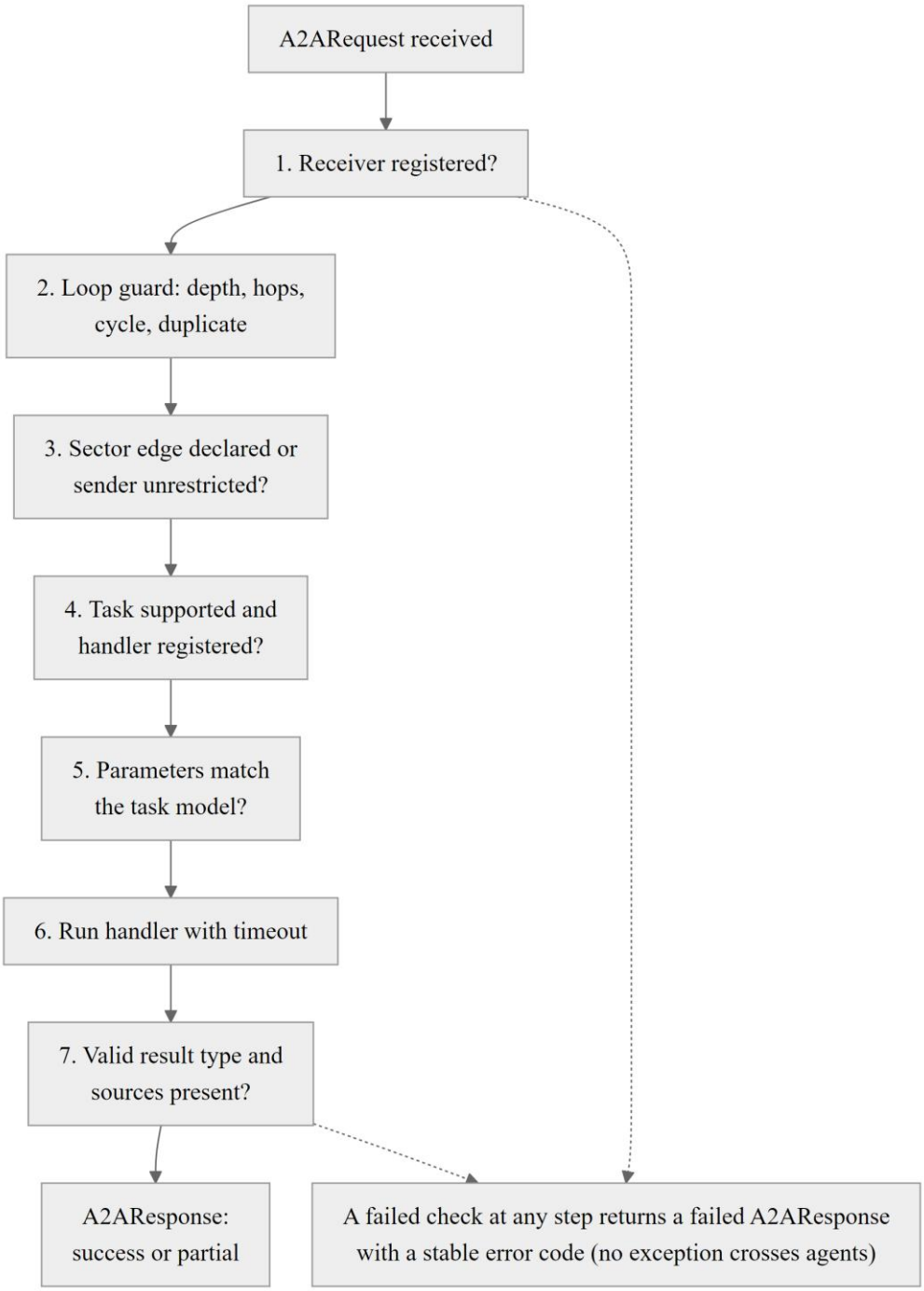


Figure 4.1: Validation pipeline applied by the A2A server to every request.

Table 4.2: A2A error codes

Code	Meaning
INVALID_REQUEST	Bad parameters or malformed call chain
AGENT_NOT_FOUND, AGENT_UNAVAILABLE	Unknown agent, or an advertised task without a handler
UNSUPPORTED_CAPABILITY	Task not listed in the receiver's supported tasks
DEPENDENCY_NOT_ALLOWED	Sector-to-sector edge not declared
MAX_DEPTH_EXCEEDED, MAX_HOPS_EXCEEDED	Delegation too deep, or too many requests in a conversation
CIRCULAR_DELEGATION, DUPLICATE_REQUEST	Receiver already on the call chain, or an identical request repeated
AGENT_TIMEOUT	Handler exceeded its timeout (retryable)
DATA_SOURCE_ERROR, LLM_ERROR, AGENT_ERROR	MCP or upstream failure, language model failure, any other handler exception
INVALID_RESPONSE	Schema violation, missing sources, wrong type or correlation mismatch
TRANSPORT_ERROR	Delivery failed (retryable)

4.2.3 LOOP PROTECTION AND RELIABILITY

Every request carries its conversation identifier, parent identifier, depth and call chain. A child request is built from the handler's call context, so depth and chain are derived and cannot be forged by the handler. Four safeguards apply: a depth limit, a cap on requests per conversation, cycle detection and duplicate detection. By default the limits are a maximum depth of five, at most twenty-five requests in a conversation, a request timeout of ninety seconds and a peer timeout of twenty-five seconds, all configurable. Only timeouts and transport errors are retried, with exponential backoff and the same request identifier; the orchestrator sends its full analyses without retries because they are expensive. Every request ends as a success, a partial result or a failure; only the retryable failures go back to the running state.

4.2.4 SEQUENCE OF A DELEGATED ANALYSIS

Figure 4.2 follows a single delegation. The orchestrator sends a *sector_analysis* request to the external-sector agent. While loading its own data, that agent asks the monetary agent for the repo rate through a peer request one level deeper. The server checks the declared edge, runs the peer handler and returns the indicator signal with its provenance. The external-sector agent then completes its analysis and returns a response whose sources include the peer's original source record.

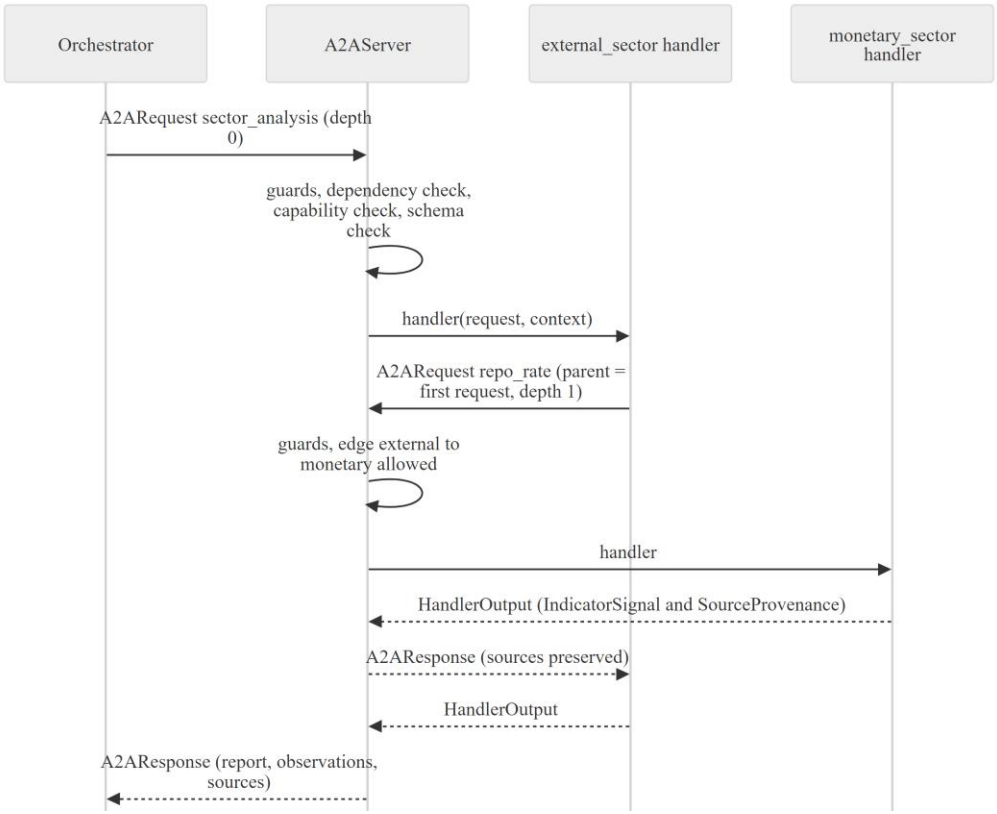


Figure 4.2: Sequence of an A2A delegation in which a sector agent requests a peer indicator.

4.3 AGENT CARDS, REGISTRY AND ROUTING

Each agent publishes an Agent Card that records its identifier, capabilities, skills, supported tasks, input and output schema names and metadata such as its sector and routing keywords. Cards are registered in one place, *registry_bootstrap.py*, which is the only module in *core* that names sector packages. The server appends a task to the card when a handler is registered, so a card always matches what is executable.

The router scores each card's capabilities, skill tags and keywords against the query. If every match counted equally, a common word such as “rbi” would pull several unrelated sectors into the answer, so a phrase advertised by several agents counts for one over the number of agents that advertise it. A language model is consulted only when no card matches distinctively, and its answer is intersected with the registered agent identifiers, so the model can select receivers but cannot bypass the protocol.

4.4 CROSS-SECTOR EXCHANGES

The orchestrator is unrestricted, but sector agents may only exchange the seven declared peer tasks listed in Table 4.3. The finance, fiscal and external agents request their peers while loading their own data through the helper

peers.gather_peer_context. A peer failure degrades to an "unavailable" line in the report and never fails the caller.

Table 4.3: Declared sector-to-sector A2A dependencies

Consumer	Provider	Task	Rationale
finance_sector	monetary_sector	repo_rate	Lending-rate spreads are measured against the policy rate.
finance_sector	prices_sector	cpi_headline	Real lending and deposit rates need headline CPI.
fiscal_sector	monetary_sector	repo_rate	Interest cost of government borrowing.
fiscal_sector	prices_sector	cpi_headline	Real deficit and revenue buoyancy.
fiscal_sector	finance_sector	bank_credit_growth	Crowding out of private credit.
external_sector	monetary_sector	repo_rate	Rate differentials drive flows and rupee pressure.
external_sector	prices_sector	cpi_headline	Imported inflation and the real exchange rate.

4.5 THE MCP DATA LAYER

Every sector exposes its data through a FastMCP server defined in *mcp_server.py*. Each server offers typed tools whose parameters are described by Pydantic models, so a client can discover and validate calls. Behind the tools, the sector's client module fetches from official sources using asynchronous HTTP with exponential-backoff retries. The sources include the MoSPI e-Sankhyiki MCP service (Ministry of Statistics and Programme Implementation, n.d.) for national accounts, industrial production, consumer prices and wholesale prices; RBI DBIE tables (Reserve Bank of India, n.d.) for rates, credit and asset quality; Yahoo Finance for market data; and budget and IMF series for fiscal data.

Each tool follows the fallback chain in Figure 4.3. It first tries the live official source. If that fails, it returns the latest snapshot stored in the sector's DuckDB, tagged as cached. If neither is available it returns a structured result with the status "unavailable" and a reason; at no point does it make up a number. Every result carries the value, unit, observation period, data status and a citation giving the source authority, table and retrieval time.

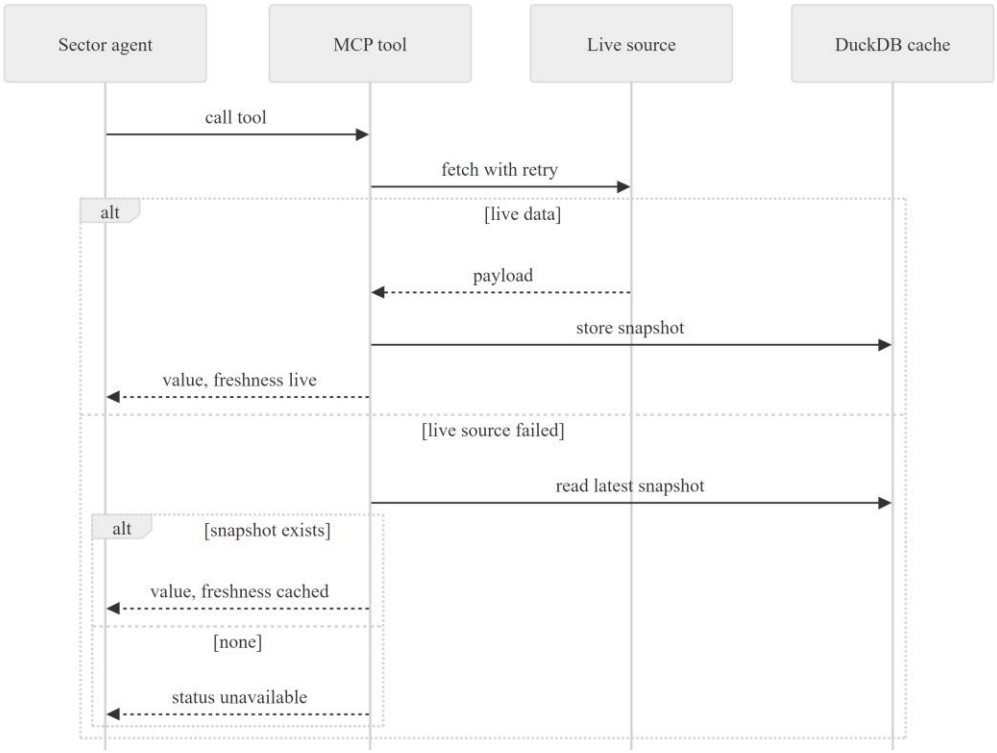


Figure 4.3: An agent retrieving data through its own FastMCP server, with the fallback order.

4.6 PROVENANCE ACROSS THE PROTOCOLS

Provenance is preserved at every step (Figure 4.4). The MCP tool result carries the source citation. The handler converts the sector's citation records into *SourceProvenance* objects; where a sector provides only a markdown report, the conversion records an artifact-level reference (agent, artifact name and content hash) instead of inventing fields. The server returns the sources unchanged. The orchestrator aggregates observations, citations and sources; each observation receives a SHA-256-based provenance hash computed from its indicator, period, value, unit and status. The final report prints the hash in its indicator matrix, and the web interface opens an evidence drawer from the same record.

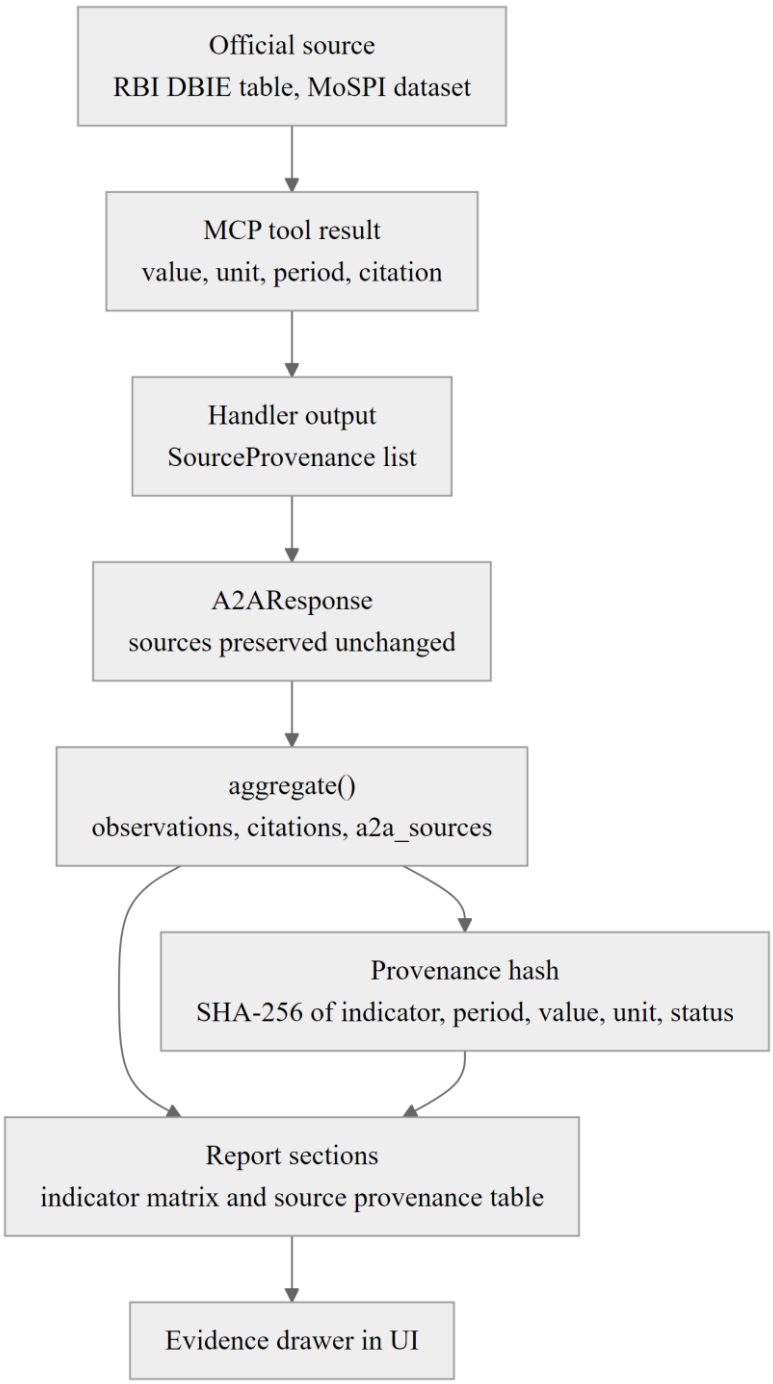


Figure 4.4: Propagation of provenance from the official source to the final report.

4.7 TRACEABILITY

A trace store records every request of a conversation. It can reconstruct the delegation tree and exposes it through *GET /a2a/v1/traces/{conversation_id}*; the chat interface streams one step event per request, and the web application renders the tree with timings. Each interaction is also logged on one line with the request identifier, parties, task, depth, status, error codes, number of sources and duration. Traces are held in memory and bounded; persisting them is listed as future work.

Chapter 5
Implementation

5.1 INTRODUCTION

The design of Chapter 3 was implemented in about 37,000 lines of Python and 11,300 lines of TypeScript, not counting tests. The order of work was roughly bottom-up: the real-sector agent came first, then the shared A2A and MCP layers, then the remaining sectors, and finally the orchestrator, the knowledge-graph integration and the research lab. This chapter follows the request path instead, from the orchestrator down to the data pipelines, and ends with the offline and failure behaviour.

5.2 DEVELOPMENT ENVIRONMENT

Table 5.1 summarises the development and runtime environment.

Table 5.1: Development and runtime environment

Item	Specification
Operating system	Windows 11 (development); the code is platform independent
Backend	Python 3.12 managed with uv and a lock file; FastAPI, FastMCP, LangGraph, Pydantic v2
Data	DuckDB, SQLite, pandas, NumPy, statsmodels, NetworkX; optional Neo4j driver
Frontend	Node.js 22, React 18, TypeScript (strict), Vite 5, Tailwind CSS 3
Models	Groq-hosted open-weight models selected by FAST_MODEL and REASONING_MODEL settings
Testing	pytest with asyncio mode auto; TypeScript compilation as the frontend check
Source control	Git; development history from 16 July to 11 October 2026

5.3 THE ORCHESTRATOR

The orchestrator is a LangGraph state graph of four nodes (Figure 5.1), each reading and writing keys of a typed *OrchestratorState*.

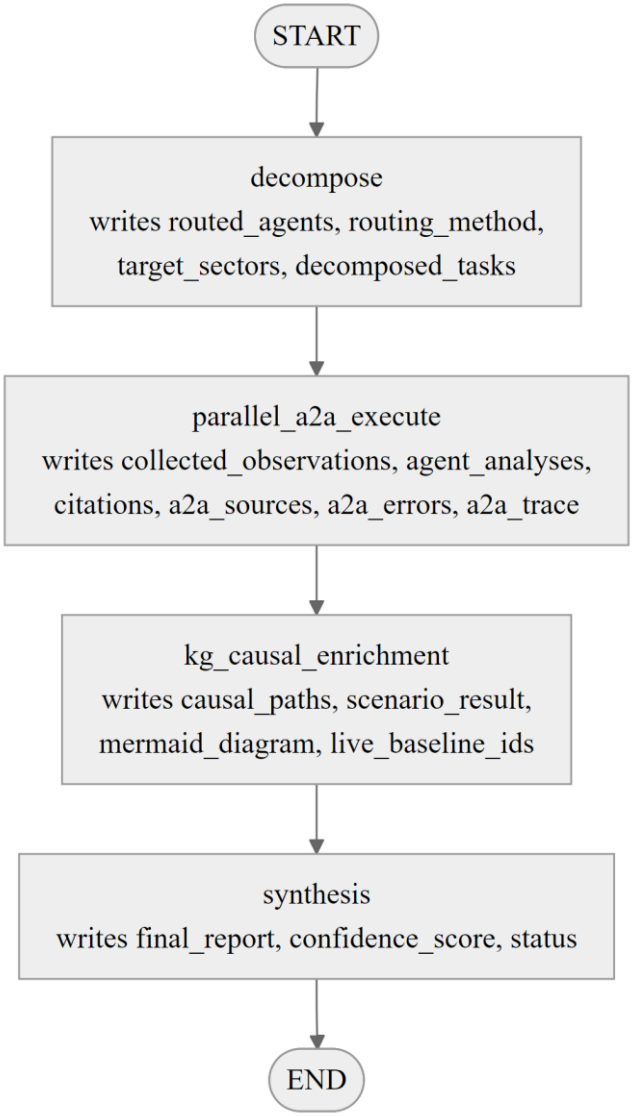


Figure 5.1: The four-node LangGraph orchestrator and the state keys written by each node.

5.3.1 DECOMPOSE

The decomposition node routes the query over the registered Agent Cards using the router described in Section 4.3. It records the selected agents, the routing method and the sectors they belong to.

5.3.2 PARALLEL A2A EXECUTION

The execution node sends one *sector_analysis* request to each selected agent concurrently. A response may include up to three follow-up requests, which are sent in one bounded second round. The aggregation step validates each response and extracts observations and citations from JSON artifacts, or keeps the report text with an artifact-level source record for agents that return markdown. Failures are appended

to the list of A2A errors and the run status becomes partial; the run continues with the data that is available.

5.3.3 KNOWLEDGE-GRAPH ENRICHMENT

The enrichment node keeps the indicators of the collected observations that are nodes of the knowledge graph and queries the shortest transmission path for every ordered pair. If the request included a scenario shock it runs the scenario engine. First-usable live observations from the sector agents replace the default baselines, so the report cannot contradict the retrieved data, and the report states which indicators used live baselines. A Mermaid diagram of the first five graph nodes is produced for the report.

5.3.4 SYNTHESIS

The synthesis node builds the final report. The language model writes only the executive synthesis, using a prompt that contains the observations, causal paths with relation identifiers, scenario outputs and the list of agents that failed. Sections two to six are assembled deterministically from structured state (Figure 5.2): the indicator matrix with provenance hashes, the causal diagram and path table, the scenario table, the source provenance table and the methodological notes. If no language model is available, the executive synthesis is a deterministic summary of the evidence, so the evidence sections never depend on model output.

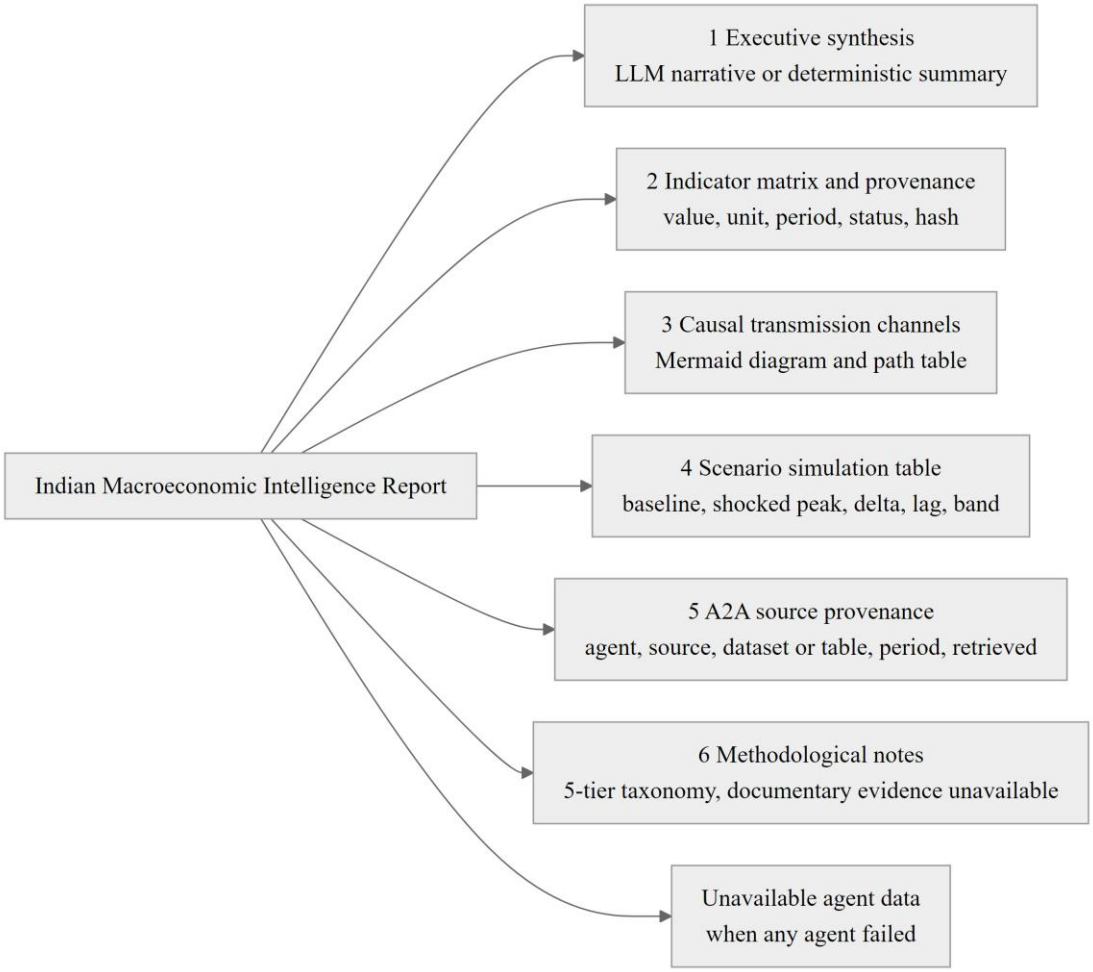


Figure 5.2: Sections of the cited intelligence report produced by the synthesis node.

5.4 CHAT INTERFACE AND ROUTING

The chat router streams progress and text with Server-Sent Events. The selected agent determines the route. A direct sector chat calls that sector's agent node and bypasses the orchestrator. When the orchestrator is selected but the query is clearly a single prices question, it is redirected to the prices agent. All other queries run the orchestrator graph. The final event carries the report, citations, the A2A summary, a consensus summary and a model card.

5.5 SECTOR AGENTS AND DATA ACQUISITION

All ten sector agents share the pipeline in Figure 5.3. Each fetches from official sources through its own client, normalises and validates the payload, stores it with parameterised SQL and exposes it through an MCP tool together with a citation and a freshness flag. Figure 5.4 lists the primary sources of each sector.

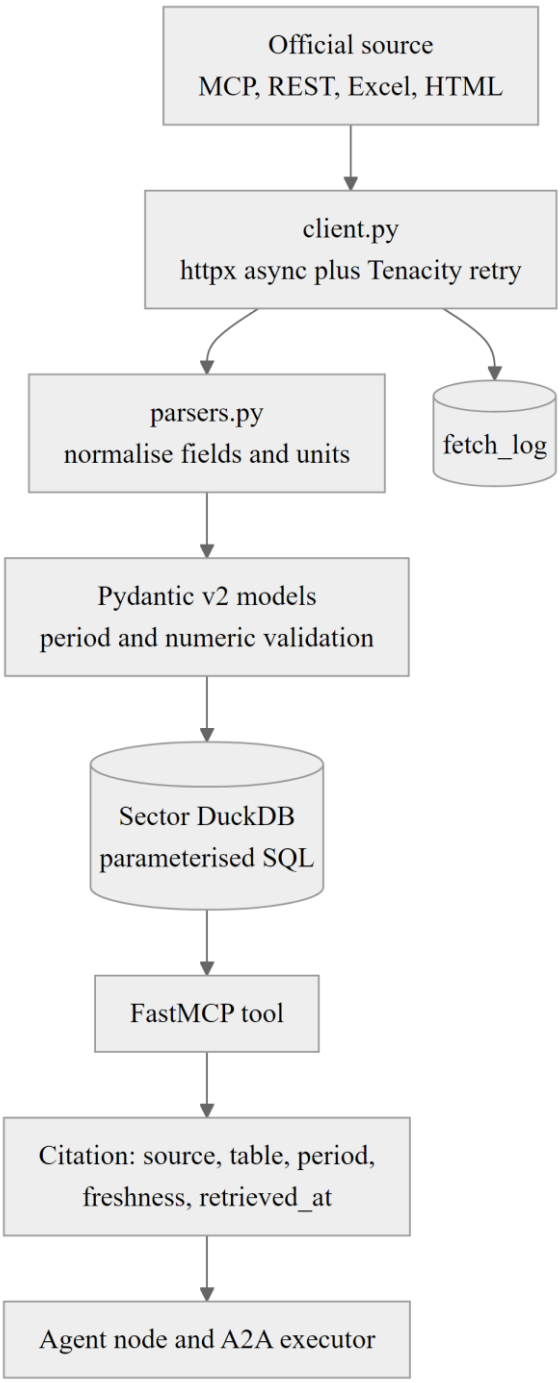


Figure 5.3: Data acquisition pipeline shared by the sector agents.

The prices, finance and several other agents combine the official observations with market proxies and news as context. A direct agent always works in the same order. Official observations are fetched first; market proxies and news are fetched concurrently as context and are never treated as authoritative data; the language model receives the consolidated dataset and must cite periods and sources. A query outside the sector's domain is declined, and the reply names the agent that should be asked instead.

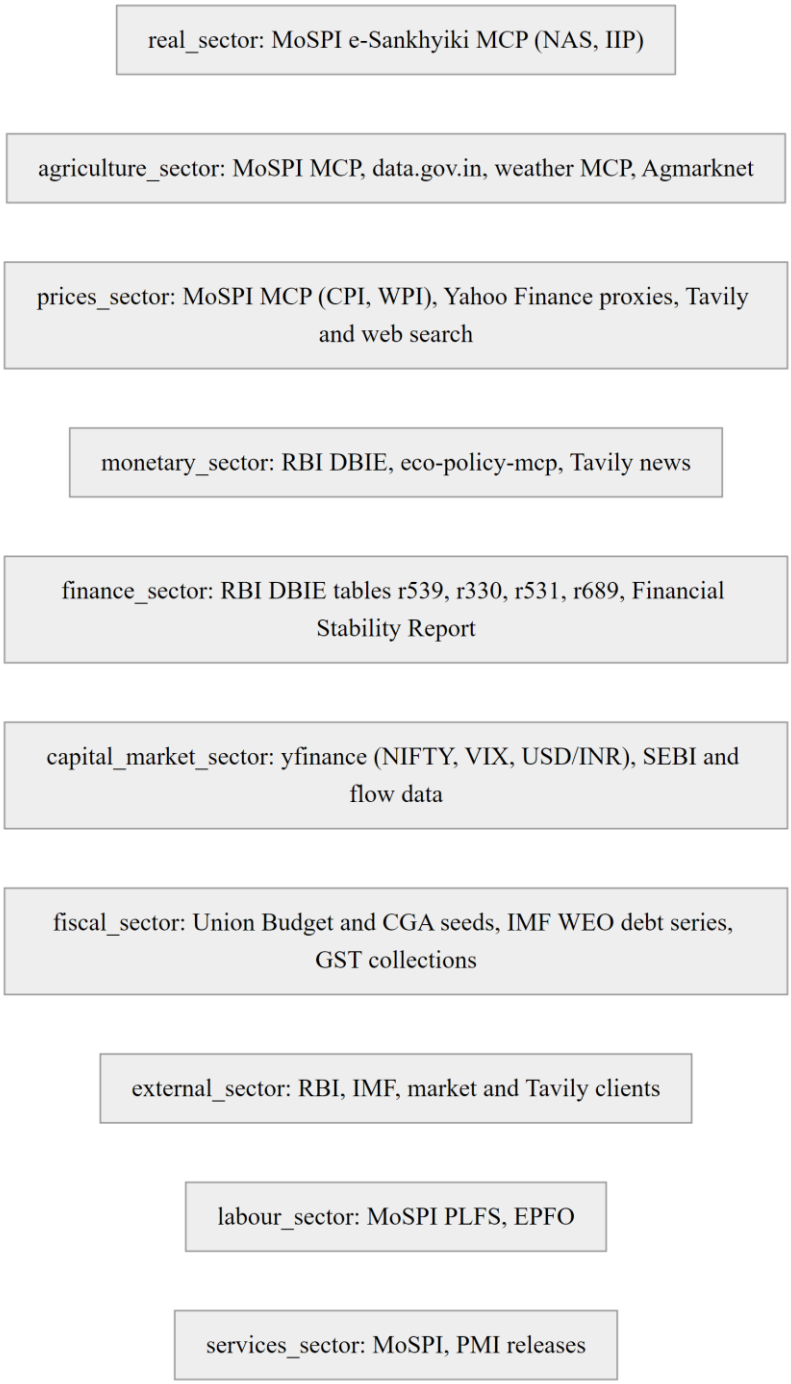


Figure 5.4: Primary data sources feeding each sector agent.

5.6 STORAGE

Sector data lives in DuckDB tables named after the indicators they hold, for example policy rates, money supply, system liquidity and monetary stance for the monetary sector, or union fiscal deficit, general government debt and GST collections for the fiscal sector. The canonical store keeps the indicator registry, canonical observations, the daily briefs and a news cache. SQL statements use parameter placeholders for all

values. Curated baseline snapshots (for example Union Budget aggregates) are seeded into the sector stores with their publication references and are tagged as cached.

The shared ingestion pipeline of the core data layer registers one adapter per canonical indicator, validates numeric ranges and period formats, normalises to a canonical observation, attaches the provenance hash and stores the result.

5.7 ROBUSTNESS AND OFFLINE OPERATION

The platform also has to keep working when no language model key is configured or when a government endpoint times out. External calls use exponential-backoff retries. Every language-model stage has a deterministic alternative built from the same structured evidence: routing falls back to card scoring, synthesis to a counted summary, the daily brief to a template and the simulated committee to rule-based votes. When a data source fails, the tool serves a cached snapshot or an unavailable status, and the failure is recorded as an A2A error and listed in the report. The test suite exercises the no-key path explicitly, because mocking the language model once hid an import error in the real client.

5.8 CODE ORGANISATION AND STATISTICS

Table 5.2 summarises the size of the implementation. Test code is excluded from the line counts.

Table 5.2: Source lines by component (tests excluded, 12 October 2026)

Component	Lines
Frontend (TypeScript)	11,287
core (orchestrator, protocols, graph, research, briefing, verification)	6,778
api (routers and helpers)	2,877
Ten sector packages (between 1,872 and 3,274 lines each)	27,257
Backend total of the packages above	36,912

Chapter 6
Knowledge Graph and Scenario Engine

6.1 INTRODUCTION

A sector agent can say what the CPI or the repo rate is. It cannot say how an oil shock reaches the repo rate, because that is a question about the links between indicators. Those links are held in a causal knowledge graph, which the orchestrator queries and the scenario engine uses to push a shock through the economy. Every number in this chapter was produced by running the implemented engine on 12 October 2026.

6.2 ONTOLOGY OF INDICATORS AND TRANSMISSION EDGES

The ontology in *core/knowledge_graph/ontology.py* registers twenty canonical indicators and twelve directed relationships. Indicators use the identifier pattern *in.macro.<domain>.<name>* and belong to eight domains: real economy, prices and inflation, monetary and banking, fiscal, external, capital markets, agriculture and labour. Figure 6.1 shows the graph. Five indicators (NIFTY 50, India VIX, forex reserves, debt-to-GDP and EPFO additions) currently have no edges; they are valid nodes that can receive relationships as the ontology grows.

Each relationship has a deterministic identifier of the form *rel-<8 hex digits>* derived from a version-5 UUID of its source, target and type, so identifiers survive restarts and are usable as provenance keys. Table 6.1 lists all edges. The relationship type follows a five-tier taxonomy, from qualitative theory to a structural causal model.

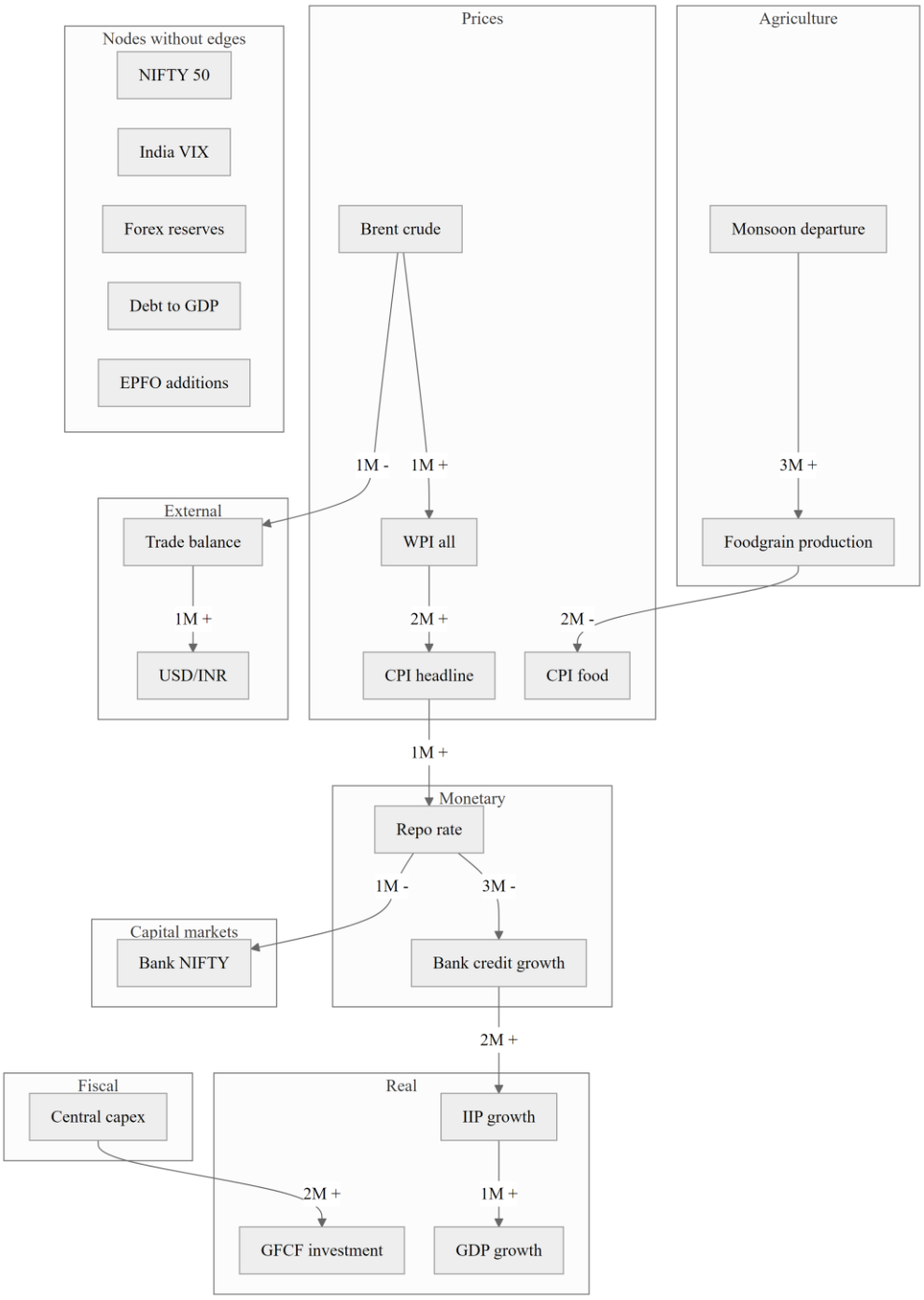


Figure 6.1: Causal transmission graph of 20 indicators connected by 12 typed edges (lag in months and sign).

Table 6.1: The twelve causal edges of the knowledge graph

Source	Target	Type	Lag (months)	Sign	Confidence
Brent crude	WPI	Lagged relationship	1	+	0.95
WPI	CPI headline	Granger predictive	2	+	0.90
CPI headline	Repo rate	Structural causal model	1	+	0.95
Repo rate	Bank credit growth	Granger predictive	3	−	0.88
Bank credit growth	IIP growth	Statistical association	2	+	0.82
IIP growth	GDP growth	Structural causal model	1	+	0.98
Brent crude	Trade balance	Theory	1	−	0.92
Trade balance	USD/INR	Statistical association	1	+	0.86
Repo rate	Bank NIFTY	Lagged relationship	1	−	0.85
Central capex	GFCF investment	Theory	2	+	0.90
Monsoon departure	Foodgrain production	Theory	3	+	0.92
Foodgrain production	CPI food	Statistical association	2	−	0.89

The ontology validates itself on load: it rejects self-loops, unknown endpoints, duplicate identifiers, malformed identifiers and signs other than plus or minus. Edge confidences and p-values in the current ontology are curated priors typed in by the designer; Section 7.5 describes how they are audited against data, and Chapter 9 reports the result.

6.3 GRAPH ENGINE AND QUERIES

NetworkXGraphEngine builds an in-memory directed multigraph from the ontology. It answers three kinds of query. The shortest transmission path between two indicators is the path with the fewest hops; when two parallel edges exist the one with the higher confidence is used. The downstream impact query enumerates all descendants of a shocked indicator and reports hop count, total lag (the sum of edge lags), cumulative confidence (the product of edge confidences) and the chain of mechanism descriptions. The causal neighbourhood query returns the indicators within a given depth. The engine also generates Mermaid diagrams for the reports.

Figure 6.2 shows how the ontology, the engine, the optional Neo4j mirror and the consumers of the graph fit together. The Neo4j mirror stores the same nodes and

edges for persistence and is disabled when no URI is configured. When enabled, the ontology is synchronised once at gateway start by an idempotent upsert with three retry attempts; NetworkX remains the query engine of the orchestrator.

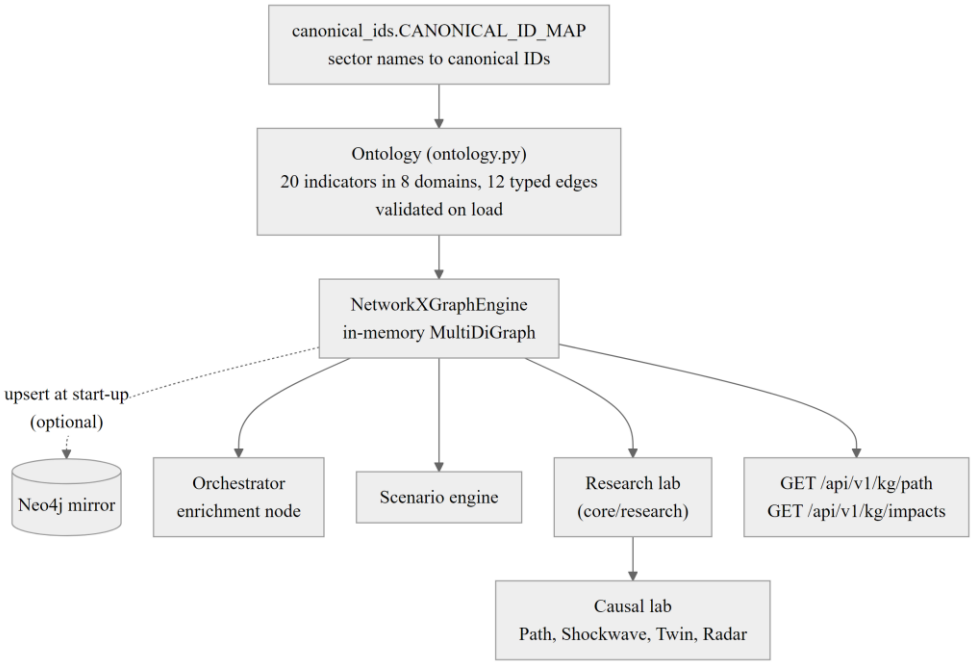


Figure 6.2: Structure, storage and consumers of the knowledge graph.

6.4 SCENARIO ENGINE

The scenario engine propagates a shock along the shortest path from the shocked indicator to each downstream indicator. For a path with hops h, edge signs s_h and confidences c_h, the engine computes the path multiplier and the target delta:

$$M(path) = \Pi \text{ over hops } h \text{ of } (s_h \times 0.55 \times c_h) \qquad (1)$$

$$\Delta(target) = shock \times M(path) \times 0.15 \qquad (2)$$

Figure 6.3 lays out these steps and the research modules that consume the result. The shocked peak is the baseline plus the delta, and a band of plus or minus 25 per cent of the delta is reported. The constants 0.55 (per-hop attenuation) and 0.15 (global scale) are design parameters, not estimates from data. The lag of the effect is the sum of the edge lags and the cumulative confidence is the product of the edge confidences. In short, the engine is a transparent heuristic for showing how a shock moves through the graph; it is not a vector autoregression or an impulse-response estimate, the deltas are in each indicator's own unit, and the band is not a statistical interval. For these reasons all scenario outputs are labelled as simulated.

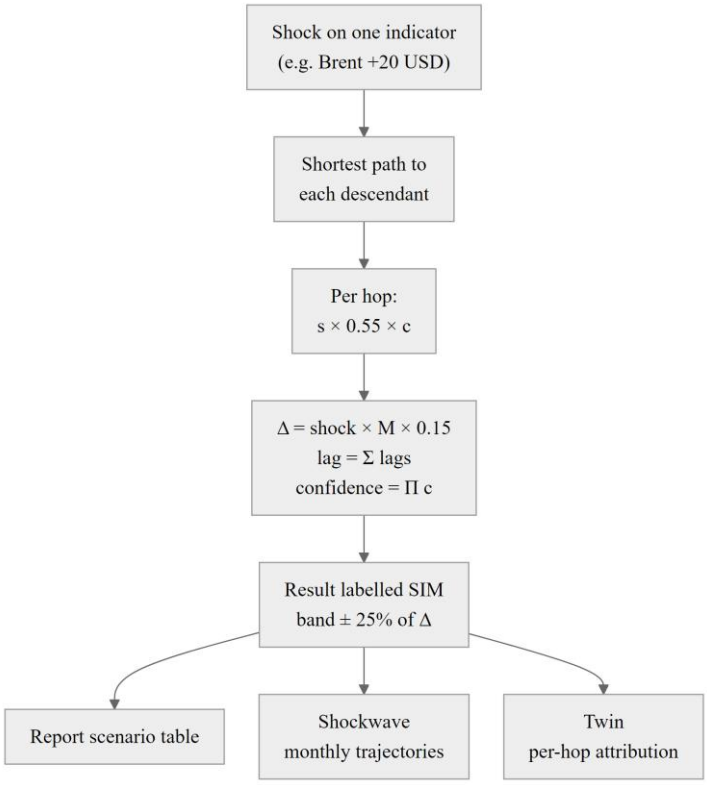


Figure 6.3: Steps of shock propagation and the modules that use its output.

6.5 WORKED EXAMPLE: CRUDE OIL SHOCK

Table 6.2 shows the engine's output for a shock of plus 20 US dollars per barrel on Brent crude, using the default baselines. The first hop illustrates the arithmetic: 20 × (0.55 × 0.95) × 0.15 = 1.57, which is added to the wholesale price inflation baseline of 2.45 to give 4.02.

Table 6.2: Simulated transmission of a USD 20 per barrel Brent increase (default baselines)

Indicator	Baseline	Shocked peak	Delta	Lag (months)	Cumulative confidence
WPI all commodities	2.45	4.02	+1.57	1	0.95
Merchandise trade balance	−22.50	−24.02	−1.52	1	0.92
CPI headline	4.26	5.04	+0.78	3	0.85
USD/INR	86.40	85.68	−0.72	2	0.79
Repo rate	6.25	6.66	+0.41	4	0.81
Bank NIFTY	51500	51499.81	−0.19	5	0.69
Bank credit growth	13.80	13.60	−0.20	7	0.71
IIP growth	4.20	4.11	−0.09	9	0.59
GDP growth	5.40	5.35	−0.05	10	0.57

Figure 6.4 shows the monthly trajectories produced by the research lab for the same shock. Each series stays at baseline until the cumulative lag of its path and then steps to its delta. The wholesale price effect arrives first and the output effects arrive after

nine to ten months, but the magnitudes decline quickly along the path because every hop multiplies the signal by less than one.

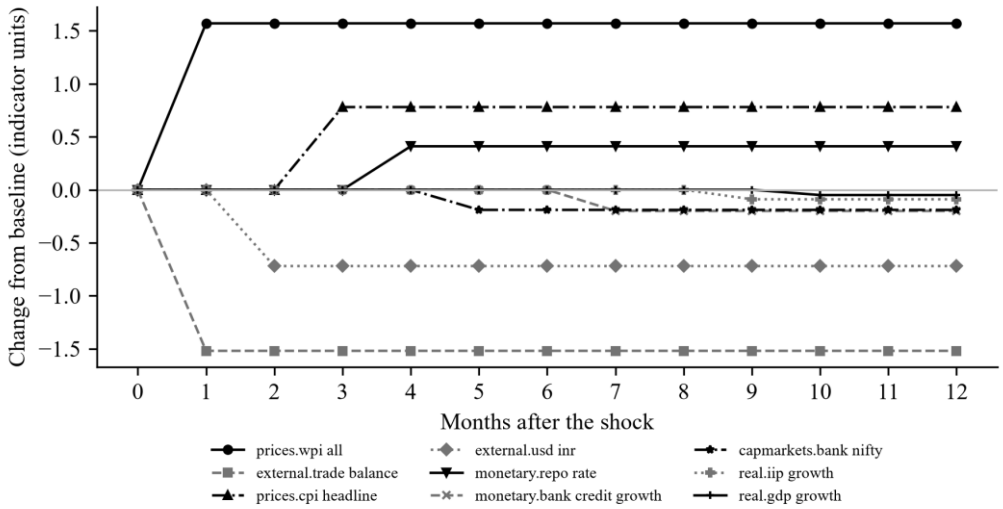


Figure 6.4: Simulated monthly change from baseline after the shock (step response, produced by the Shockwave module).

Two things in this example affect how it should be read. The cumulative confidence drops with every hop (0.95 after one hop to 0.57 after six), so distant effects should be read with more caution. More seriously, USD/INR falls after the oil price rise in this run, which makes no economic sense. The cause is the plus sign on the edge from the trade balance to USD/INR: a wider trade deficit lowers the trade balance, so with a plus sign the rupee-per-dollar rate falls. The code already marks this edge as under review. I kept the example in the report because it is exactly the kind of error the evidence labelling is meant to bring to the surface; fixing the sign is the first item of future work.

When the same shock is run inside the orchestrator with live data, the baselines of the indicators that the sector agents retrieved are replaced. In the end-to-end run of Section 9.4, the live CPI of 5.54 per cent (September 2026) and the repo rate of 5.25 per cent replaced the defaults, giving a simulated CPI peak of 6.32 and repo rate of 5.66 for the same deltas of +0.78 and +0.41.

Chapter 7

Research Lab, Daily Brief and Claim Verification

7.1 INTRODUCTION

Answering questions is only half of what the platform does; the other half is letting the user inspect and test the model behind the answers. The research lab (*core/research*) consists of pure functions over the frozen knowledge graph, the scenario engine and series already held by the dashboard helpers; it never mutates engine state. The daily brief (*core/briefing*) summarises the state of the economy, and the claim verification library (*core/verification*) checks generated statements against evidence. Figure 7.1 shows how the lab modules connect to endpoints and interface tabs. Every payload carries labels (SIM, CACHED, heuristic) so that the interface can distinguish simulated, cached and measured values.

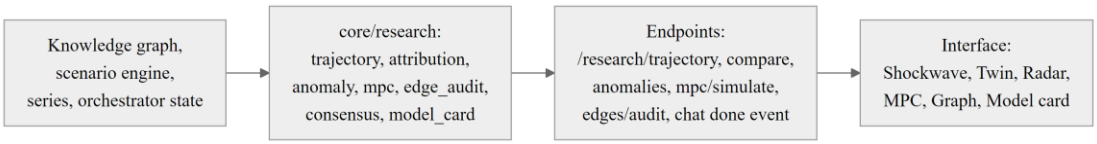


Figure 7.1: Research lab modules, API endpoints and user interface tabs.

7.2 SHOCKWAVE: MONTHLY TRAJECTORIES

For each node the scenario engine gives only a peak delta and a lag. The trajectory module turns these into monthly series over a horizon of one to sixty months. Each series stays at the baseline until the lag month and then either steps to the peak or decays exponentially with a chosen half-life, using baseline + delta × 0.5^((t − lag) / half-life). The module also returns the edges on each path with their evidence class and review notes. The notes state that deltas are in each indicator's unit and that the band is not a statistical interval.

7.3 TWIN: BASE WORLD VERSUS SHOCKED WORLD

The twin module runs the engine twice, with zero shock and with the requested shock, and compares the results node by node. For each target it also attributes the effect to the hops of its path. Each hop receives a share proportional to the logarithm of its attenuation factor (0.55 times its confidence), so the hop with the largest share is the weakest link, which is the hop that attenuates the signal the most. The module counts the inverse (negative-sign) edges on the path as sign flips. Figure 7.2 shows the

attribution for the path from Brent crude to GDP growth: the shares are close to each other (15 to 19 per cent), and the weakest link is the edge from bank credit growth to IIP growth, whose confidence of 0.82 is the lowest on the path.

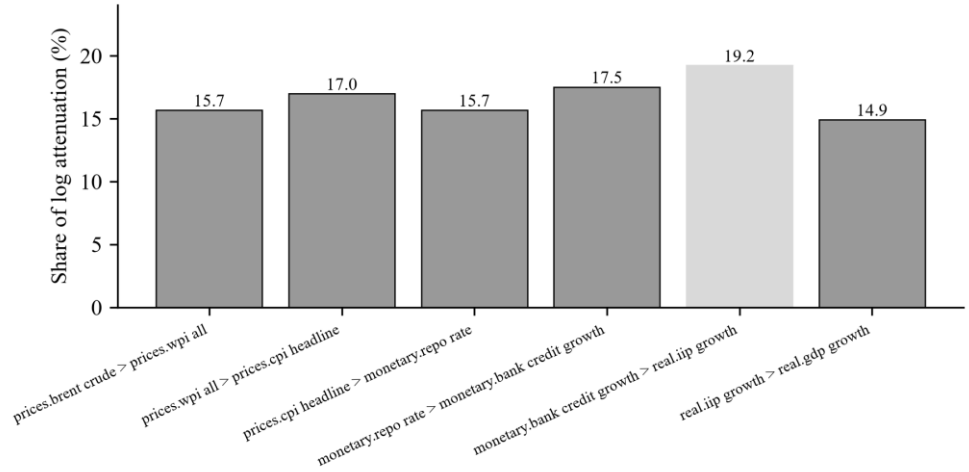


Figure 7.2: Per-hop share of attenuation along the path from Brent crude to GDP growth (Twin module, real output).

7.4 ANOMALY RADAR

The anomaly radar looks for latest readings that are out of line with recent history and then asks the graph whether the neighbouring indicators moved as well. For every series with at least eight points, the latest value is compared with the trailing window of up to twelve observations, excluding the latest point from its own baseline, and a z-score is computed. A reading with an absolute z-score above 2.0 is an anomaly. The module then looks at the graph neighbours of the indicator, upstream and downstream. If at least one neighbour with data is also anomalous the anomaly is classified as propagating, and the culprit path of relation identifiers is returned. If neighbours have data but none is anomalous it is isolated. If no neighbour has data it is unlinked. Isolated and unlinked anomalies are marked for verification of the source because they may be data errors. A simulation mode can inject a synthetic shock into a series to demonstrate propagation.

7.5 EDGE EVIDENCE AUDIT

The edge evidence audit tests whether the seeded edges of the graph are supported by data (Figure 7.3). For every edge it aligns the source and target series by calendar month. With fewer than ten overlapping months the edge remains untestable. Otherwise a bivariate Granger causality test with a maximum lag of two is run, and

the edge is labelled EMPIRICAL with its p-value and optimal lag. Edges that cannot be tested remain SEEDED. Results are cached for five minutes. The audit applies the distinction between evidence typed in by a designer and evidence estimated from data, and the interface shows the label on every edge.

The method has known weaknesses. The minimum of ten months is low for reliable Granger inference, series are not tested for stationarity, and no correction is applied for testing many edges. The planned improvements are described in Chapter 10. Section 9.5 reports the result of running the audit on the data currently available.

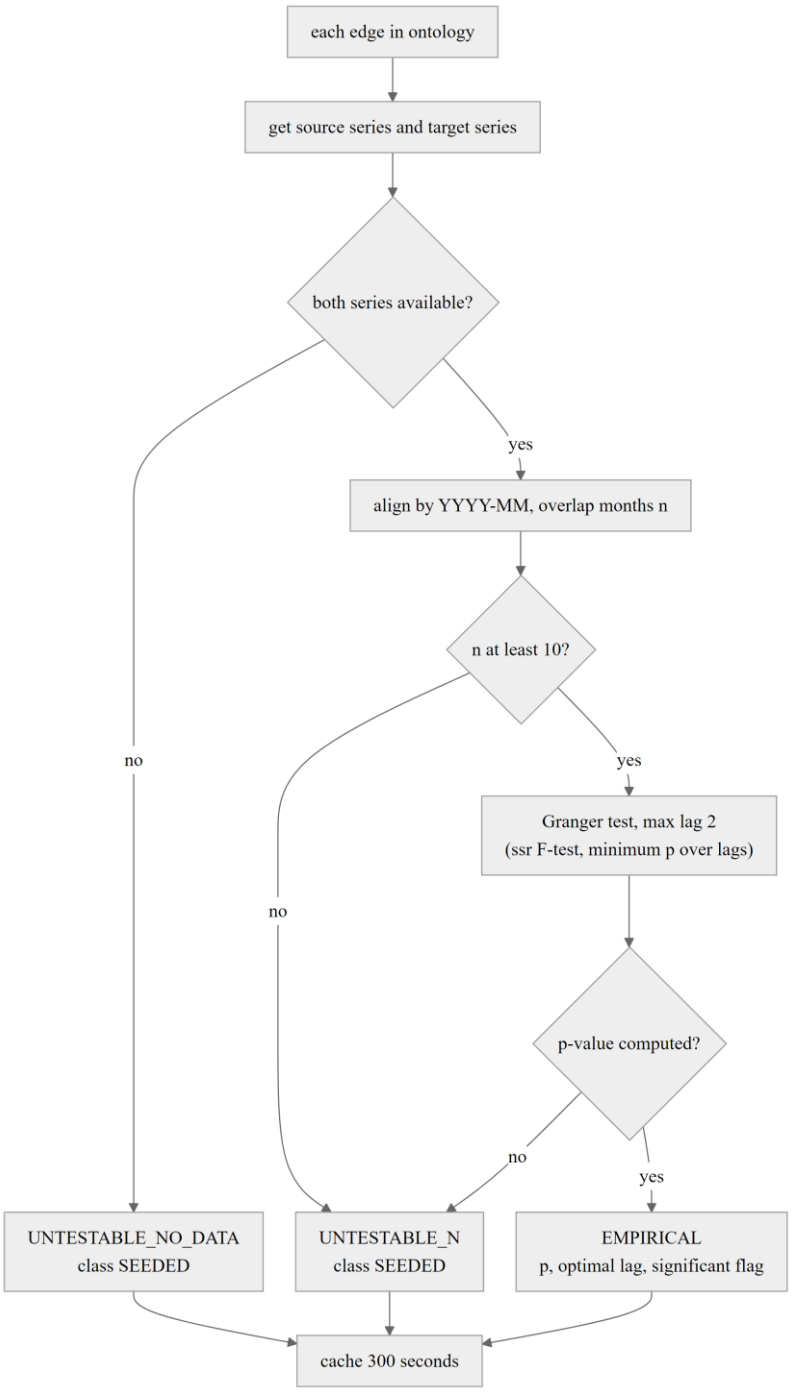


Figure 7.3: Edge evidence audit: seeded priors tested against live series.

7.6 SIMULATED MONETARY POLICY COMMITTEE

The simulated committee has six seats with archetype personas: a centrist governor, a hawkish deputy governor, a centrist officer, one hawkish and two dovish external members. An evidence pack collects the latest CPI, repo rate, GDP growth, USD/INR and bank credit growth from the canonical store and the dashboard series. Each persona first votes by a deterministic rule. A hawk hikes when CPI exceeds 6 per cent and otherwise holds. A dove cuts when CPI is below 2 per cent, or when growth is

weakening and CPI is not above 6 per cent, and otherwise holds. A centrist hikes above 6 per cent, cuts below 2 per cent, cuts when growth is weakening and CPI is below 4 per cent, and otherwise holds. The thresholds are the inflation-targeting band of 4 per cent with a tolerance of 2 per cent on either side.

Figure 7.4 summarises the procedure. When a language model key is available, each of the three persona types additionally produces a vote as structured JSON that must quote a number and cite an indicator from the evidence pack. An answer that fails this validation is discarded and the rule vote is kept. The six seats share the three persona results, so the committee has three distinct voices rather than six independent members. The decision is the majority, with the governor's vote deciding a tie. The rule-only decision is always reported next to the final one, which makes it possible to measure what the language model adds. The meeting is always labelled as simulated and is not a forecast of an actual decision.

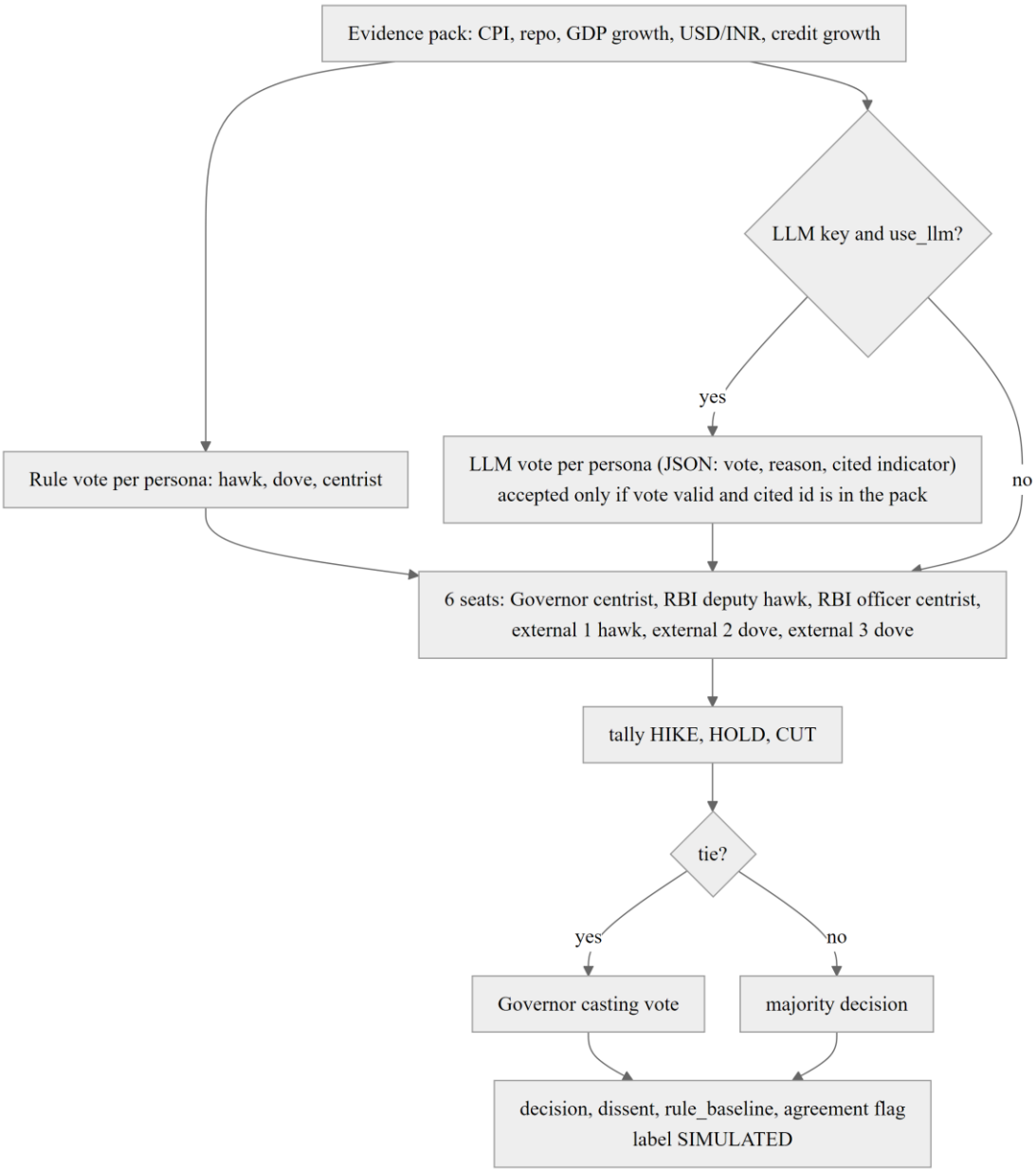


Figure 7.4: Simulation of a six-seat Monetary Policy Committee with a deterministic rule baseline.

7.7 CONSENSUS AND MODEL CARD

Two summaries are attached to every orchestrated answer. The consensus summary reports, per agent, the status, duration, number of observations and number of usable observations, and computes three fractions: agent success (success counts as 1 and partial as 0.5), evidence cover (usable observations over all observations) and knowledge-graph cover (connected pairs over all pairs of graph indicators). Their product, with the graph cover weighted as 0.5 + 0.5 times the cover, is reported as consensus quality. The model card reports the evidence counts, the relations used with their evidence class, the scenario formula with its constants, an inflation regime note

and a trust score of 0.4 times evidence cover plus 0.3 times mean edge confidence plus 0.3 times agent success. These scores are labelled heuristic because their weights are design choices and have not been calibrated against verification outcomes.

7.8 DAILY EARLY-WARNING BRIEF

The daily brief reads telemetry for four pillars (monetary, fiscal, external and markets), runs the anomaly scan, fetches capped and cached news with at most four queries and a six-hour cache, and makes at most one language model call. If the model is unavailable, or returns invalid JSON, a deterministic template is used. Briefs are stored by Asia/Kolkata date in the canonical DuckDB store and can be generated on demand or by an optional scheduler at 08:15. A debounce interval of fifteen minutes prevents repeated runs unless forced.

7.9 CLAIM VERIFICATION

Grounding still leaves one gap open: nothing so far checks that a figure written in the generated text is the same as the figure that was retrieved. The claim verification library closes that gap. It was first built for the prices sector and is implemented as a shared library (Figure 7.5). Retrieved observations and source passages are frozen into content-addressed evidence: the identifier of an observation is the SHA-256 digest of its canonical form without volatile fields, so repeated retrievals of the same published value share an identifier while a revision or definition change receives a new one.

Claims are proposed in a strict schema with six types: an observation value, a rate change between two observations, the direction of a year-on-year price change, a reported explanation quoted from a retrieved passage, a reviewed general mechanism and a statistical finding. Extra fields are forbidden, so a model cannot introduce a new claim type, a formula or free text. Numerical claims are recomputed with exact decimal arithmetic and compared at the displayed precision. The checks cover identifiers, finite values, monthly periods, units, geography, sector, series, source, index base, period order, conflicting observations and the requested scope. A quoted explanation is accepted only if it is a complete sentence that occurs verbatim in the retrieved passage, and even then it is rendered as “source attributed” and the text states that causation is not established.

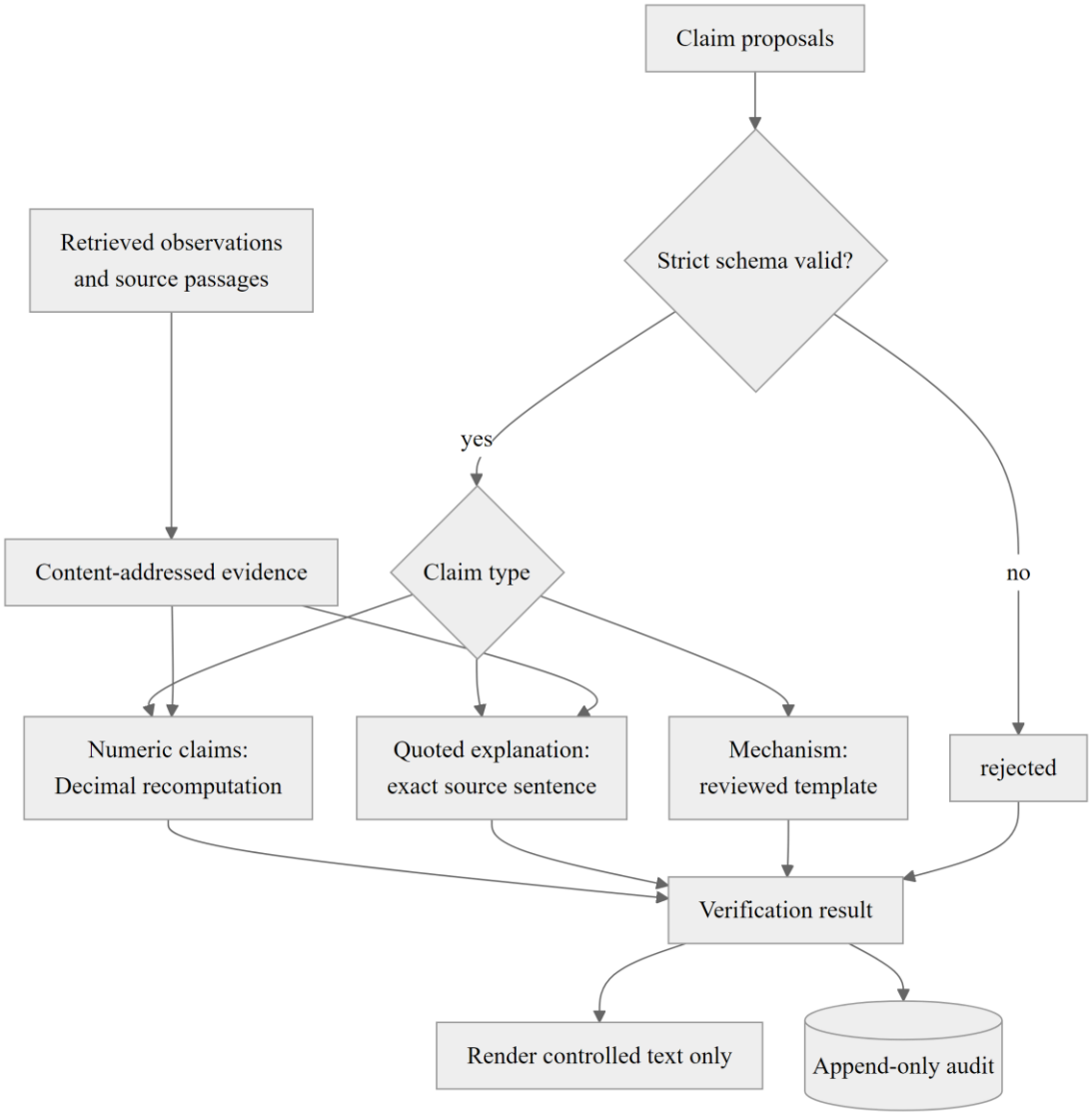


Figure 7.5: Claim verification pipeline from retrieved evidence to rendered text and audit record.

Each claim ends in one of five statuses: verified, source attributed, general mechanism, rejected or insufficient evidence. Only verified, source-attributed and general-mechanism results are rendered. Rejected claims and claims with insufficient evidence are withheld, counted and kept in an append-only SQLite audit that stores the complete evidence snapshot, proposals and results. The library is covered by tests that include altered quotations, wrong arithmetic, mismatched index bases and conflicting observations.

There is an important catch. The library is not yet connected to the answer path: the pipeline that proposed claims for the prices agent was removed from that agent, and neither the orchestrator nor any sector currently calls the verifier. The planned integration (Figure 7.6) makes the language model return structured claims, verifies

them against the A2A evidence, renders only verified statements and hedges causal claims according to the evidence class of the edge on which they rely.

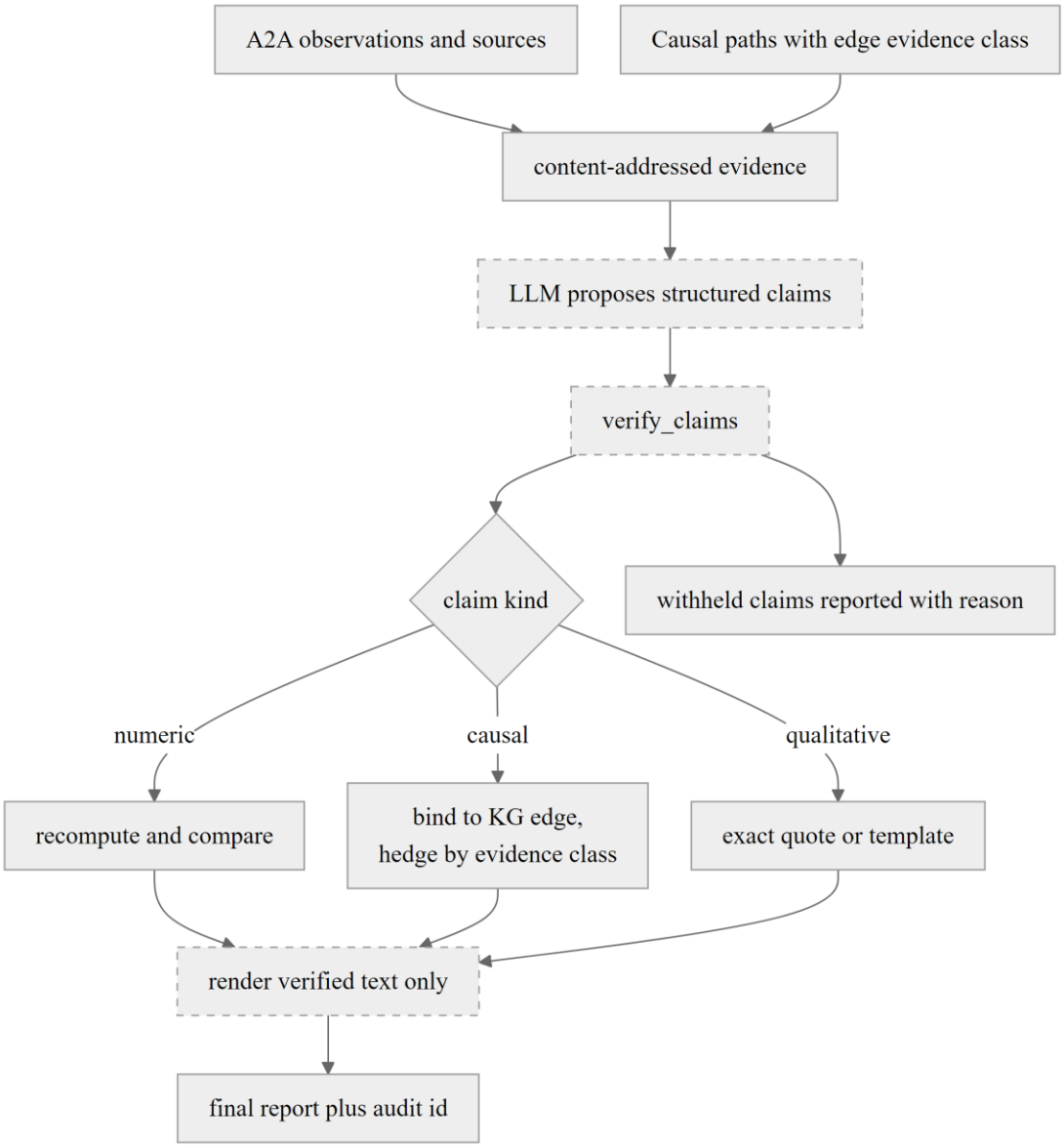


Figure 7.6: Proposed integration of the claim verifier into orchestrator synthesis (not yet implemented).

Chapter 8
User Interface and API

8.1 INTRODUCTION

Users reach the platform through a web application, and other programs through an HTTP interface; both are described here. The screenshots were captured from the running application on 13 October 2026 with the language model and search keys disabled, so the system operated in its deterministic offline mode and the data shown is the sector data held by the platform at that time (live where the sources responded and cached otherwise, as tagged in the interface).

8.2 FRONTEND ARCHITECTURE

The frontend is a React 18 application written in strict TypeScript, bundled with Vite and styled with Tailwind CSS. The root component keeps track of two things only: the signed-in user and the current view. An unauthenticated visitor sees the landing page; after Google sign-in, or when a stored session exists, the sidebar switches among the dashboard, the agent research workspace, the causal transmission graph with the causal lab, and the A2A agent registry. Sign-out clears the stored profile.

The component hierarchy is shown in Figure 8.1. The dashboard is split into eight sector tabs, studio widgets (scenario simulator, cross-sector studio, anomaly feed and early-warning brief) and reusable charts. The chat workspace renders streaming markdown, scrollable tables, Mermaid diagrams, the A2A trace panel, the debate graph and the model card strip. The causal lab is a tab shell around the knowledge-graph view and the four research tabs. Network requests use an *AbortController*, so a new question cancels the previous one.

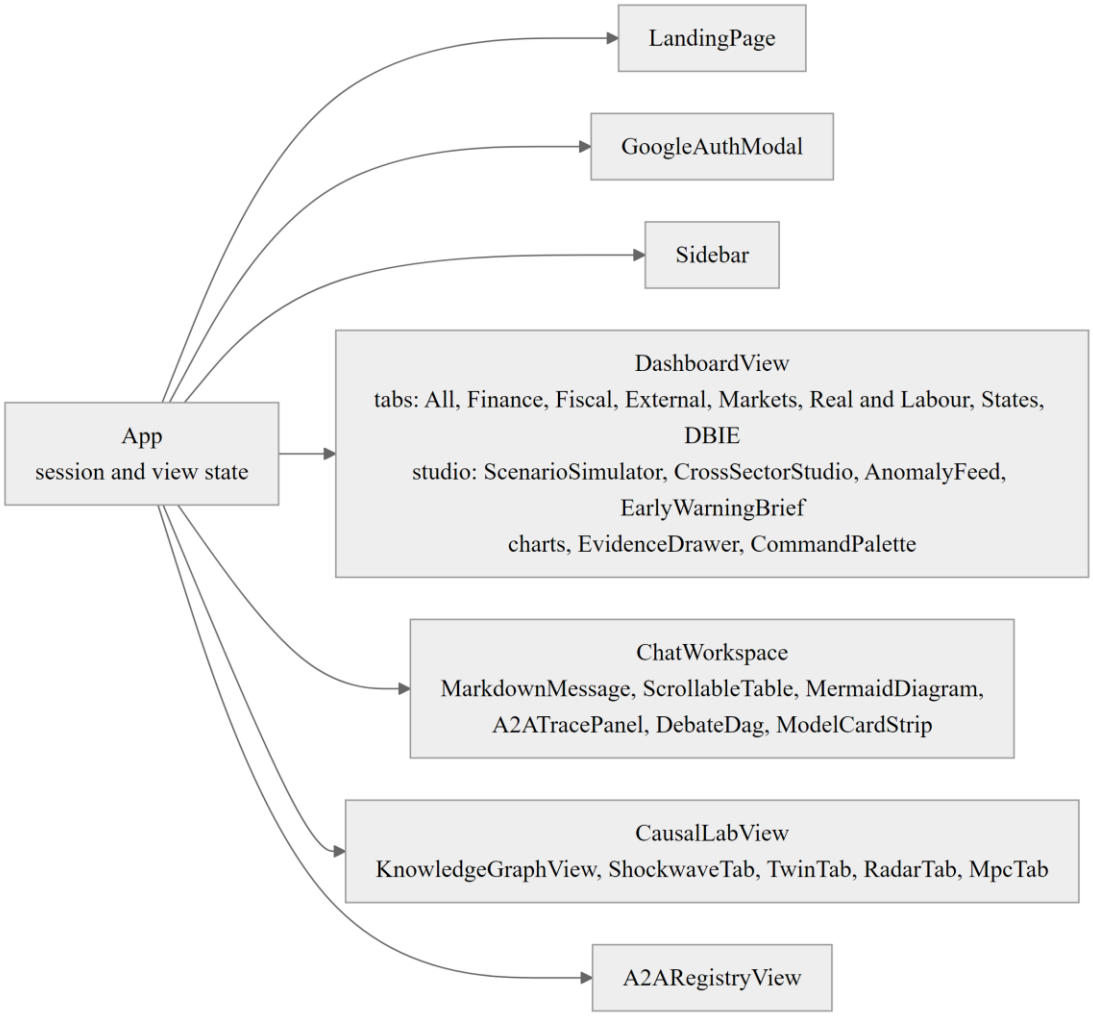


Figure 8.1: React component hierarchy of the application.

8.3 LANDING PAGE AND AUTHENTICATION

Figure 8.2 shows the landing page. It presents the platform and its dual-protocol design and offers Google sign-in. Sign-in uses Google Identity Services. The decoded profile is stored in the browser's local storage, and there is no server-side session; consequently the gateway endpoints themselves are not protected, which is listed as a limitation in Chapter 9.

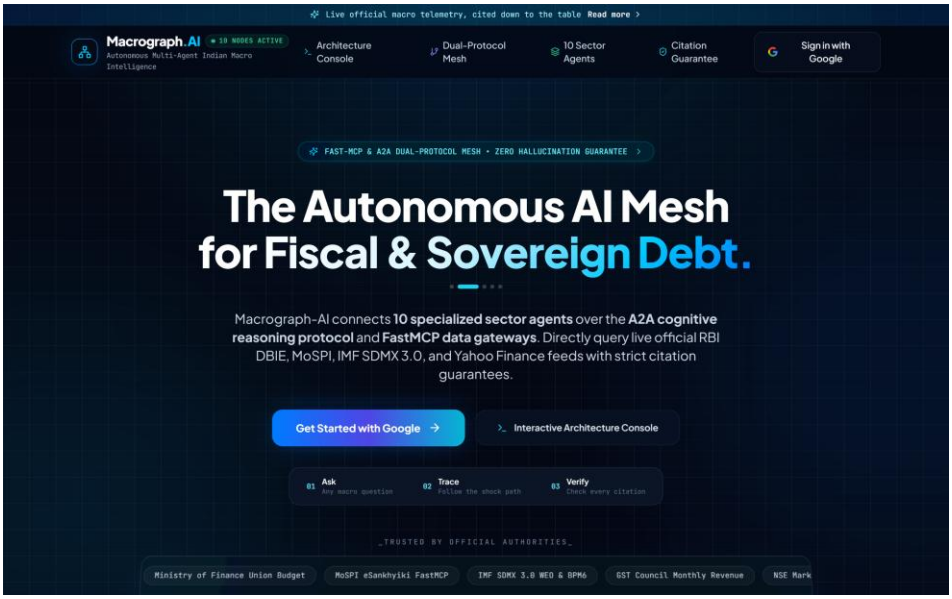


Figure 8.2: Landing page of Macrograph AI.

8.4 DASHBOARD

The dashboard aggregates telemetry from the sector stores, open datasets and the RBI DBIE explorer into eight tabs. Each indicator keeps its citation, and clicking a value opens an evidence drawer with the source, table, period and calculation. Figure 8.3 shows the first tab, which combines the ticker bar of policy rate, yields and exchange rate with the early-warning brief panel and the cross-sector summary. The finance and banking tab (Figure 8.4) adds credit expansion, asset quality gauges and lending-rate spreads.

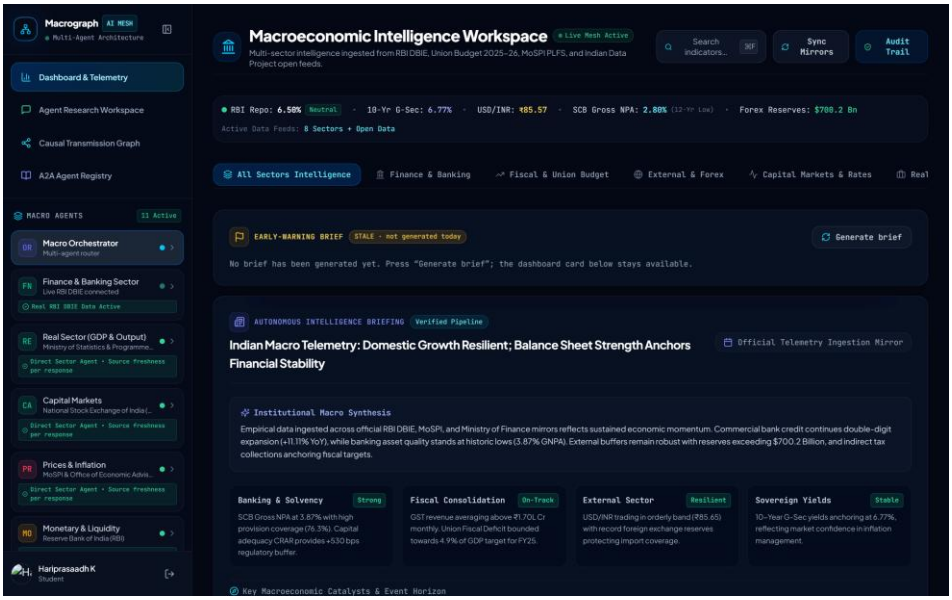


Figure 8.3: Dashboard, all-sectors tab with ticker bar and early-warning brief panel.

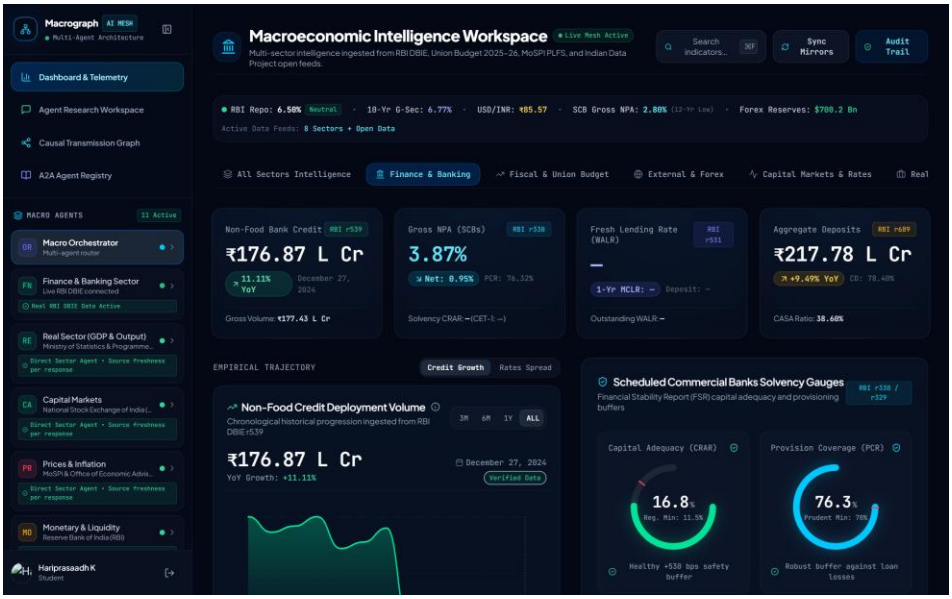


Figure 8.4: Dashboard, finance and banking tab: credit expansion, asset quality gauges and lending rates.

8.5 AGENT RESEARCH WORKSPACE

The workspace is a conversational interface to the orchestrator and to each sector agent. The sequence in Figure 8.5 shows the streaming protocol. The browser posts the message and the selected agent and reads the response stream. Step events report which agent or tool is running and are updated in place by request identifier; token events carry the report text; and the final event carries the complete report, citations, the A2A summary, consensus summary and model card. The workspace then renders tables, the causal diagram, the trace panel and the model card.

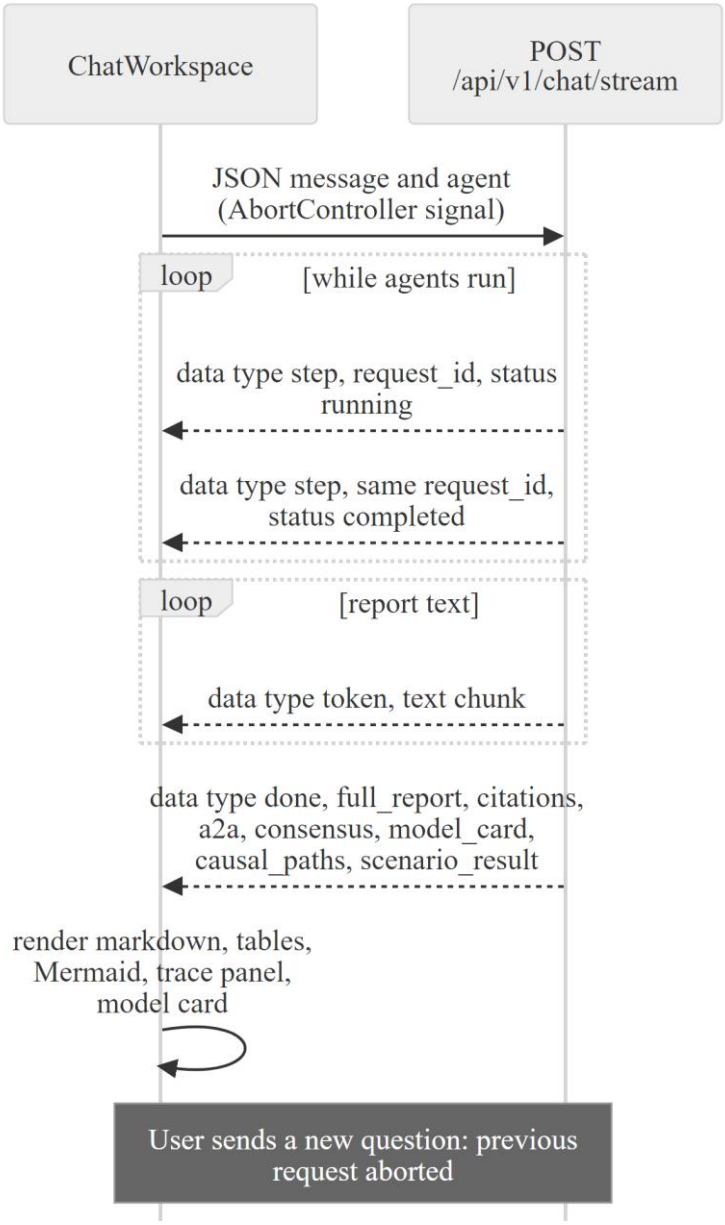


Figure 8.5: Server-Sent Events protocol between the chat workspace and the gateway.

8.6 A2A AGENT REGISTRY

Figure 8.6 shows the registry view. It lists the Agent Cards returned by the registry endpoint, with each agent's domain, supported tasks, skills and verification status, together with the declared dependency edges.

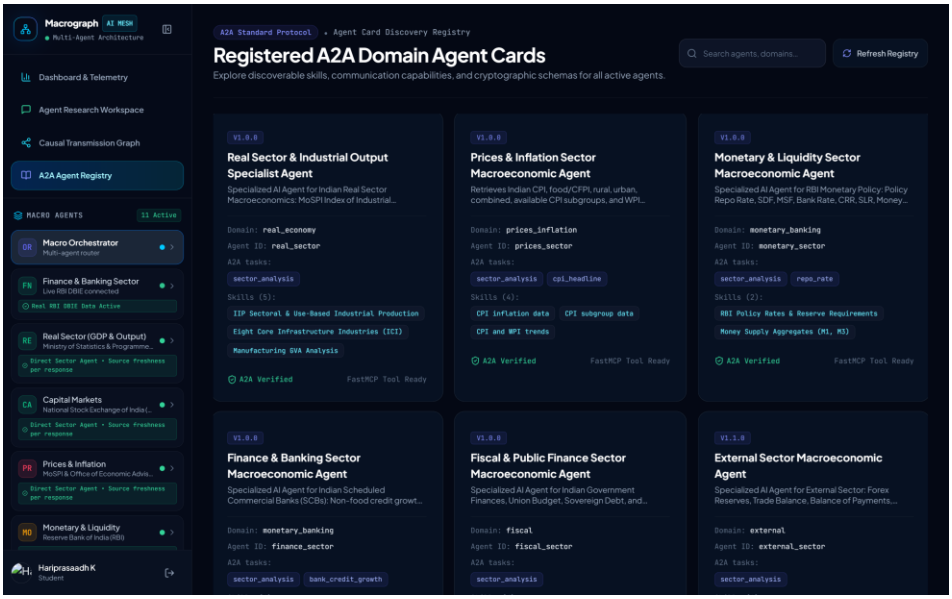


Figure 8.6: A2A agent registry view listing the Agent Cards of the sector agents.

8.7 CAUSAL LAB

The causal lab has five tabs backed by the research endpoints of Chapter 7. The path tab (Figure 8.7) queries the transmission path between any two indicators and lists the relations with their lag, sign, type and confidence.

The Shockwave tab animates the monthly propagation of a shock through the graph with adjustable magnitude and response shape (Figure 8.8). The Twin tab compares baseline and shocked worlds and shows the attribution table. The Radar tab lists anomalies and their classification. The MPC tab runs the simulated committee and shows each seat's vote, the tally, the dissent and the rule baseline (Figure 8.9). The labels on every panel mark the data as simulated or cached.

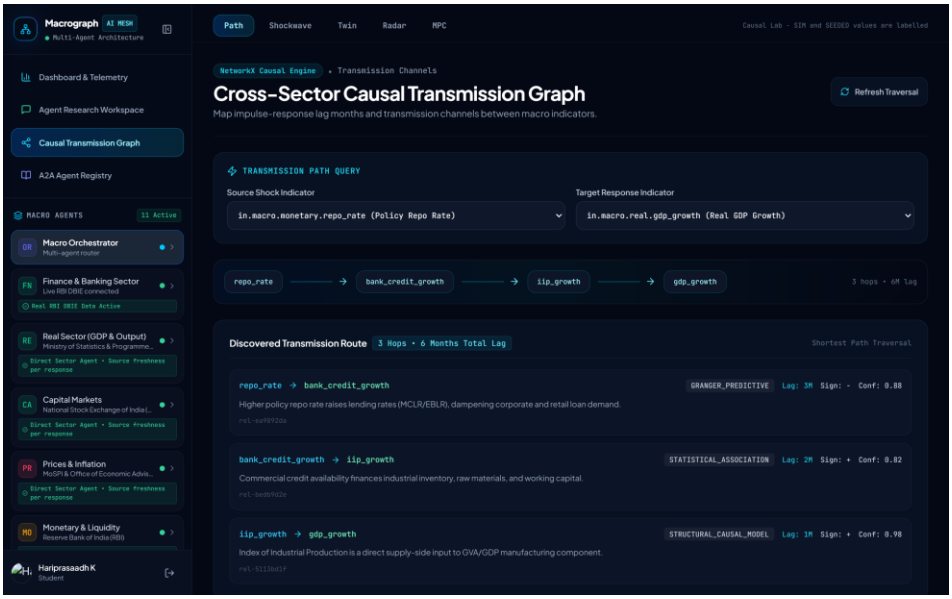


Figure 8.7: Causal lab, path tab: transmission path from the repo rate to GDP growth.

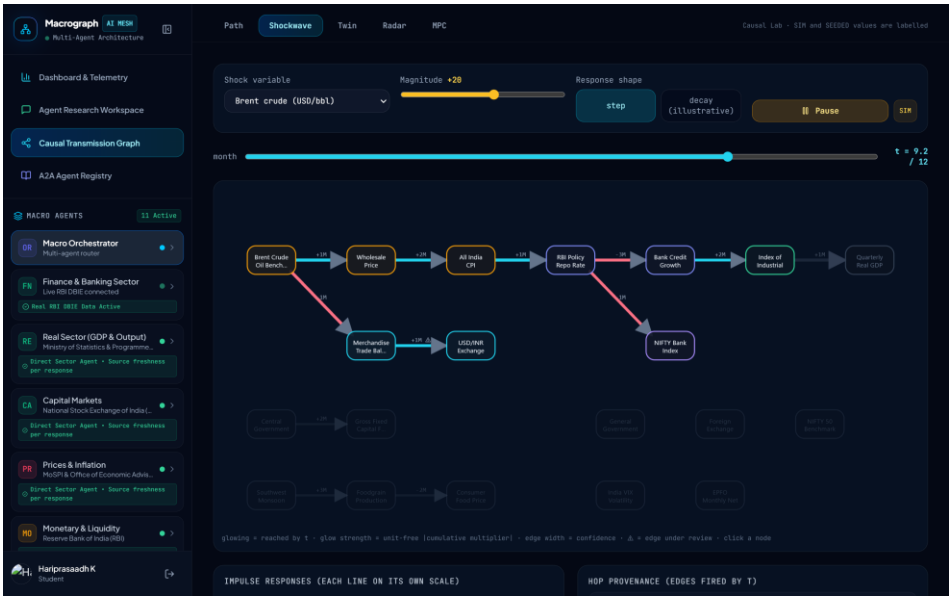


Figure 8.8: Causal lab, Shockwave tab: animated propagation of a Brent crude shock.

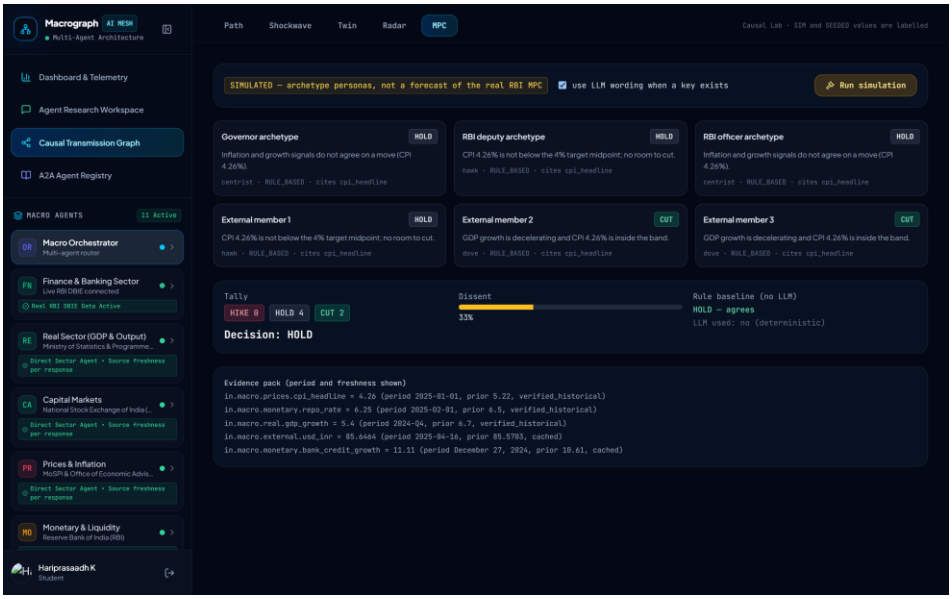


Figure 8.9: Causal lab, MPC tab: simulated six-seat committee labelled as simulated.

8.8 HTTP INTERFACE

The gateway exposes the interface summarised in Table 8.1. Interactive documentation is generated by FastAPI. The research and brief endpoints call paid language-model and search services when keys are configured and are unauthenticated in the current version.

Table 8.1: Principal HTTP endpoints

Endpoint	Purpose
POST /api/v1/analyze	Full orchestrator run with optional scenario shock
POST /api/v1/chat/stream, POST /api/v1/chat	Streaming and synchronous chat with direct or orchestrated routing
POST /api/v1/simulate	Scenario shock through the knowledge graph
GET /api/v1/kg/path, GET /api/v1/kg/impacts	Transmission path and downstream impacts
GET /api/v1/dashboard/overview	Cross-sector telemetry with citations
GET /api/v1/research/kg/graph, /edges/audit	Graph for display and edge evidence audit
POST /api/v1/research/trajectory, /compare	Shockwave trajectories and twin comparison
GET /api/v1/research/anomalies	Anomaly radar, optional simulated injection
POST /api/v1/research/mpc/simulate	Simulated committee
GET /api/v1/brief/today, POST /brief/run, GET /brief/history	Daily early-warning brief
GET /a2a/registry, /a2a/v1/agents, /capabilities, /traces	Agent Cards, capabilities and conversation traces
GET /health	Status, number of agents, graph size, Neo4j flag

Chapter 9
Testing, Results and Discussion

9.1 INTRODUCTION

All measurements in this chapter come from running the repository on 12 and 13 October 2026. Several of them reveal weaknesses, and these are reported alongside the successes. The comparative evaluation against baselines has not been carried out yet; Section 9.7 describes the plan, and nothing in that section should be read as a result.

9.2 TESTING STRATEGY

Tests are organised in four layers. Unit tests cover parsers, validators, the claim verification library and sector clients and databases. Component tests cover the A2A protocol and its hardening, the knowledge-graph schema and engine, a mocked Neo4j store and the causal engine. Integration tests cover A2A delegation and streaming, the orchestrator with the knowledge graph, and the dashboard and chat interfaces, using stub handlers and a mocked language model. A final layer is a manual system demonstration with *demo_platform.py* and a sector validation script. Changes pass the quality gates of four kinds: automated tests, a scan of the engineering rules, the frontend build and the demo.

9.3 TEST RESULTS

The repository contains 455 test functions, which execute as 551 test cases after parametrisation. Figure 9.1 and Table 9.1 show their distribution across areas. The core and cross-cutting suite is the largest, followed by agriculture, external, monetary, services and finance.

Table 9.1: Test functions by area (12 October 2026)

Area	Test functions
Core and cross-cutting (A2A, orchestrator, knowledge graph, causal engine, verification, dashboard)	198
Agriculture sector	48
External sector	45
Monetary sector	43
Services sector	33
Finance sector	32
Capital markets sector	27
Fiscal sector	15
Labour sector	7
Real sector	7
Total	455

The complete run of all sector test directories took 11 minutes 55 seconds and produced **550 passed and 1 failed**. The failure is the monetary-sector test that checks the fallback from the policy-rate fetcher to the stance fetcher: it expects a repo rate of 5.0 from its stub but the agent returned 5.25, which means that the agent now prefers another source over the stubbed fallback. It is a drift between the test and the code, not an observed failure of the running system, and it is recorded as an open item. The default *pytest* invocation collects 374 tests, because the other sector test directories are not listed in the project's test paths; running all directories is required to obtain the figures above. A repeat of the full run on 17 October 2026 gave 548 passed and 3 failed. Apart from the same monetary test, one real-sector test timed out waiting for the RBI DBIE MCP service, and an external-sector test named as mocked reached the live Tavily search API and was refused with HTTP status 432. Both of these depend on outside services, which shows that a few tests are not yet isolated from the network.

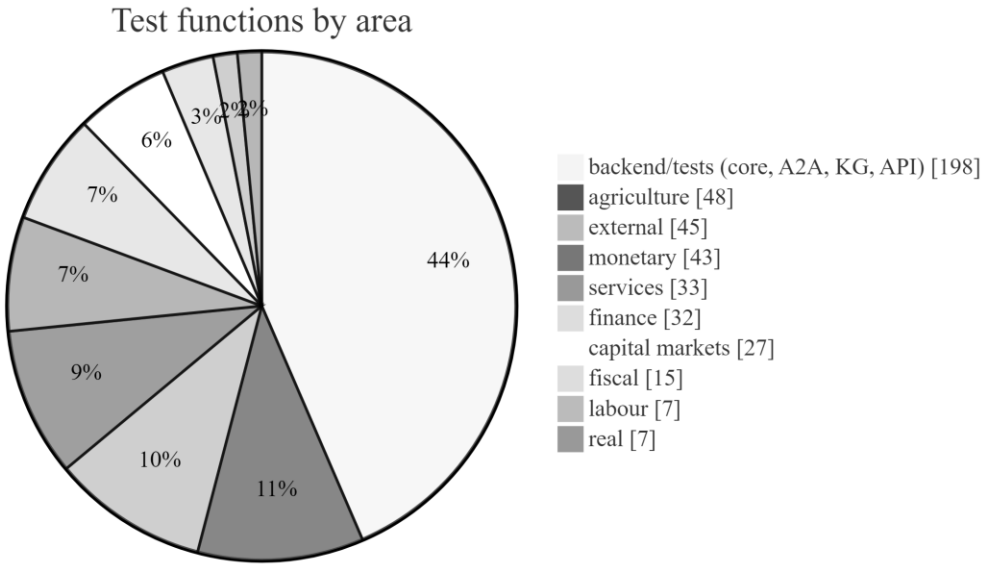


Figure 9.1: Number of test functions per test area (455 functions).

9.4 END-TO-END ORCHESTRATOR RUN

One cross-sector question was run through the full orchestrator with a crude oil scenario: “How does a rise in crude oil prices affect inflation and RBI repo rate policy?” with a shock of plus 20 US dollars per barrel on Brent crude. The run used live official sources and the deterministic offline mode (no language model key), and completed in about eleven seconds. The router selected the monetary, prices and agriculture agents. Table 9.2 lists the observations returned, and the report is reproduced in Appendix 3.

Table 9.2: Observations collected in the end-to-end run

Indicator	Value	Period	Source / status
CPI headline (YoY %)	5.54	September 2026	MoSPI, official
Repo rate (%)	5.25	February 2026	RBI DBIE
M3 growth (YoY %)	unavailable (None)	30 June 2026	RBI DBIE

The knowledge graph returned three transmission paths among the observed indicators: CPI to repo rate (1 hop, 1 month), Brent to CPI (2 hops, 3 months) and Brent to repo rate (3 hops, 4 months). The scenario used the live CPI and repo rate as baselines and the defaults for all other indicators. The report contained the indicator matrix with provenance hashes, a causal diagram, the path and scenario tables, a source provenance table and the statement that documentary evidence is unavailable. The run confidence score, which is the fraction of observations with usable status, was 1.0.

The run also exposed three defects. First, the M3 growth observation had no value, yet the indicator matrix printed it as “None%” instead of marking it unavailable. Second, the router included the agriculture agent for a question about crude oil, inflation and the repo rate, which shows that the keyword scoring is too permissive for that agent. Third, when no language model key is configured the prices agent logs a configuration error and continues with official observations; the behaviour is correct but the error log is misleading. These defects are recorded in the project's status document.

9.5 EDGE EVIDENCE AUDIT RESULT

The edge evidence audit was run on the series available in the running system. Table 9.3 and Figure 9.2 show the result: **none of the twelve edges could be tested empirically**. Seven edges had fewer than ten overlapping months (between zero and two), and five had no series for one of their endpoints. All twelve edges therefore remain SEEDED, and the p-values stored in the ontology are priors that the platform cannot currently confirm or reject. This is the most important finding about the causal model: the graph is a documented set of assumptions, and the scenario outputs depend on them.

Table 9.3: Result of the edge evidence audit

Edge	Overlap (months)	Status
Brent crude to WPI	0	Untestable (too few months)
WPI to CPI headline	1	Untestable (too few months)
CPI headline to repo rate	1	Untestable (too few months)
Repo rate to bank credit growth	0	Untestable (too few months)
Bank credit growth to IIP growth	0	Untestable (too few months)
IIP growth to GDP growth	0	Untestable (no data)
Brent crude to trade balance	1	Untestable (too few months)
Trade balance to USD/INR	2	Untestable (too few months)
Repo rate to bank NIFTY	0	Untestable (no data)
Central capex to GFCF investment	0	Untestable (no data)
Monsoon departure to foodgrain production	0	Untestable (no data)
Foodgrain production to CPI food	0	Untestable (no data)

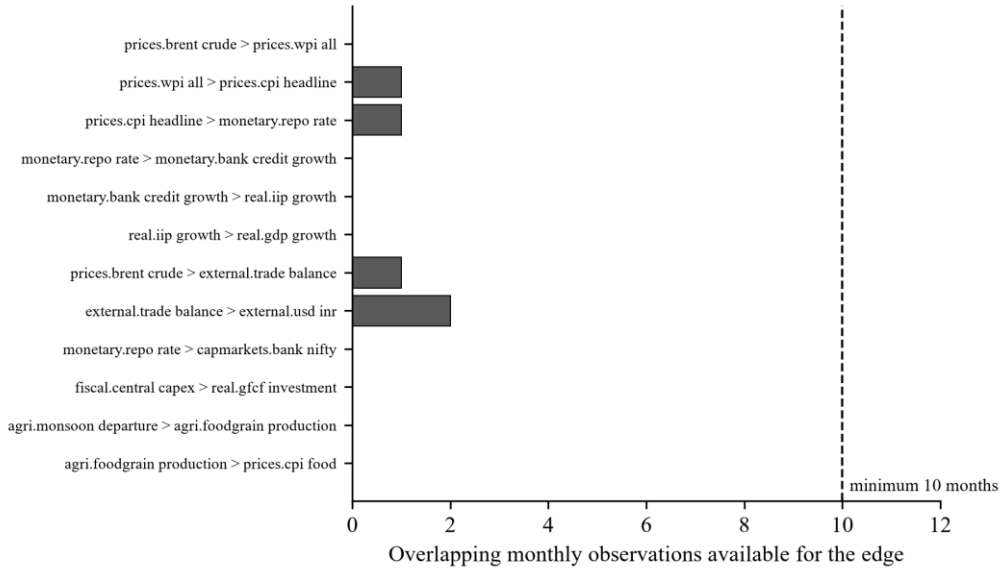


Figure 9.2: Overlapping monthly observations available for each causal edge (minimum required: 10).

9.6 ANOMALY RADAR AND SIMULATED COMMITTEE

The anomaly radar scanned 18 series. Ten of them were skipped because they had fewer than eight observations (between one and three points for GDP growth, IIP growth, CPI, WPI, Brent crude, the repo rate, debt-to-GDP, NIFTY 50, foodgrain production and EPFO additions). Of the eight series analysed, one exceeded the threshold: the merchandise trade balance for August 2024 with a z-score of −2.65. Its downstream neighbour, USD/INR, had a z-score of only 1.16, so the anomaly was classified as isolated and marked for source verification. The result shows that the radar works as designed but that the available history is too short for most indicators to be scanned.

The simulated committee (Table 9.4) voted four to two to hold, with two dovish external members voting to cut because GDP growth was decelerating while CPI was inside the band. The rule-only decision was the same (hold) and the dissent was 0.33. The evidence pack used the canonical store, whose latest CPI was 4.26 per cent for January 2025 and whose repo rate was 6.25 per cent for February 2025, while the end-to-end run above retrieved live values of 5.54 per cent and 5.25 per cent. The simulated meeting is therefore based on older data than the live agents, a weakness that must be removed before the simulator is evaluated against actual decisions.

Table 9.4: Votes of the simulated committee (rule-based mode)

Seat	Persona	Vote	Reason (rule)
Governor	Centrist	HOLD	Inflation and growth signals do not agree on a move (CPI 4.26%).
RBI deputy	Hawk	HOLD	CPI 4.26% is not below the 4% target midpoint; no room to cut.
RBI officer	Centrist	HOLD	Inflation and growth signals do not agree on a move (CPI 4.26%).
External member 1	Hawk	HOLD	CPI 4.26% is not below the 4% target midpoint; no room to cut.
External member 2	Dove	CUT	GDP growth is decelerating and CPI 4.26% is inside the band.
External member 3	Dove	CUT	GDP growth is decelerating and CPI 4.26% is inside the band.

9.7 PLANNED EVALUATION

The measurements above show that the system works, but they do not show that it is better than alternatives. The planned evaluation uses a question set of about 250 questions (point lookups, changes, cross-sector syntheses, causal and scenario questions and unanswerable questions) whose gold answers are computed from frozen official data. Five systems are compared: a single language model, a model with web search, retrieval-augmented generation, the multi-agent system without verification and the multi-agent system with verification. The principal metrics are the unsupported numeric claim rate, citation validity, abstention correctness, coverage, latency and cost, reported as means with bootstrap intervals over repeated runs and at least two language model backends.

Ablations remove one mechanism at a time: the ownership constraint, claim verification, edge evidence grading, the knowledge graph, and the multi-agent decomposition. A second planned benchmark compares the simulated committee with actual decisions and vote splits using point-in-time data.

9.8 LIMITATIONS

The current system has the following limitations, roughly in order of how much they affect the credibility of its output.

1. The edge confidences and p-values are curated priors, and Section 9.5 shows that none of them can be tested on the data available today. Every scenario output depends on them.

2. The scenario engine is a heuristic that ignores units: its constants are design parameters, each delta is in its indicator's own unit and the band is not a statistical interval. One edge sign is known to be doubtful.
3. The claim verification library is tested but the orchestrator does not call it, so generated prose is not machine-checked.
4. Many series have fewer than eight observations, which restricts both the radar and the audit.
5. Policy documents are not indexed, and every report says so.
6. The research and brief endpoints, which can incur language-model and search costs, have neither authentication nor rate limits.
7. No formal comparison with baselines has been done.
8. Some engineering rules are broken: nine non-test modules are longer than 400 lines, the frontend still contains some *any* types and inline styles, and one test is out of date.

9.9 LESSONS LEARNED DURING DEVELOPMENT

A few problems cost noticeably more time than expected, and they are worth recording because they are not obvious from the code. The most misleading was a test-suite issue. Tests that replaced the language model client with a mock passed, while the real client had an import error that only appeared when no mock was installed. A regression test that runs the no-key path without any mock now guards against this. A similar blind spot existed in the test configuration: the *testpaths* setting listed only some sector directories, so for a while most sector suites were silently not being collected. Comparing *--collect-only* counts after adding a directory is now part of the routine, and it is also the reason the default run collects 374 tests while the full run executes 551.

Two smaller issues were environmental. Importing statsmodels takes around thirty seconds on a cold start on the development machine, which at first looked like a hung process. In the knowledge-graph package, the module *neo4j_store* is shadowed by a singleton object of the same name re-exported from the package, so tests have to import the submodule explicitly through *importlib*. The broadest lesson concerned documentation. Early design documents and the README described PydanticAI, a Qdrant vector store, a library-based A2A implementation, vector autoregression and a

live verification layer, none of which existed in the code at the time. Each claim in this report was therefore checked against the repository before it was written down.

9.10 DISCUSSION

Three conclusions can be drawn. The engineering goals were met: ten sector agents, a typed protocol with guards and provenance, an orchestrator that degrades gracefully, and an interface that exposes sources and uncertainty. A single question produced a cited report in about eleven seconds with live data and no language model. The platform is also open about what it does not know: the audit reports that no edge can be tested, the radar reports which series are too short, and the simulated committee is labelled simulated. The main scientific claims, on the other hand, are not yet established. The causal graph rests on assumptions, the scenario engine is a heuristic, and the verification layer is not connected to the answers. Chapter 10 sets out the work needed to turn these into evaluated contributions.

Chapter 10
Conclusion and Future Work

10.1 SUMMARY OF THE WORK

This project designed and implemented Macrograph AI, a multi-agent platform for Indian macroeconomic intelligence. The economy is decomposed into ten sector agents, each owning its indicators and obtaining them from official sources through its own MCP server. The agents cooperate through a native typed agent-to-agent layer that enforces declared dependencies, rejects loops and refuses successful responses without sources. A four-node LangGraph orchestrator routes questions over Agent Cards, collects observations, maps transmission paths in a causal knowledge graph of twenty indicators and twelve edges, runs an optional scenario, and produces a report in which every observation carries its period, status and a provenance hash. When data or a language model is unavailable the system degrades to cached values, structured unavailable statuses and deterministic summaries instead of invented figures. A web application provides a cited dashboard, streaming chat, an A2A registry and an interactive causal lab.

The project also produced research instruments: an edge evidence audit, an anomaly radar, trajectory and twin views, a simulated Monetary Policy Committee with a rule baseline, a daily early-warning brief and a claim verification library. The implementation comprises about 37,000 lines of Python and 11,300 lines of TypeScript and is covered by 455 test functions, of which the complete run passed 550 of 551 executed cases.

10.2 ACHIEVEMENT OF THE OBJECTIVES

Table 10.1 compares the delivered work with the objectives defined in Chapter 1.

Table 10.1: Status of the project objectives

Objective	Status	Evidence
O1 Sector-wise data access	Delivered	Ten MCP servers and local stores (Chapters 3 to 5)
O2 Coordinated multi-agent reasoning	Delivered	A2A layer with guards and trace; orchestrator; end-to-end run in Section 9.4
O3 Source-grounded answers	Partly delivered	Provenance enforced at protocol level; claim verification not yet in the answer path
O4 Explicit causal model	Partly delivered	Graph and scenario engine delivered; edge statistics are priors and the engine is heuristic
O5 Traceable provenance	Delivered	SourceProvenance, provenance hashes, conversation traces
O6 Research instruments	Delivered as prototypes	Audit, radar, trajectory, twin, MPC, brief; results in Sections 9.5 and 9.6
O7 Usable interface	Delivered	Dashboard, chat, causal lab and registry (Chapter 8)

10.3 CONTRIBUTIONS AND INSIGHTS

On the engineering side, the project delivered a complete and tested platform in which indicator ownership, protocol-level provenance and graceful degradation were part of the design from the beginning. What I learned most from, though, were the negative findings. The edge audit shows that, with the data currently available, none of the twelve causal edges can be confirmed, which makes explicit that the knowledge graph is a set of documented assumptions. The worked example exposes a doubtful edge sign through an implausible exchange rate response. The end-to-end run exposes display and routing defects. Measuring and reporting these facts instead of presenting the platform as finished follows from the rule the whole system is built on: a statement without a source must not be presented as fact.

10.4 FUTURE WORK

Reviewing the implementation produced a list of further work, given here in order of priority.

The most urgent task is to connect claim verification to the answers. The orchestrator's language model should return structured claims, which are then verified against the A2A evidence. Only verified statements would be shown, withheld claims would be counted in the report, and causal claims would be hedged according to the evidence class of the edge they rely on (Figure 10.1).

Next comes the causal model. The sign of the trade balance to USD/INR edge has to be corrected, and the typed confidences and p-values should be replaced by estimates from long monthly series, using stationarity checks, Granger tests with false-discovery-rate control, local projections and stability tests, so that the report can say which of the seeded edges survive. The scenario heuristic itself should then give way to impulse responses estimated with local projections or a Bayesian vector autoregression, tested on episodes such as the 2022 oil price shock and the 2022-23 policy rate cycle, with the present engine kept as a baseline.

With those pieces in place, the evaluation of Chapter 9 can be built: the question set, the baselines and ablations, and a point-in-time comparison of the simulated committee with actual decisions. Documentary evidence is a separate decision. Either RBI minutes and the Economic Survey are indexed with passage-level attribution, or all retrieval language is removed from the system for good.

The remaining items are engineering work. Cost-bearing endpoints need authentication and rate limits. Database initialisation should move from import time to an explicit start-up step, degraded components should be reported visibly, and A2A traces should be persisted. Over-long modules need splitting, the remaining *any* types and inline styles need removing, and the out-of-date test needs repairing. The defects seen in Section 9.4 also need fixing: unavailable values must display correctly, routing to the agriculture agent must be tightened, and the committee's evidence pack must use the same live data as the agents. Table 10.2 gives an indicative schedule of about twelve weeks for this work.

Table 10.2: Proposed roadmap for future work

Weeks	Work package	Deliverable
1 to 2	Credibility fixes	Claim verification wired into synthesis; edge sign corrected; silent failures surfaced; failing test repaired.
3 to 6	Causal modelling	Edge statistics estimated from long series; local projection or BVAR scenario engine with back-tests.
5 to 8	Evaluation	Question set with gold answers; baselines and ablations; point-in-time committee benchmark.
9 to 12	Consolidation	Results analysis, hardening of endpoints, documentation and paper draft.

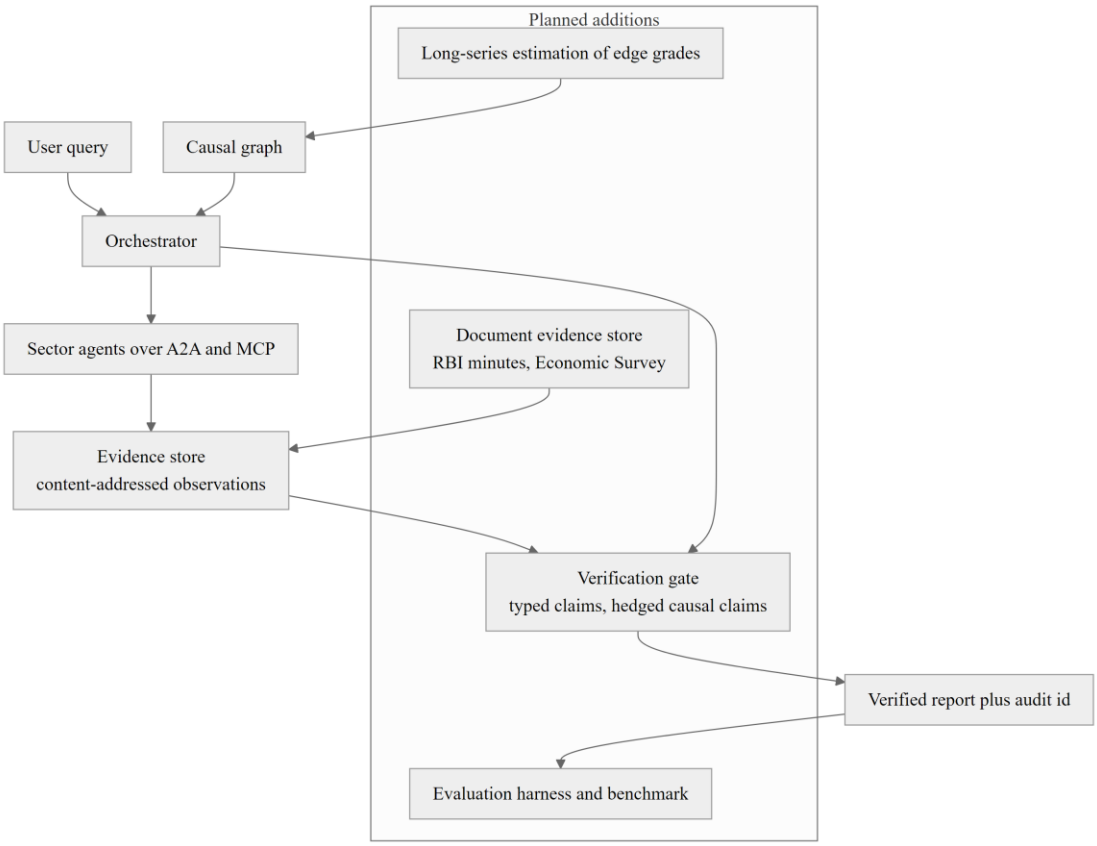


Figure 10.1: Target architecture after planned improvements.

10.5 CONCLUDING REMARKS

Macrograph AI shows that a multi-agent system for macroeconomic analysis can be built so that responsibility for each number lies with one agent, every number is accompanied by its source, and missing evidence is reported openly. It also shows, through its own audit, how much of the causal reasoning still rests on assumptions. Whether its analytical claims hold up will be decided by the estimation, verification and evaluation work planned above, for which the platform now provides the necessary groundwork.

REFERENCES

(All references are in APA format)

[1]. Anthropic. (2024, November 25). *Introducing the Model Context Protocol*. https://www.anthropic.com/news/model-context-protocol

[2]. Benjamini, Y., & Hochberg, Y. (1995). Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B (Methodological), 57*(1), 289–300.

[3]. Dadopoulos, M., Ladas, A., Moschidis, S., & Negkakis, I. (2025). *Metadata-driven retrieval-augmented generation for financial question answering*. arXiv. https://arxiv.org/abs/2510.24402

[4]. Dickey, D. A., & Fuller, W. A. (1979). Distribution of the estimators for autoregressive time series with a unit root. *Journal of the American Statistical Association, 74*(366), 427–431.

[5]. Google. (2025, April 9). *Announcing the Agent2Agent Protocol (A2A)*. Google Developers Blog. https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/

[6]. Granger, C. W. J. (1969). Investigating causal relations by econometric models and cross-spectral methods. *Econometrica, 37*(3), 424–438.

[7]. Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. In G. Varoquaux, T. Vaught, & J. Millman (Eds.), *Proceedings of the 7th Python in Science Conference* (pp. 11–15).

[8]. Hao, Y., & Xie, D. (2025). *A multi-LLM-agent-based framework for economic and public policy analysis*. arXiv. https://arxiv.org/abs/2502.16879

[9]. Hogan, A., Blomqvist, E., Cochez, M., d'Amato, C., de Melo, G., Gutierrez, C., Kirrane, S., Gayo, J. E. L., Navigli, R., Neumaier, S., Ngomo, A.-C. N., Polleres, A., Rashid, S. M., Rula, A., Schmelzeisen, L., Sequeda, J., Staab, S., & Zimmermann, A. (2021). Knowledge graphs. *ACM Computing Surveys, 54*(4), Article 71. https://doi.org/10.1145/3447772

[10]. Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., & Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys, 55*(12), Article 248. https://doi.org/10.1145/3571730

[11]. Jordà, Ò. (2005). Estimation and inference of impulse responses by local projections. *American Economic Review, 95*(1), 161–182.

[12]. Kato, M. (2026). *AI economist agent: An agentic framework for model-grounded economic analysis with RAG, knowledge graphs, and large language models*. arXiv. https://arxiv.org/abs/2606.20041

[13]. LangChain. (n.d.). *LangGraph documentation*. Retrieved October 12, 2026, from https://langchain-ai.github.io/langgraph/

[14]. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems, 33*, 9459–9474.

[15]. Lin, J., Sun, L., & Yan, Y. (2026). *Simulating macroeconomic expectations in survey experiments with LLM-based economic agents*. arXiv. https://arxiv.org/abs/2505.17648

[16]. Lowin, J. (n.d.). *FastMCP* [Computer software]. GitHub. https://github.com/jlowin/fastmcp

[17]. Ministry of Statistics and Programme Implementation. (n.d.). *e-Sankhyiki: MoSPI data and statistics services*. Government of India. https://mcp.mospi.gov.in/

[18]. Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology* (Article 2). ACM. https://doi.org/10.1145/3586183.3606763

[19]. Pearl, J. (2009). *Causality: Models, reasoning, and inference* (2nd ed.). Cambridge University Press.

[20]. Pydantic Services Inc. (n.d.). *Pydantic documentation*. Retrieved October 12, 2026, from https://docs.pydantic.dev/

[21]. Raasveldt, M., & Mühleisen, H. (2019). DuckDB: An embeddable analytical database. In *Proceedings of the 2019 International Conference on Management of Data* (pp. 1981–1984). ACM. https://doi.org/10.1145/3299869.3320212

[22]. Ramírez, S. (n.d.). *FastAPI* [Computer software]. https://fastapi.tiangolo.com/

[23]. Reserve Bank of India. (n.d.). *Database on Indian Economy (DBIE)*. https://data.rbi.org.in/

[24]. Seabold, S., & Perktold, J. (2010). Statsmodels: Econometric and statistical modeling with Python. In S. van der Walt & J. Millman (Eds.), *Proceedings of the 9th Python in Science Conference* (pp. 92–96).

[25]. Sims, C. A. (1980). Macroeconomics and reality. *Econometrica, 48*(1), 1–48.

[26]. Wooldridge, M., & Jennings, N. R. (1995). Intelligent agents: Theory and practice. *The Knowledge Engineering Review, 10*(2), 115–152.

[27]. Xiao, Y., Sun, E., Luo, D., & Wang, W. (2025). *TradingAgents: Multi-agents LLM financial trading framework*. arXiv. https://arxiv.org/abs/2412.20138

[28]. Yang, Y., Pang, Y., Huang, G., & E, W. (2020). *The knowledge graph for macroeconomic analysis with alternative big data* (SSRN Scholarly Paper No. 3707964). Social Science Research Network. https://ssrn.com/abstract=3707964

[29]. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations (ICLR 2023)*. https://arxiv.org/abs/2210.03629

[30]. Zheng, S., Trott, A., Srinivasa, S., Parkes, D. C., & Socher, R. (2022). The AI Economist: Taxation policy design via two-level deep multiagent reinforcement learning. *Science Advances, 8*(18), Article eabk2607. https://doi.org/10.1126/sciadv.abk2607

[31]. Zhu, X., Xie, Y., Liu, Y., Li, Y., & Hu, W. (2025). *Knowledge graph-guided retrieval augmented generation*. arXiv. https://arxiv.org/abs/2502.06864

APPENDICES

Appendices are provided to give supplementary information that is not included in the main text. They are numbered with Arabic numerals (Appendix 1, Appendix 2, and so on); tables, figures and listings in them carry the appendix number and are referred to from the main text.

APPENDIX 1: REPOSITORY STRUCTURE AND CONFIGURATION

Figure A1.1 shows the layout of the repository, and Table A1.1 lists the configuration variables read from *backend/.env*.

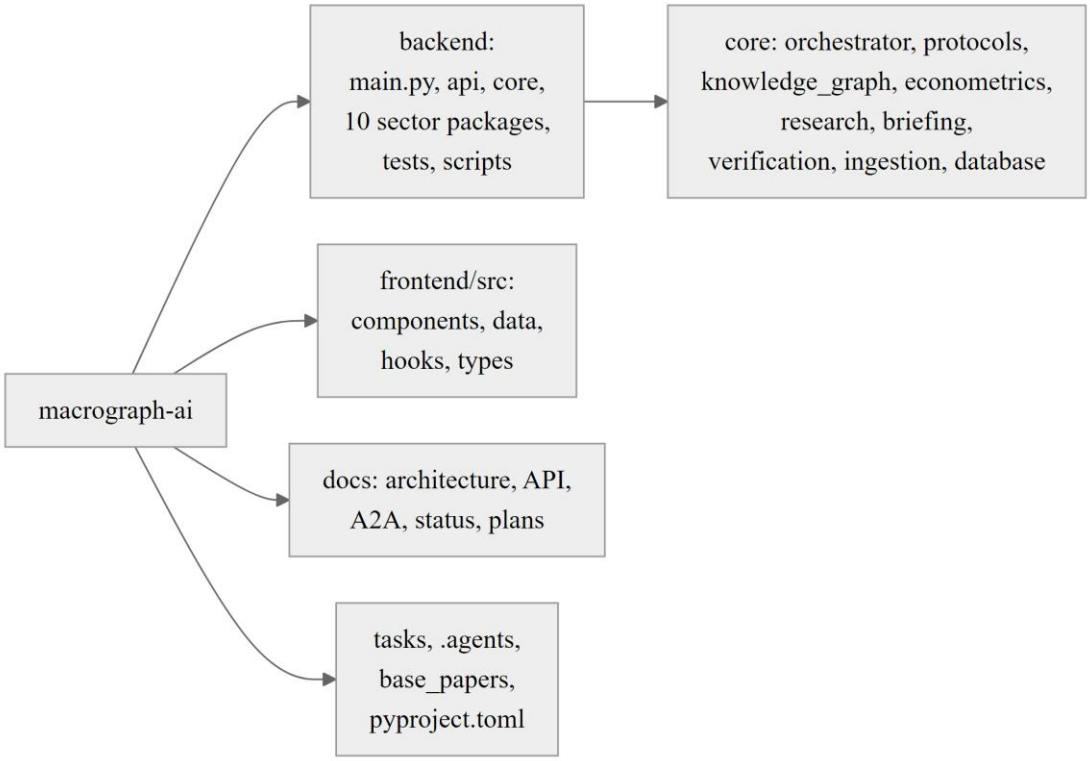


Figure A1.1: Repository layout of Macrograph AI.

Table A1.1: Principal configuration variables

Variable	Purpose
AGR_REAL_KEY, FIN_FIS_KEY, CAP_MON_KEY, PRIC_LAB_KEY, SERV_EXT_KEY	Language model keys, each shared by a pair of sector agents
ORCH_KEY	Language model key of the orchestrator
FAST_MODEL, REASONING_MODEL	Model identifiers for fast and reasoning calls (defaults openai/gpt-oss-20b and openai/gpt-oss-120b)
TVLY_KEY_1 to TVLY_KEY_3	Search keys for news retrieval
A2A_MAX_DEPTH, A2A_MAX_HOPS	Loop protection limits (defaults 5 and 25)
A2A_REQUEST_TIMEOUT, A2A_PEER_TIMEOUT	Timeouts in seconds (defaults 90 and 25)
A2A_API_KEY, A2A_ALLOW_ANONYMOUS_REQUESTS	Access control of the A2A HTTP interface
NEO4J_URI, NEO4J_USER, NEO4J_PASSWORD	Optional graph mirror (disabled when the URI is empty)
BRIEF_CRON_ENABLED	Enables the 08:15 daily brief scheduler
CORS_ORIGINS	Allowed browser origins (default local development origins)

APPENDIX 2: CANONICAL INDICATORS OF THE KNOWLEDGE GRAPH

Table A2.1 lists the twenty canonical indicators registered in the ontology on 12 October 2026.

Table A2.1: Canonical indicators of the ontology

Identifier	Name	Domain	Unit	Frequency
in.macro.real.gdp_growth	Quarterly real GDP growth rate	Real economy	% YoY	Quarterly
in.macro.real.iip_growth	Index of Industrial Production growth	Real economy	% YoY	Monthly
in.macro.real.gfcf_investment	Gross fixed capital formation	Real economy	% of GDP	Quarterly
in.macro.prices.cpi_headline	All India CPI combined headline inflation	Prices	% YoY	Monthly
in.macro.prices.cpi_food	Consumer food price index inflation	Prices	% YoY	Monthly
in.macro.prices.wpi_all	Wholesale price index, all commodities	Prices	% YoY	Monthly
in.macro.prices.brent_crude	Brent crude oil benchmark price	Prices	USD/barrel	Daily
in.macro.monetary.repo_rate	RBI policy repo rate	Monetary	% p.a.	Policy cycle
in.macro.monetary.bank_credit_growth	Bank credit growth	Monetary	% YoY	Fortnightly
in.macro.fiscal.debt_to_gdp	General government debt to GDP ratio	Fiscal	% of GDP	Annual
in.macro.fiscal.central_capex	Central government capital expenditure	Fiscal	INR crore	Monthly
in.macro.external.forex_reserves	Foreign exchange reserves of India	External	Billion USD	Weekly
in.macro.external.usd_inr	USD/INR exchange rate	External	INR/USD	Daily
in.macro.external.trade_balance	Merchandise trade balance	External	Billion USD	Monthly
in.macro.capmarkets.nifty_50	NIFTY 50 benchmark index	Capital markets	Points	Daily
in.macro.capmarkets.bank_nifty	NIFTY Bank index	Capital markets	Points	Daily
in.macro.capmarkets.india_vix	India VIX volatility index	Capital markets	Points	Daily
in.macro.agri.monsoon_departure	Southwest monsoon rainfall departure from LPA	Agriculture	% departure	Seasonal
in.macro.agri.foodgrain_production	Foodgrain production volume	Agriculture	Million tonnes	Annual
in.macro.labour.epfo_additions	EPFO monthly net payroll additions	Labour	Thousand	Monthly

APPENDIX 3: SAMPLE REPORT GENERATED BY THE ORCHESTRATOR

The following tables reproduce the structured sections of the report produced by the end-to-end run of Section 9.4 (query: how does a rise in crude oil prices affect inflation and RBI repo rate policy, with a Brent crude shock of plus 20 USD per barrel; deterministic mode, live official sources, retrieved on 12 October 2026). The executive synthesis of this run read: "LLM narrative unavailable; deterministic summary of retrieved evidence follows. Observations retrieved from sector agents: 3. Causal transmission paths mapped in the knowledge graph: 3. Scenario simulated: Crude +20 on in.macro.prices.brent_crude. Documentary evidence: unavailable (no document store is configured); no quotations are provided."

Table A3.1, Table A3.2 and Table A3.3 reproduce the structured tables of the report.

Table A3.1: Indicator matrix of the generated report

Indicator	Value	Unit	Period	Data Status	Provenance Hash
in.macro.monetary.repo_rate	5.25	%	Feb 2026	RBI DBIE	37e0e36c25c48aea
in.macro.monetary.m3_growth	None%	% YoY	2026-06-30	RBI DBIE	37e0e36c25c48aea
in.macro.prices.cpi_headline	5.54	% YoY	2026-09	official	fed3aaaa910e5b26

Table A3.2: Scenario table of the generated report (live baselines for CPI and repo rate)

Target Indicator	Baseline	Shocked Peak	Impact (Delta)	Transmission Lag	Confidence Band
Wholesale Price Index All Commodities	2.45	4.02	+1.57	1M	[3.63, 4.41]
Merchandise Trade Balance	-22.5	-24.02	-1.52	1M	[-24.4, -23.64]
All India CPI Combined Headline Inflation	5.54	6.32	+0.78	3M	[6.12, 6.52]
USD/INR Exchange Rate	86.4	85.68	-0.72	2M	[85.5, 85.86]
RBI Policy Repo Rate	5.25	5.66	+0.41	4M	[5.56, 5.76]
Bank Credit Growth	13.8	13.6	-0.20	7M	[13.55, 13.65]
NIFTY Bank Index	51500.0	51499.81	-0.19	5M	[51499.76, 51499.86]
Index of Industrial Production Growth	4.2	4.11	-0.09	9M	[4.09, 4.13]
Quarterly Real GDP Growth Rate	5.4	5.35	-0.05	10M	[5.34, 5.36]

Table A3.3: Source provenance table of the generated report

Agent	Source	Dataset / Table	Period	Retrieved
monetary_sector	Monetary & Liquidity Sector Macroeconomic A...	Monetary Sector Analysis	n/a	2026-10-12 18:48:45
prices_sector	National Statistical Office (NSO), MoSPI	CPI / CPI:2024:division=0:state=1:sector=3:...	2026-09	2026-10-12 18:48:49
agriculture_sector	Agriculture Agent	Agriculture findings	n/a	2026-10-12 18:48:42

APPENDIX 4: SELECTED SOURCE CODE LISTINGS

The listings below are extracted from the repository without modification (long lines may wrap): Listing A4.1, Listing A4.2, Listing A4.3 and Listing A4.4.

Listing A4.1: A2A request and response models (core/protocols/a2a/messages.py)

```
class A2ARequest(BaseModel):
    """One agent-to-agent task request."""
    model_config = ConfigDict(extra="forbid", str_strip_whitespace=True,
frozen=True)

    request_id: str = Field(default_factory=lambda: new_id("req"),
min_length=1)
    sender_agent: str = Field(..., min_length=1)
    receiver_agent: str = Field(..., min_length=1)
    task: str = Field(..., min_length=1)
    parameters: dict[str, Any] = Field(default_factory=dict)
    context: Optional[dict[str, Any]] = None
    conversation_id: str = Field(..., min_length=1)
    parent_request_id: Optional[str] = None
    depth: int = Field(default=0, ge=0)
    call_chain: tuple[str, ...] = Field(default_factory=tuple)
    timeout_seconds: Optional[float] = Field(default=None, gt=0)
    timestamp: datetime = Field(default_factory=_now)

    @model_validator(mode="after")
    def _no_self_parent(self) -> "A2ARequest":
        if self.parent_request_id == self.request_id:
            raise ValueError("parent_request_id must differ from
request_id")
        return self


class A2AResponse(BaseModel):
    """Result of an A2ARequest; failures are data, never raised across
agents."""
    model_config = _STRICT

    request_id: str
    sender_agent: str
    receiver_agent: str
    status: Literal["success", "partial", "failed"]
    result: Optional[dict[str, Any] | list[Any] | str] = None
    sources: list[SourceProvenance] = Field(default_factory=list)
    errors: list[A2AErrorInfo] = Field(default_factory=list)
    follow_ups: list[FollowUp] = Field(default_factory=list)
    metadata: dict[str, Any] = Field(default_factory=dict)
    timestamp: datetime = Field(default_factory=_now)

    @property
    def ok(self) -> bool:
        return self.status in ("success", "partial")

    @property
    def error_codes(self) -> list[ErrorCode]:
        return [e.code for e in self.errors]
```

Listing A4.2: Shortest transmission path query (core/knowledge_graph/networkx_engine.py)

```
    def get_shortest_transmission_path(self, source_id: str, target_id:
str) -> List[CausalRelationship]:
        """Finds the most direct transmission chain between two
macroeconomic indicators."""
```

```
        if not self.graph.has_node(source_id) or not
self.graph.has_node(target_id):
            return []

        try:
            path_nodes = nx.shortest_path(self.graph, source=source_id,
target=target_id)
        except (nx.NetworkXNoPath, nx.NodeNotFound):
            return []

        relationships: List[CausalRelationship] = []
        for u, v in zip(path_nodes[:-1], path_nodes[1:]):
            edge_data = self.graph.get_edge_data(u, v)
            if edge_data:
                # Take the highest confidence edge if multiple exist
                best_rel = max(edge_data.values(), key=lambda d:
d.get("confidence", 0.0))["relation"]
                relationships.append(best_rel)

        return relationships
```

Listing A4.3: Scenario propagation loop (core/econometrics/causal_engine.py)

```
        for impact in downstream_impacts:
            target_id = impact["target_indicator_id"]
            path =
self.graph.get_shortest_transmission_path(shock_variable, target_id)
            if not path:
                continue

            target_base = baselines.get(target_id, 5.0)
            # Calculate cumulative elasticity and attenuation
            cum_multiplier = 1.0
            for r in path:
                traversed_relations.append(r.relation_id)
                sign = 1.0 if r.elasticity_sign == "+" else -1.0
                # Attenuation coefficient per hop based on confidence
                decay = 0.55 * r.confidence_score
                cum_multiplier *= (sign * decay)

            delta = round(shock_magnitude * cum_multiplier * 0.15, 2)
            shocked_target = round(target_base + delta, 2)
            ci_low = round(shocked_target - abs(delta * 0.25), 2)
            ci_high = round(shocked_target + abs(delta * 0.25), 2)

            forecasts[target_id] = {
                "indicator_name": impact["target_name"],
                "sector": impact["sector"],
                "baseline": target_base,
                "shocked_peak": shocked_target,
                "delta": delta,
                "confidence_band": [ci_low, ci_high],
                "transmission_lag_months": impact["total_lag_months"],
                "confidence_score": impact["cumulative_confidence"],
                "mechanism_summary": " ->
".join(impact["transmission_chain"])
            }
```

Listing A4.4: Rolling z-score of the anomaly radar (core/research/anomaly.py)

```
def zscore(values: List[float]) -> Optional[float]:
    """z of the latest point against the trailing window, latest excluded
from its own baseline."""
    n = len(values)
    if n < MIN_N:
        return None
```

```
    window = min(WINDOW, n - 1)
    base = values[-window - 1:-1]
    mean = sum(base) / len(base)
    sd = (sum((x - mean) ** 2 for x in base) / len(base)) ** 0.5
    if sd == 0:
        return None
    return (values[-1] - mean) / sd
```

APPENDIX 5: ERROR TAXONOMY AND OPEN ISSUES

Figure A5.1 groups the A2A error codes. Table A5.1 lists the principal open issues identified in the review of 12 October 2026 and referred to in Chapters 9 and 10.

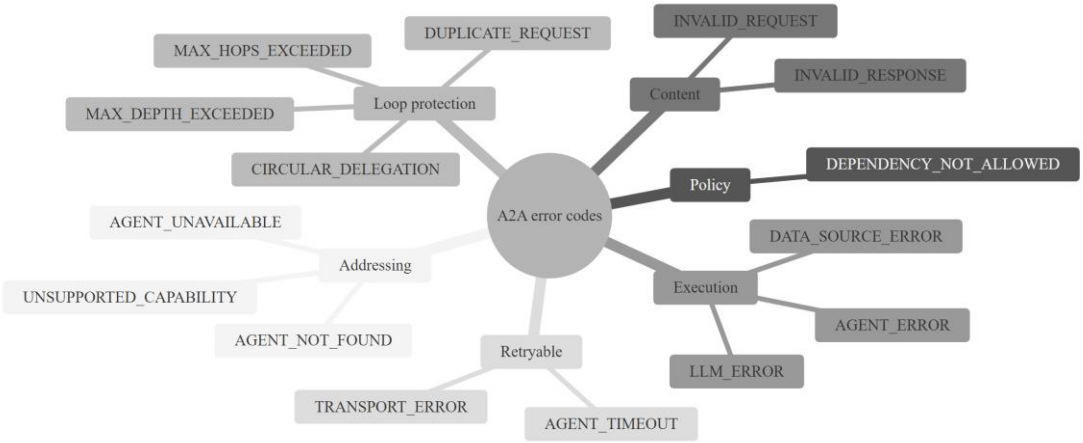


Figure A5.1: Taxonomy of A2A error codes.

Table A5.1: Principal open issues from the code review

Issue	Severity	Planned action
Claim verification library is not called by any answer path	High	Integrate into synthesis
Edge p-values and confidences are curated priors	High	Estimate from long series; audit
Scenario engine is heuristic, unit-blind, with fixed default baselines	High	Local projections or BVAR
Trade balance to USD/INR edge has a doubtful sign	High	Correct sign and test
Optional sector mounts and some start-up steps fail silently	High	Log and surface degradation
Edge audit returns an empty result on any exception	Medium	Surface the error
Constants duplicated in engine and research module	Medium	Single shared definition
Private language model client members used by callers	Medium	Public availability property
Granger audit lacks stationarity checks and multiple-testing control	Medium	Differencing, BH correction
Research and brief endpoints unauthenticated	Medium	Authentication and rate limits
Sector test directories missing from default test paths	Medium	Add to configuration; fix drifted test
Nine non-test modules exceed 400 lines; frontend has any types and inline styles	Low	Refactor